



MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Capstone Project Document

Applied AI for Farm - Smart Farm Management System

Group Name:	FA24AI11
Group Members:	Nguyễn Phan Nhat Minh SE171587 Nguyen Hoang Anh SE171292 Le Lam Nhat Linh SE162158
Supervisor:	Mr. Nguyen Quoc Trung Mr. Tran Thanh Nguyen
Capstone Project code:	FA24AI32

Ho Chi Minh, 12/2024

Acknowledgement

The completion of this capstone project would not have been possible without the invaluable support and guidance of many individuals and organizations. First and foremost, we would like to express our deepest gratitude to our supervisors, Mr. Nguyễn Quốc Trung and Mr. Trần Thanh Nguyên, for their unwavering support, expert guidance, and constructive feedback throughout the development of our Smart Farm Management System

We would like to extend our sincere appreciation to our review committee members: Mr. Huỳnh Văn Thống, Mr. Lê Thanh Hải, and Mr. Nguyễn Quốc Tiến, whose insightful comments and constructive criticism during the review sessions have significantly improved the quality of our project. Their expertise and thorough evaluation have helped us refine our approach and strengthen our implementation.

We are particularly grateful to FPT University for providing us with the educational foundation and resources necessary to undertake this challenging project. The university's commitment to fostering innovation and practical application of artificial intelligence has been instrumental in shaping our approach to this research.

Special thanks go to the farm owner and staff in Ho Chi Minh and Dong Nai who granted us access to their facilities for data collection and testing. Their cooperation and insights into real-world pig farming operations have been crucial in developing a practical and effective solution. This project represents not just our technical achievement, but also the collective effort and support of all these individuals and institutions.

Definition and Acronyms

Acronym	Definition	Acronym	Definition
AI	Artificial Intelligence	ML	Machine Learning
CV	Computer Vision	IoT	Internet of Things
CNN	Convolutional Neural Network	YOLO	You Only Look Once
ViT	Vision Transformer	NAS	Neural Architecture Search
ResNet	Residual Network	GAN	Generative Adversarial Network
GPU	Graphics Processing Unit	CPU	Central Processing Unit
RAM	Random Access Memory	API	Application Programming Interface
mAP	mean Average Precision	HOTA	Higher Order Tracking Accuracy
MOTA	Multi-Object Tracking Accuracy	FPS	Frames Per Second
LSTM	Long Short-Term Memory	GRU	Gated Recurrent Unit
TSM	Temporal Shift Module	F1 Score	Harmonic mean of Precision and Recall
MAE	Mean Absolute Error	R ²	Coefficient of Determination
SGD	Stochastic Gradient Descent	CSPDarknet	Cross Stage Partial Darknet
RTSP	Real Time Streaming Protocol	NVR	Network Video Recorder
PAN	Path Aggregation Network	SPP	Spatial Pyramid Pooling
TLS	Transport Layer Security	CUDA	Compute Unified Device Architecture
FLOPS	Floating Point Operations Per Second	CI/CD	Continuous Integration/-Continuous Deployment
IoU	Intersection over Union	MSE	Mean Squared Error
LR	Learning Rate	AES	Advanced Encryption Standard
GDPR	General Data Protection Regulation	ECB	Efficient Convolutional Block
PGI	Programmable Gradient Information	ROI	Region of Interest

Table 1: List of Definitions and Acronyms

Contents

1	Project Introduction	10
1.1	Overview	10
1.1.1	Project Information	10
1.1.2	Project Overview	10
1.2	Project Background	11
1.2.1	Current Challenges in Traditional Farming	11
1.2.2	Scale and Complexity	12
1.2.3	Technological Advancements	12
1.2.4	Economic and Industry Context	12
1.2.5	Vietnam's Agricultural Modernization	12
1.3	Project Objective	12
1.4	Problem Statement	13
1.4.1	Population Monitoring Challenges	13
1.4.2	Individual Tracking and Movement Analysis	14
1.4.3	Weight Estimation Difficulties	14
1.4.4	Behavior Classification Limitations	14
1.4.5	Data Integration and Analysis	14
1.5	Significance of the Project	15
1.6	Project Scope & Limitations	16
1.6.1	Project Scope	16
1.6.2	Project Limitations	17
1.6.3	Out of Scope	17
1.7	Team Work	18
1.7.1	Team Structure and Roles	18
1.7.2	Communication Plan	18
1.8	Project Management Approach	18
1.8.1	Project Overview	18
1.8.2	Project Organization	19
1.8.3	Component Integration Strategy	19
1.8.4	Quality Management	20
1.8.5	Risk Management	20
1.8.6	Progress Monitoring and Control	20
2	Existing Systems/State of the Art	21
2.1	Overview of the Field	21
2.2	Historical Context	22
2.3	Key Studies and Theories	23
2.3.1	Object Detection and Tracking Systems	23
2.3.2	Behavior Analysis and Recognition	23
2.3.3	Weight Estimation Methods	23

2.3.4	Disease Detection and Health Monitoring	23
2.3.5	Theoretical Foundations and Frameworks	24
2.4	Technological Advancements	24
2.4.1	Evolution of Detection and Tracking Systems	24
2.4.2	Advancement in Deep Learning Architectures	25
2.4.3	Edge Computing Integration	25
2.4.4	Multi-Modal Sensing and Data Fusion	25
2.4.5	Automated Behavior Analysis Systems	25
2.4.6	Weight Estimation and Health Monitoring	25
2.5	Comparison of Existing Systems	26
2.5.1	Commercial Vision-Based Systems	26
2.5.2	Cost and Implementation Considerations	28
2.6	Gaps in the Literature/Technology	28
2.6.1	Technical Limitations	28
2.6.2	Processing and Implementation Challenges	28
2.6.3	Behavioral Analysis Limitations	29
2.6.4	Weight Estimation Accuracy	29
2.6.5	Disease Detection Capabilities	29
2.6.6	Integration and Standardization	29
2.6.7	Cost and Accessibility	29
2.6.8	Research Opportunities	29
2.7	Justification for the Project	30
2.7.1	Educational and Research Value	30
2.7.2	Technical Exploration	30
2.7.3	Cost-Effective Alternative	30
2.7.4	Learning Through Implementation	30
2.7.5	Foundation for Future Research	31
3	Methodology	32
3.1	Research Questions and Objectives	32
3.1.1	Research Questions	32
3.1.2	Technical Objectives	32
3.1.3	Expected Outcomes	33
3.2	Data Collection and Preprocessing	33
3.2.1	Detection Dataset	33
3.2.2	Behavior Analysis Dataset	34
3.2.3	Health Monitoring Dataset	34
3.2.4	Weight Estimation Dataset	35
3.2.5	Data Preprocessing Methodology	36
3.2.6	Dataset Statistics and Distribution	39
3.3	Feature Selection and Engineering	39
3.3.1	Computer Vision Feature Engineering	39
3.3.2	Feature Integration and System Components	46
3.3.3	Pig ID Management System Implementation	49
3.3.4	Escape Detection System Implementation	50
3.3.5	Behavior Analysis System Implementation	51
3.3.6	Weight Estimation	53
3.4	Model Architectures	65
3.4.1	Detection Architecture	65
3.4.2	Vision Transformer	69

3.4.3	Architecture of YOLOv11	70
3.4.4	ResNet50v2 Architecture	72
3.5	Model Training and Validation	75
3.5.1	Training Infrastructure	75
3.5.2	Detection Model Baseline Training Methodology	75
3.5.3	Model Fine-tuning Strategy	79
3.5.4	ViT	81
3.5.5	Weight Estimation with Segment and Keypoint Detection	83
3.5.6	Health classify ResNet50v2	85
3.6	Evaluation Metrics	85
3.6.1	Detection and Tracking Metrics	85
3.6.2	Behavior Analysis Metrics	87
3.6.3	Weight Estimation Metrics	88
3.6.4	Disease Detection Metrics	88
4	System Design and Implementation	89
4.1	System Architecture	89
4.2	Component Design	89
4.2.1	Hardware Components	89
4.2.2	Processing Pipeline Components	91
4.3	AI Model Integration	91
4.3.1	Detection and Tracking	91
4.3.2	Analysis Modules	91
4.4	Real-time Processing Flow	91
4.4.1	Frame Acquisition	91
4.4.2	Detection and Tracking	91
4.4.3	Parallel Analysis	92
4.4.4	Result Integration	92
4.5	Interface Design	92
4.6	Testing and Quality Assurance	92
4.6.1	Performance Testing	92
4.6.2	Reliability Testing	92
4.6.3	Accuracy Validation	92
4.7	Deployment Strategy	93
4.7.1	API	93
4.7.2	Hardware Setup	94
4.7.3	System Initialization	94
4.7.4	Performance Monitoring	94
4.7.5	Maintenance Procedures	94
5	Results and Discussion	95
5.1	Model Performance Evaluation	95
5.2	Real-world Detection and Tracking Analysis	95
5.3	Behavior Classification Analysis	95
5.4	Aggressive Behavior Classification Results	97
5.4.1	Model Performance Analysis	97
5.4.2	Feature Analysis	98
5.4.3	Performance Limitations	98
5.4.4	Real-world Application Impact	99
5.5	Weight Estimation Results	99

5.5.1	Data Preprocessing in Weight Estimation	99
5.6	Health Classification	104
5.6.1	Accuracy	104
5.6.2	Loss metric	104
5.6.3	Confusion Matrix and Metrics	105
6	Conclusion	108
6.1	Project Summary	108
6.2	Key Contributions	108
6.3	Future Work	109
6.3.1	Enhancing Motion Analysis for Behavioral Recognition	109
6.3.2	Improving ViT for Aggressive Behavior Detection	109
6.3.3	Advanced Weight Estimation Techniques	109
6.3.4	System Scalability and Multi-Stream Processing	109
6.3.5	Integration with IoT Systems	109
6.3.6	Cross-Species Adaptation	109
6.3.7	Economic and Environmental Benefits Analysis	110
6.4	Closing Remarks	110
7	Appendices	111
7.1	Appendix: Code Snippets	111
7.1.1	DetectBon and Tracking Pipeline	111
7.1.2	Behavior Analysis Algorithms	112
7.1.3	Motion Analysis with Kalman Filtering	113
7.1.4	Escape Detection System	114
8	References	116

List of Tables

1	List of Definitions and Acronyms	2
2.1	Comprehensive Feature Comparison of Commercial Pig Farming Systems	27
3.1	Detection and Tracking Dataset Distribution	39
3.2	Aggressive Behavior Dataset Distribution	39
3.3	Health Monitoring Dataset Distribution	39
3.4	Weight Estimation Dataset Distribution	40
3.5	Dimensional progression through ResNet50v2 layers	75
3.6	Training Hardware Specifications	75
3.7	Model Architecture Comparison	75
3.8	Training Hyperparameters	76
3.9	Model-Specific Optimization Techniques	76
3.10	Training Pipeline Features	76
3.11	Model Validation Metrics	77
3.12	Data Augmentation Strategies	77
3.13	Memory Management Strategies	77
3.14	Model Training Time Analysis	78
3.16	Baseline Models Performance Comparison	78
3.19	Fine-tuning Hyperparameters	79
3.20	Layer Freezing Strategy	79
3.21	Fine-tuning Augmentation Strategy	80
3.22	Fine-tuning Performance Comparison	80
3.23	Convergence Analysis	80
3.24	Resource Utilization During Fine-tuning	81
3.25	Video Processing Pipeline Components	81
3.26	Detailed Configuration Parameters	82
3.27	Initialization and Optimization Strategy	82
3.28	Training Monitoring Framework	83
3.29	Comparison of YOLO segmentation models.	84
3.30	Comparison of YOLO keypoint detection models.	84
3.31	Augmentation techniques applied to the dataset.	85
3.32	Model training configuration details.	85
3.33	Dataset Preparation and Augmentation Details	86
3.34	Training configuration details.	86
3.35	Training and Validation Strategy Details	87
5.1	Vision Transformer Performance Metrics	98
5.2	Performance metrics comparison for YOLO models.	99
5.3	Performance metrics comparison for YOLO models.	101
5.4	Performance Metrics for Healthy and Sick Pigs	106

List of Figures

1.1	System Architecture Overview	11
3.1	Sample annotations demonstrating bounding box placement for pig detection . .	34
3.2	Composition and examples of aggressive and non-aggressive behaviors in data[5].	35
3.3	Electronic weighing system used for ground truth measurements	37
3.4	Aisle configuration showing camera placement and pig movement path	38
3.5	ByteTrack architecture showing the two-stage association process and track management system. The architecture employs high-confidence detection association followed by secondary matching for low-confidence detections.	41
3.6	Behavior Pattern Features	45
3.7	ID Management System	49
3.8	Demonstration of Weight Estimation Flow. The process flow shows: (left) input images of pigs in different poses, (center) YOLOv11 processing with segmentation masks highlighted in blue, and (right) the resulting individual mask outputs. This illustrates the model's ability to accurately segment individual pigs even in varying positions and lighting conditions.	53
3.9	Distribution of Laplacian function value over dataset	55
3.10	Value of Qbeta over beta through dataset	56
3.11	Demonstration of YOLOv11's instance segmentation pipeline on pig monitoring. The process flow shows: (left) input images of pigs in different poses, (center) YOLOv11 processing with segmentation masks highlighted in blue, and (right) the resulting individual mask outputs. This illustrates the model's ability to accurately segment individual pigs even in varying positions and lighting conditions.	57
3.12	YOLOv11 keypoint detection framework applied to pig monitoring with anatomical landmark identification. The diagram illustrates the model's capability to detect five key anatomical points (head, neck, back, rump, and tail) marked in blue. The bidirectional arrows indicate the model's filtering process to remove images with missing keypoints, ensuring data quality for subsequent analysis. . .	58
3.13	Plot of the tangent function showing it is always increasing.	58
3.14	Illustration of the coordinate system with angles and points. The center is at (cx, cy) , and the points $(px1, py1)$, $(px2, py2)$, $(px3, py3)$, $(px4, py4)$ are represented in quadrants I, II, III, and IV, respectively.	59
3.15	Diagram showing points A, B, C, D, E, and F	60
3.16	Neural Network Architecture with Input, Hidden Layer, and Output Layer . . .	64
3.17	YOLOv8 Architecture Overview showing the backbone, neck, and head components. Diagram based in [11]	66
3.18	YOLOv9 Architecture highlighting the PGI mechanism and enhanced feature processing [12]	68

3.19	YOLO-NAS architecture. The architecture is found automatically via a neural architecture search (NAS) system called AutoNAC to balance latency vs. throughput. They generated three architectures called YOLO-NASS (small), YOLO-NASM (medium), and YOLO-NASL (large), varying the depth and positions of the QSP and QCI blocks. The figure shows the YOLO-NASL architecture. Diagram based in [11]	69
3.20	Architecture of our Vision Transformer model for video-based aggressive behavior classification. The model processes video frames by splitting them into patches, applying transformer encoding with self-attention, and using temporal mean pooling for final classification.	70
3.21	YOLOv11 Architecture Overview showing the backbone, neck, and head components.	71
3.22	ResNet50v2 architecture overview showing the progression of convolutional layers and residual blocks	72
4.1	AI Pipeline Flow diagram to show real-time processing	90
5.1	Oblique camera view showing detection and behavior classification across two pens. Green boxes indicate moving pigs, red boxes show stationary pigs, and cyan boxes represent feeding behavior. The system maintains individual tracking IDs while handling perspective distortion and occlusion challenges.	96
5.2	Overhead camera view demonstrating perfect detection accuracy and tracking stability. The simplified perspective enables reliable individual tracking and precise behavior classification without occlusion challenges.	97
5.3	ViT attention patterns during aggressive behavior detection.	98
5.4	Segmentation performance comparison for YOLO models.	99
5.5	Segmentation result	100
5.6	Keypoint detection performance comparison across YOLO models.	100
5.7	Pose result	101
5.8	Single-pig weight estimation demonstrating successful individual segmentation and weight prediction.	102
5.9	Multi-pig weight estimation demonstrating successful individual segmentation and weight prediction. The system maintains accurate tracking and weight estimation even with multiple subjects in close proximity.	103
5.10	Model training accuracy with 150 epochs	104
5.11	Binary cross-entropy loss with 150 epochs	105
5.12	Confusion Matrix of Health and sick	105
5.13	Confidence bar chart comparing the probabilities for Healthy and Sick classes. .	107

Chapter 1

Project Introduction

1.1 Overview

1.1.1 Project Information

- **Project Name:** Applied AI for Farm - Smart Farm Management System
- **Vietnamese Name:** Ứng dụng AI trong chăn nuôi - Hệ thống Quản lý Trang trại Thông minh
- **Duration:** 09/2024 - 12/2024
- **Group Name:** FA24AI32
- **Team Members:**
 - Nguyen Phan Nhat Minh (SE171387) - Team Leader
 - Nguyen Hoang Anh (SE171292) - Member
 - Le Lam Nhat Linh (SE162158) - Member
- **Supervisors:**
 - Mr. Nguyen Quoc Trung (TrungNQ46@fe.edu.vn)
 - Mr. Tran Thanh Nguyen (NguyenTT15@fe.edu.vn)

1.1.2 Project Overview

In modern agriculture, efficient livestock management is crucial for maintaining productivity and ensuring animal welfare. Traditional pig farming practices rely heavily on manual monitoring and supervision, which not only consume significant time and labor but also pose challenges in maintaining consistent oversight of animal behavior, health, and development. This project introduces an Smart Farm Management System that leverages artificial intelligence and computer vision technologies to transform conventional pig farming practices.

The system addresses critical aspects of pig farm management through automated monitoring and analysis:

- Real-time pig detection and tracking for accurate population counting
- Behavioral analysis for identifying movement patterns, feeding behaviors, and social interactions

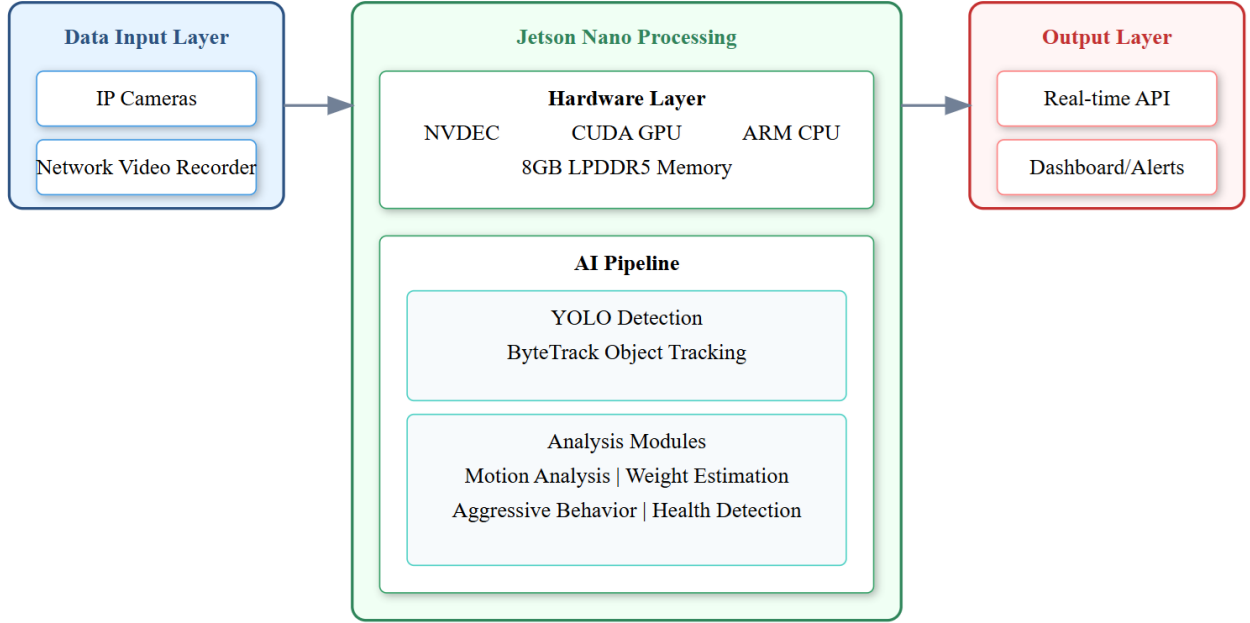


Figure 1.1: System Architecture Overview

- Visual weight estimation for growth monitoring without physical measurements
- Early warning system for detecting abnormal or aggressive behaviors

The technical implementation combines state-of-the-art deep learning models optimized for edge computing devices. Using Jetson Nano hardware, the system provides real-time processing capabilities directly at the farm site, minimizing latency and reducing dependency on internet connectivity. This approach ensures reliable operation in real-world farm environments.

Our solution is designed with scalability and adaptability in mind, capable of operating across various farm layouts and environmental conditions. Through a well-defined API, the system integrates easily with existing farm management platforms, enabling seamless data exchange and incorporation into broader agricultural management systems.

Through this Smart Farm Management System, we demonstrate the practical application of modern AI technologies in enhancing agricultural practices, improving animal welfare, and increasing operational efficiency in pig farming operations.

1.2 Project Background

The pig farming industry in Vietnam and globally has undergone significant transformations in recent years, driven by increasing demands for pork products and the need for more efficient, sustainable farming practices. Traditional pig farming methods, which rely heavily on manual observation and experience-based management, face numerous challenges in meeting these modern demands while maintaining high standards of animal welfare and operational efficiency.

1.2.1 Current Challenges in Traditional Farming

In traditional farm settings, monitoring pig health, behavior, and growth requires constant human presence and manual documentation. Farm workers must physically inspect animals multiple times daily to assess their condition, count the population, estimate weights, and identify any health or behavioral issues. This manual approach is not only labor-intensive

but also prone to human error and inconsistency. Furthermore, the limited ability to monitor animals during night hours or in all areas of large facilities creates significant gaps in surveillance and care.

1.2.2 Scale and Complexity

The increasing scale of modern pig farming operations compounds these challenges. Large-scale farms can house thousands of pigs, making individual monitoring and accurate record-keeping extremely difficult through traditional methods. The risk of missing early signs of health issues or aggressive behaviors increases with scale, potentially leading to significant economic losses and animal welfare concerns. Additionally, the growing pressure to optimize feed efficiency and maintain consistent growth rates requires more precise and continuous monitoring than manual methods can provide.

1.2.3 Technological Advancements

Recent advancements in artificial intelligence, particularly in computer vision and deep learning, have opened new possibilities for addressing these challenges. The success of AI applications in various industrial settings has demonstrated the potential for similar technologies to revolutionize livestock management. Computer vision systems have shown remarkable capabilities in object detection, tracking, and behavior analysis, making them ideal for automated livestock monitoring.

1.2.4 Economic and Industry Context

The pig farming industry faces increasing pressure to improve efficiency while reducing operational costs. Labor shortages in the agricultural sector and rising wages make manual monitoring increasingly expensive and unsustainable. Meanwhile, the growing emphasis on animal welfare and the need for detailed documentation of farming practices for regulatory compliance create additional demands that are difficult to meet with traditional methods.

1.2.5 Vietnam's Agricultural Modernization

In Vietnam specifically, the modernization of the agricultural sector is a key priority in the country's economic development strategy. The government's push for smart agriculture adoption, combined with the country's significant role in pork production, creates an opportune environment for implementing innovative farming solutions. Several pilot projects using smart technologies in livestock farming have already demonstrated promising results, indicating a growing readiness for more comprehensive AI-based management systems.

This background sets the stage for our Smart Farm Management System, which aims to address these challenges through the application of modern technology. By automating crucial monitoring tasks and providing data-driven insights, our system represents a timely solution to the evolving needs of the pig farming industry, bridging the gap between traditional farming practices and the demands of modern agricultural production.

1.3 Project Objective

The main objectives of this project include:

- Develop and optimize training and testing environments for AI models

- Enhance existing methodologies in object detection, tracking, and behavior classification, Weight estimation, Health classify for livestock
- Create a versatile and resilient system capable of adapting to diverse farm environments

To develop a smart farm management system that operates reliably across different pig farming settings, we have formulated a comprehensive approach to achieve our desired outcomes. The process begins with extensive data collection at a pig farm in Ho Chi Minh City and Dong Nai, where we install sophisticated camera systems to capture real-world data. This approach ensures the collection of diverse datasets featuring pigs in authentic farm environments, including various lighting conditions, pig ages, and behaviors. The on-site data collection provides a rich and relevant dataset for training and testing our AI models, effectively capturing the specific characteristics and challenges of local pig farming operations.

Our methodology involves carefully breaking down complex challenges into manageable components, focusing on key aspects such as detection, tracking, counting, weight estimation, and behavior classification. We devise specific strategies for each component, often employing multiple complementary models to achieve a holistic solution. This systematic approach allows us to address each challenge effectively while ensuring seamless integration of all components.

Model customization forms a crucial part of our development process. We refine and optimize our models to suit various farm environments through careful architecture adjustments and the integration of supporting structures. This optimization process significantly improves their applicability, particularly in scenarios where data availability might be limited. Our focus on customization ensures that the models can perform effectively across different farm layouts and environmental conditions.

For real-world implementation, we deploy our models on Jetson Nano devices, enabling real-time processing of live video feeds directly on-site. This edge computing approach ensures swift and accurate pig monitoring and behavior analysis without relying on cloud infrastructure. The on-site processing capability significantly reduces latency and enhances system reliability in practical farm settings.

Performance evaluation constitutes the final critical component of our approach. We rigorously test the system’s performance using both recorded datasets and real-time scenarios to ensure its reliability and adaptability. Our evaluation process employs detailed metrics including accuracy measurements, real-time processing speed assessments, and adaptability tests across diverse farm conditions. This comprehensive evaluation helps validate the system’s effectiveness and identifies areas for potential improvement.

1.4 Problem Statement

The manual management of pig farms presents several critical challenges that significantly impact operational efficiency, animal welfare, and economic sustainability. Through extensive research and consultation with farm operators, we have identified five key interconnected problems that require urgent technological intervention.

1.4.1 Population Monitoring Challenges

The first critical challenge lies in the accurate and continuous monitoring of pig populations within farm facilities. Traditional manual counting methods are highly time-consuming and prone to errors, particularly in large-scale operations where pigs are constantly moving and redistributing themselves across different areas. The inability to maintain accurate, real-time population counts affects inventory management, feed allocation, and overall farm planning.

This problem is exacerbated during night hours or in poorly lit conditions, creating significant gaps in monitoring coverage.

1.4.2 Individual Tracking and Movement Analysis

The second significant issue concerns the tracking of individual pig movements and behaviors. Without automated tracking systems, it is virtually impossible to monitor individual animals consistently over time. This limitation prevents farmers from identifying early signs of health issues or behavioral problems that might be indicated by changes in movement patterns. The lack of continuous behavioral monitoring also makes it difficult to optimize pen layouts and understand group dynamics that could affect pig welfare and productivity.

1.4.3 Weight Estimation Difficulties

Weight estimation presents the third major challenge. Traditional methods require physical handling of pigs for weighing, which is not only stressful for the animals but also labor-intensive and time-consuming. The inability to perform frequent weight measurements limits farmers' ability to optimize feed programs and monitor growth rates effectively. This results in suboptimal feed utilization and potential delays in identifying growth-related issues.

1.4.4 Behavior Classification Limitations

The fourth problem involves the detection and classification of pig behaviors, particularly concerning feeding patterns and aggressive interactions. Manual observation cannot provide continuous monitoring of pig behavior, making it difficult to identify and prevent aggressive encounters that could lead to injuries. Similarly, monitoring feeding behaviors and patterns is crucial for optimizing feed schedules and identifying health issues early, but this is impractical to accomplish through human observation alone.

1.4.5 Data Integration and Analysis

Finally, there is a significant challenge in data integration and analysis. Even when observations are made, the manual recording and analysis of pig behavior, growth, and health data is cumbersome and often delayed. This lag in data processing prevents farmers from making timely, data-driven decisions about farm management. The lack of systematic data collection also makes it difficult to identify long-term patterns or trends that could inform improvements in farm operations.

These problems are further complicated by several factors:

- The 24/7 nature of farming operations requires constant monitoring that exceeds human capabilities
- Environmental conditions such as lighting variations and dust affect visibility and monitoring accuracy
- The dynamic nature of pig behavior requires rapid response times to prevent adverse events
- Labor costs and availability make intensive manual monitoring economically unfeasible
- The scale of modern farming operations magnifies the impact of these challenges

Our Smart Farm Management System aims to address these specific problems through an integrated approach that combines advanced computer vision, machine learning, and real-time monitoring capabilities. By tackling these core issues, we seek to transform pig farm management from a reactive, labor-intensive process to a proactive, data-driven operation that enhances both productivity and animal welfare.

1.5 Significance of the Project

The Smart Farm Management System represents a significant advancement in agricultural technology, offering far-reaching benefits that extend beyond immediate operational improvements to impact the broader livestock farming industry, economic sustainability, and social welfare.

From an operational perspective, the implementation of AI-powered monitoring systems fundamentally transforms pig farm management practices. The continuous, automated surveillance capability significantly reduces the reliance on manual labor for routine monitoring tasks, allowing farm staff to focus on more strategic aspects of farm management. This shift not only improves operational efficiency but also creates opportunities for workforce development and skill enhancement in modern agricultural technologies. The system's ability to provide real-time monitoring and instant alerts enables faster response times to critical situations, potentially preventing significant losses and ensuring optimal farm performance.

The economic significance of this project is particularly noteworthy in the context of Vietnam's pork industry, which accounts for a substantial portion of the country's agricultural GDP. By implementing automated monitoring and analysis systems, farms can achieve significant cost reductions in several areas. Labor costs are optimized through automated surveillance, while improved feeding efficiency and growth monitoring lead to better feed utilization ratios. Early detection of health issues and behavioral problems helps prevent costly veterinary interventions and reduces mortality rates. These improvements in operational efficiency directly translate to enhanced profitability and economic sustainability for farm operators.

From an animal welfare perspective, the project makes substantial contributions to improving the quality of life for livestock. The system's ability to continuously monitor and analyze pig behavior enables early detection of stress, aggression, or health issues, allowing for prompt intervention. This proactive approach to animal care not only meets growing consumer demands for ethical farming practices but also aligns with increasingly stringent regulatory requirements for animal welfare. The systematic collection of behavioral data also contributes to better understanding of pig behavior patterns, potentially leading to improvements in housing design and management practices.

The project's significance extends to environmental sustainability as well. Through optimized feed management and improved resource allocation, the system helps reduce waste and improve resource efficiency. Better monitoring of pig growth and development enables more precise feed allocation, reducing overfeeding and associated environmental impacts. This alignment with sustainable farming practices positions the technology as a valuable tool in the industry's efforts to reduce its environmental footprint.

In the broader context of agricultural innovation, this project serves as a practical demonstration of how advanced technologies can be successfully integrated into traditional farming practices. It provides a replicable model for the modernization of livestock farming, potentially influencing the adoption of similar technologies across the agricultural sector. The project's success could accelerate the digital transformation of agriculture in Vietnam and similar developing economies, contributing to the overall modernization of the farming industry.

The research and development aspects of the project also hold significant academic value. The integration of computer vision, machine learning, and edge computing in a practical agricultural application provides valuable insights for future technological developments in the field.

The methodologies developed and lessons learned contribute to the growing body of knowledge in agricultural technology, potentially inspiring further innovations in smart farming solutions.

Furthermore, the project addresses critical food security concerns by enhancing the efficiency and reliability of pork production. In an era of increasing global food demand, improving the productivity and sustainability of livestock farming is crucial. The system's ability to optimize farm operations and reduce losses contributes to more stable and efficient food production systems, supporting food security objectives at both national and regional levels.

The socioeconomic impact of the project is particularly relevant for rural communities where pig farming is a significant source of income. By making pig farming more efficient and profitable, the technology helps sustain rural livelihoods while promoting the modernization of agricultural practices. This balance between technological advancement and economic development is crucial for the sustainable growth of the agricultural sector.

1.6 Project Scope & Limitations

This section clearly defines the boundaries of our Smart Farm Management System, outlining both the specific areas of focus and the acknowledged limitations of the current implementation. Understanding these parameters is crucial for managing expectations and ensuring effective project delivery.

1.6.1 Project Scope

The core scope of our Smart Farm Management System encompasses several key functional areas. In terms of pig detection and tracking, the system is designed to operate in standard indoor pig farm environments, processing video feeds from overhead cameras to maintain continuous monitoring of pig populations. The system specifically focuses on finishing farms, where pigs are in their growth phase prior to market, typically ranging from 30 to 100 kilograms in weight.

The behavior analysis component of our system concentrates on identifying and classifying three primary behavioral patterns: movement activities (moving, stationary, lying), feeding behaviors, and aggressive interactions. This classification is designed to operate in real-time, providing immediate alerts for aggressive behaviors that require intervention. The system maintains a rolling 24-hour history of behavioral patterns for trend analysis and reporting.

For weight estimation functionality, our system implements vision-based techniques to approximate pig weights without physical intervention. The system is calibrated to provide estimates within a reasonable margin of error, focusing on tracking growth trends rather than replacing precise physical measurements entirely. This feature operates on individual pigs when they are clearly visible in the camera's field of view.

The technical implementation scope includes:

- Deployment on Jetson Nano edge computing devices for real-time processing
- Integration with standard IP cameras with minimum 720p resolution
- Local data storage and processing with periodic cloud backup
- Basic REST API for data access and system configuration
- Web-based dashboard for monitoring and alert management

1.6.2 Project Limitations

While our system provides comprehensive monitoring capabilities, it is important to acknowledge its current limitations:

Technical Limitations: The system's performance is constrained by several technical factors. The accuracy of pig detection and tracking may be affected by poor lighting conditions, particularly during extreme variations in natural light or in areas with inadequate artificial lighting. The system's ability to maintain individual pig tracking is limited in situations of dense crowding or when pigs are in physical contact, potentially leading to temporary tracking losses.

Environmental Constraints: The current implementation is optimized for indoor environments with controlled lighting conditions. The system's performance may be compromised in outdoor areas or in facilities with significant environmental variations. Factors such as dust, moisture, and varying light conditions can impact the system's reliability and accuracy.

Behavioral Analysis Limitations: While the system can identify and classify basic behavioral patterns, it may not capture all nuanced interactions between pigs. Complex social behaviors or subtle signs of health issues might not be detected. The accuracy of behavior classification may decrease in situations where multiple behaviors occur simultaneously or when pigs are partially obscured from view.

Weight Estimation Constraints: The vision-based weight estimation feature provides approximate values and should not be considered a replacement for physical weighing when precise measurements are required. The accuracy of weight estimates may vary based on the pig's positioning, camera angle, and environmental conditions.

Processing Limitations: The real-time processing capabilities are constrained by the computational power of the Jetson Nano device. While sufficient for standard monitoring tasks, the system may experience latency when processing multiple high-resolution video streams simultaneously or during periods of intense activity.

Scalability Constraints: The current implementation is designed and tested for facilities housing up to 70 pigs per monitored area. Scaling beyond this capacity would require additional hardware resources and potential architectural adjustments to maintain real-time performance.

1.6.3 Out of Scope

The following aspects are explicitly outside the scope of the current project:

- Integration with automated feeding systems or climate control equipment
- Genetic or breeding analysis
- Detailed health diagnostics or disease identification
- Financial management and cost analysis features
- Integration with external farm management systems beyond basic API access
- Support for other livestock types or farming operations

These scope definitions and limitations serve as a framework for project development and deployment, ensuring that resources are focused on delivering core functionality while maintaining realistic expectations about system capabilities. Future iterations of the system may address some of these limitations through enhanced hardware capabilities and improved algorithms.

1.7 Team Work

1.7.1 Team Structure and Roles

The project team consists of three members, each taking on different roles based on their knowledge and skills. Nguyen Phan Nhat Minh (SE171387) serves as the team leader, responsible for project management and coordinating the team's overall activities. Minh's main tasks include coordinating with the farm for data collection, supervising dataset development and labeling processes, and developing pig disease classification models. Additionally, Minh is responsible for system integration and ensuring all components work synchronously.

Nguyen Hoang Anh (SE171292) takes on the specialized role of system optimization and weight estimation algorithm development. Hoang Anh focuses on deploying the system on the Jetson Nano platform, developing APIs, and optimizing real-time processing performance. With his knowledge of embedded systems, Hoang Anh is responsible for ensuring the system can operate efficiently in the actual farm environment.

Le Lam Nhat Linh (SE162158) focuses on developing machine learning models for the project. Linh's main work involves building and implementing pig detection and tracking models using YOLOv8/v9/NAS, along with developing behavior analysis systems using Vision Transformer (ViT). Linh also participates in data collection and labeling processes, while developing monitoring features such as pig counting and aggressive behavior detection.

1.7.2 Communication Plan

To ensure effective project execution, the team has established specific communication and work plans. Weekly meetings with supervisors are held during Slot 6 on Mondays to report progress and receive project feedback. During these meetings, the team presents achieved results, discusses encountered difficulties, and plans for the following week.

Among team members, communication is maintained regularly through online channels like Discord to discuss technical issues and update work progress. The team also organizes technical meetings every two weeks for in-depth discussions about implementation issues and proposing solutions for technical challenges.

For task management, the team uses Notion to track individual tasks and progress. Code is managed through GitHub, enabling efficient collaboration and code review. Project documentation is stored and regularly updated on Google Docs, facilitating report writing and document compilation later on.

Important project decisions are discussed with the participation of the entire team. For technical issues, the member responsible for that area researches and proposes solutions, followed by group discussions to choose the most optimal approach. Decisions that significantly impact the project are consulted with supervisors before implementation.

1.8 Project Management Approach

1.8.1 Project Overview

This section outlines the comprehensive project management approach for developing an AI-based Smart Farm Management System. The system integrates advanced artificial intelligence technologies to create a robust monitoring and management solution for pig farms. Our approach emphasizes systematic development, thorough testing, and careful integration of multiple AI components to ensure a reliable and effective final product.

1.8.2 Project Organization

Project Phases

The project is structured into five main phases:

Phase 1: Initiation & Planning The initial phase establishes the project's foundation through comprehensive requirements analysis and planning. Our team conducts extensive stakeholder interviews with farm managers and technical experts to gather detailed requirements. We perform thorough technical feasibility studies, particularly focusing on the implementation challenges of AI systems in farm environments. This phase includes developing detailed project timelines, allocating resources, and establishing communication protocols with all stakeholders.

Phase 2: Research & Design The research phase involves comprehensive literature reviews across multiple domains, including object detection, behavior analysis, and disease classification in agricultural settings. Our team analyzes current technological capabilities, particularly focusing on YOLOv8/v9/NAS architectures, ViT-based models, and ResNet50v2 implementations. The design process incorporates findings from this research to create a robust system architecture that can handle real-world farm conditions.

Phase 3: Development Development proceeds along parallel tracks to optimize resource utilization and meet project timelines. The core detection and tracking system development focuses on implementing and optimizing YOLOv8/v9/NAS architectures for pig detection. Behavior analysis development includes creating sophisticated algorithms for monitoring multiple behaviors, including aggressive interactions, feeding patterns, and movement analysis. The weight estimation component combines computer vision techniques with machine learning models to provide accurate measurements. Disease classification development emphasizes creating robust models capable of early detection of health issues.

Phase 4: Testing & Validation Our testing strategy implements a multi-layered approach to ensure system reliability. Unit testing covers individual components, while integration testing verifies proper interaction between modules. Performance validation includes stress testing under various farm conditions, ensuring the system maintains accuracy and speed under real-world scenarios. We conduct extensive field testing in actual farm environments to validate system performance across different conditions.

Phase 5: Deployment & Documentation The final phase focuses on successful system deployment and comprehensive documentation. This includes installing and configuring hardware components, setting up monitoring systems, and training farm staff. Documentation covers technical specifications, user manuals, and maintenance procedures. We also prepare detailed handover documentation to ensure smooth system operation after project completion.

1.8.3 Component Integration Strategy

Component integration follows a systematic approach to ensure smooth functioning of the complete system. The detection system serves as the foundation, providing accurate pig identification and tracking. Behavior analysis modules integrate with tracking data to monitor animal activities. Weight estimation and disease classification components operate independently but share data through a central system. All components communicate through a standardized API, ensuring consistent data flow and system stability.

1.8.4 Quality Management

Performance Standards

Our quality management approach establishes strict performance standards for each system component. Detection accuracy must exceed 90% across varying lighting conditions and farm scenarios. Behavior classification systems must achieve minimum 85% accuracy for key behaviors, with particular emphasis on aggressive interaction detection. Weight estimation error margins must remain within 5% of actual measurements, verified through regular calibration checks. Disease classification requires 90% accuracy with minimal false positives.

System Performance Requirements

The integrated system must maintain real-time processing capabilities on the Jetson Nano platform, consistently achieving 15 FPS while running all components. API response times must remain below 100ms for critical operations, ensuring timely system responses to important events. System stability is verified through extended running tests under various farm conditions, with requirements for 99.9% uptime during operational hours.

1.8.5 Risk Management

Our risk management strategy addresses both technical and operational challenges. Technical risks related to processing capabilities on the Jetson Nano platform are managed through continuous performance optimization and early prototype testing. Environmental risks, including varying lighting conditions and farm atmosphere effects, are addressed through robust preprocessing techniques and environmental control measures. Data collection risks are mitigated through redundant collection systems and rigorous quality control protocols.

Integration risks are managed through careful component design and systematic testing procedures. Regular system audits identify potential compatibility issues early in the development process. Communication risks are addressed through established protocols and regular team meetings.

1.8.6 Progress Monitoring and Control

Progress monitoring utilizes a comprehensive tracking system. Weekly team meetings review development progress, address technical challenges, and adjust project timelines as needed. Supervisor consultations during Monday Slot 6 provide strategic guidance and ensure alignment with project objectives. Monthly progress reports track key performance indicators and milestone achievements. Regular stakeholder updates ensure transparency and maintain project alignment with business objectives.

The project employs a formal change management process to handle scope modifications and technical adjustments. All changes undergo impact assessment and require appropriate approvals before implementation. Documentation is continuously updated to reflect system evolution and maintain accurate technical records.

Chapter 2

Existing Systems/State of the Art

2.1 Overview of the Field

The current landscape of smart pig farming represents a convergence of advanced technologies aimed at optimizing livestock management and welfare. Modern farming operations increasingly rely on integrated systems that combine sensors, artificial intelligence, and automation to address traditional farming challenges.

Leading companies in the field have developed comprehensive monitoring solutions that showcase the current state of technology. For example, Fancom's eYeScan system demonstrates modern computer vision capabilities by automatically tracking pig movement and estimating weights without physical interaction. Similarly, the PigWatch system developed by researchers at the University of Leeds employs AI algorithms to detect and alert farmers to aggressive behaviors, achieving detection accuracy rates of over 90% in commercial farm settings.

Weight monitoring technology has evolved significantly, with systems like Velos' Smart Weighing utilizing 3D cameras and machine learning to estimate pig weights within 3% accuracy of physical scales. This technology allows farmers to monitor growth rates daily without the stress and labor of traditional weighing methods. The system processes over 1000 measurements per day in typical finishing pens, providing unprecedented insight into growth patterns.

Behavior analysis systems have become increasingly sophisticated. The eYeGrow platform, implemented in farms across Europe, combines computer vision with machine learning to monitor feeding patterns, movement, and social interactions. The system processes video feeds from multiple cameras to create comprehensive behavior profiles for each pen, enabling early detection of health and welfare issues.

Disease detection has advanced through the integration of thermal imaging and AI analysis. Systems like the Fever Check-AI, deployed in Asian pig farms, can automatically detect potential health issues by monitoring body temperature patterns and behavioral changes. These systems have demonstrated the ability to identify illness up to 24 hours before visible symptoms appear, enabling earlier intervention and better disease control.

Edge computing has made these sophisticated systems practical for farm implementation. The deployment of NVIDIA Jetson platforms in commercial farms has shown that complex AI algorithms can run effectively on-site, processing up to 30 frames per second while maintaining accuracy rates above 95

Current challenges in the field primarily relate to system integration and environmental variability. While individual components show impressive capabilities, creating fully integrated systems that work reliably across different farm layouts and conditions remains an active area of development. Companies like Big Dutchman and Agrollogic are working to develop standardized platforms that can integrate various monitoring technologies into cohesive farm management systems.

These technological developments occur against a backdrop of increasing regulatory requirements and consumer interest in animal welfare. The European Union’s PigWatch project, for instance, has established benchmarks for automated welfare monitoring, driving the development of more sophisticated behavior analysis systems. Similar initiatives in Asia and North America are pushing the advancement of smart farming technologies while ensuring their practical applicability in commercial settings.

2.2 Historical Context

The evolution of pig farming technology over the past several decades demonstrates a clear progression from basic mechanization to sophisticated AI-driven systems. This transformation can be traced through several key technological milestones that have fundamentally changed farm management practices.

The mechanization era of the 1960s-1970s introduced the first significant technological advances in pig farming. Companies like Big Dutchman pioneered automated feeding systems, while early environmental control systems from manufacturers such as Fancom began regulating basic barn conditions. These innovations marked the first departure from purely manual farming methods, though monitoring and management still relied heavily on human observation.

The 1980s-1990s saw the emergence of digital farm management systems. PigCHAMP, introduced in 1983, became one of the first widely-adopted computerized record-keeping systems for pig farms, revolutionizing how farmers tracked and analyzed production data. The system allowed farmers to maintain detailed records of breeding, health, and production metrics, though data entry remained a manual process.

The early 2000s brought significant advances in individual animal identification and tracking. The implementation of RFID systems, such as the Allflex electronic ear tags, enabled automated individual pig identification. These systems, while revolutionary, were limited by their requirement for close-range scanning and the need for physical tags on each animal.

Video monitoring systems emerged as a significant tool in the late 2000s, with companies like Axis Communications developing specialized cameras for livestock monitoring. These early systems, though primarily used for security and basic observation, laid the groundwork for future computer vision applications. By 2010, farms began installing network-connected camera systems that could provide remote viewing capabilities, though analysis still required human interpretation.

The integration of artificial intelligence and computer vision marks the most recent and perhaps most significant advancement. Since 2015, companies like Fancom with their eYeScan system and MLV Group’s PigVision have demonstrated the potential of AI in pig monitoring. These systems can automatically track multiple animals, estimate weights, and analyze behavior patterns without human intervention.

Edge computing capabilities, exemplified by NVIDIA’s Jetson platform, have addressed many previous limitations in processing power and connectivity. The Jetson Nano, introduced in 2019, has made sophisticated on-site AI processing accessible to farms of various sizes, enabling real-time analysis without requiring constant internet connectivity or cloud processing.

This technological progression has been driven not only by advances in computing and sensor technology but also by changing industry needs. Increasing labor costs, growing emphasis on animal welfare, and stricter regulatory requirements have all contributed to the acceleration of technological adoption in pig farming. Understanding this historical context provides valuable insight into both current systems and likely future developments in smart farming technology.

2.3 Key Studies and Theories

2.3.1 Object Detection and Tracking Systems

Recent studies have shown significant advancement in the application of deep learning for pig detection and tracking. The YOLO (You Only Look Once) architecture has emerged as a leading solution in this domain, with various improved versions being proposed for pig monitoring applications. Notably, Zhong et al. proposed YOLO-DLHS-P, a lightweight variant of YOLOv8 specifically optimized for pig behavior recognition. Their model introduced the C2f-DRB structure and LSKA attention mechanism, achieving improved precision while maintaining computational efficiency [1].

In the realm of instance segmentation, Brünger et al. developed a panoptic segmentation framework that achieves pixel-accurate segmentation of individual pigs, reaching detection rates of approximately 95% F1 Score despite challenging conditions such as occlusions and dirty lenses [2]. This approach provides more detailed information about pig morphology compared to traditional bounding box detection methods.

For tracking multiple pigs in group housing scenarios, Tu et al. introduced Pig-ByteTrack, an improved version of the ByteTrack algorithm specifically designed for pig tracking. Their system achieved impressive results with 72.9% Higher Order Tracking Accuracy (HOTA) and 91.7% Multi-Object Tracking Accuracy (MOTA) in short-term tracking scenarios [3].

2.3.2 Behavior Analysis and Recognition

Behavior recognition in pigs has seen significant developments through the application of various deep learning architectures. Hao et al. proposed a two-stream mutual learning network (TSML) that combines RGB and flow streams for pig behavior recognition, achieving state-of-the-art recognition performance with an accuracy of 96.52% [4].

For aggressive behavior detection, Ji et al. developed an efficient method using the Temporal Shift Module (TSM) integrated with various CNN architectures. Their ResNeXt50-T model achieved remarkable results with 95.69% accuracy and 95.65% F1 score in recognizing aggressive behaviors [5]. This approach successfully addresses the challenge of processing both spatial and temporal features without significantly increasing computational complexity.

2.3.3 Weight Estimation Methods

Weight estimation through computer vision has emerged as a crucial area of research in smart pig farming. Jiang et al. proposed a framework combining Mask R-CNN and ensemble regression models for contactless pig weight estimation. Their method addresses key challenges such as uneven illumination and body bending, achieving high accuracy with an MAE of 0.389 and R^2 of 0.995 [6].

Alternative approaches to weight estimation have also been explored through behavioral data analysis. He et al. investigated the use of feeding behavior data for weight prediction using various machine learning algorithms, including LASSO regression, Random Forest, and Long Short-term Memory networks. Their study achieved correlations up to 0.87 and accuracy of 0.89 for individual-informed predictions [7].

2.3.4 Disease Detection and Health Monitoring

Recent studies have demonstrated the potential of behavioral analysis for early disease detection in pigs. Kavlak et al. investigated disease detection based on feeding behavior traits using machine learning approaches. Their study utilized XGBoost algorithms analyzing features

such as number of visits per day, time spent feeding, and daily feed intake, achieving 80% AUC and 67% sensitivity in disease prediction [8].

The integration of these monitoring systems into smart farming environments represents a significant advancement in pig welfare and health management. Wang et al.'s comprehensive review highlights how artificial intelligence technologies have become core components of smart pig farming, enabling real-time monitoring of biometric information and early detection of health issues [9].

2.3.5 Theoretical Foundations and Frameworks

The theoretical framework for smart pig farming systems typically incorporates several key components:

- **Computer Vision Architecture:** Most modern systems utilize deep learning-based architectures, particularly variants of YOLO and Mask R-CNN, for object detection and segmentation.
- **Temporal Analysis:** Behavioral recognition systems often employ temporal analysis methods such as TSM or two-stream networks to capture both spatial and temporal features of pig behavior.
- **Edge Computing Integration:** Many studies emphasize the importance of edge computing capabilities, particularly using platforms like Jetson Nano, for real-time processing of video feeds in farm environments.
- **Data Fusion Techniques:** Advanced systems often combine multiple data sources, including visual data, behavioral metrics, and environmental factors, for comprehensive monitoring and analysis.

These theoretical foundations provide the basis for developing integrated systems that can effectively monitor and analyze pig behavior, health, and welfare in commercial farming operations.

2.4 Technological Advancements

Recent years have witnessed significant technological breakthroughs in artificial intelligence and computer vision applications for agricultural practices, particularly in smart pig farming. These advancements have transformed traditional farming methods into data-driven, automated systems capable of real-time monitoring and decision-making.

2.4.1 Evolution of Detection and Tracking Systems

The development of object detection systems has progressed significantly with the evolution of YOLO architectures. From YOLOv3 to the recent YOLOv8 and YOLOv9, each iteration has brought improvements in accuracy and processing speed. The latest versions incorporate advanced features such as cross-stage partial connections and spatial pyramid pooling, enabling better detection performance in challenging farm environments with varying lighting conditions and occlusions.

Modern pig tracking systems have evolved to handle multiple objects simultaneously while maintaining individual identities through complex interactions. Advanced tracking algorithms like ByteTrack and DeepSORT have been adapted specifically for livestock monitoring, incorporating temporal consistency and motion prediction to maintain tracking accuracy even during pig overlapping and crowded scenarios.

2.4.2 Advancement in Deep Learning Architectures

Vision Transformers (ViT) have emerged as a powerful alternative to traditional convolutional neural networks for visual recognition tasks. In pig farming applications, ViT-based models have demonstrated superior performance in behavior analysis by better capturing long-range dependencies in temporal sequences. These models can effectively process sequential data to identify complex behavioral patterns and social interactions among pigs.

The integration of attention mechanisms has significantly improved feature extraction capabilities. Recent implementations combine spatial and channel attention modules to enhance the network's ability to focus on relevant features while suppressing noise, leading to more robust performance in real-world farm conditions.

2.4.3 Edge Computing Integration

The deployment of AI systems in farm environments has been revolutionized by advances in edge computing platforms. The NVIDIA Jetson family, particularly the Jetson Nano, has enabled real-time processing of multiple video streams directly on-site. These platforms support concurrent execution of multiple AI models for detection, tracking, and behavior analysis while maintaining low latency and power consumption.

Modern edge computing solutions incorporate optimized inference engines and model quantization techniques, allowing complex deep learning models to run efficiently on resource-constrained devices. This advancement has made sophisticated AI systems accessible to farms of various scales without requiring extensive infrastructure investments.

2.4.4 Multi-Modal Sensing and Data Fusion

Recent technological developments have emphasized the integration of multiple sensing modalities. Modern systems combine visual data with other sensor inputs such as depth cameras, thermal imaging, and environmental sensors. This multi-modal approach provides a more comprehensive understanding of pig behavior and health status.

Advanced data fusion techniques have been developed to synchronize and integrate information from various sources effectively. These methods employ sophisticated algorithms to combine visual observations with behavioral metrics and environmental data, enabling more accurate assessment of pig welfare and early detection of health issues.

2.4.5 Automated Behavior Analysis Systems

The latest behavior analysis systems incorporate temporal modeling capabilities through architectures like Temporal Shift Modules (TSM) and 3D convolution networks. These advances allow for more nuanced understanding of pig behaviors over time, enabling the detection of subtle behavioral changes that might indicate health or welfare issues.

Real-time behavioral monitoring systems have evolved to include automated stress detection and aggressive behavior prediction capabilities. These systems utilize advanced pattern recognition algorithms to identify potential issues before they escalate, allowing for proactive management interventions.

2.4.6 Weight Estimation and Health Monitoring

Recent advances in computer vision have enabled more accurate contactless weight estimation through improved segmentation techniques and 3D modeling. Modern systems combine Mask

R-CNN architectures with specialized regression models to account for various pig poses and positions, achieving accuracy levels comparable to physical weighing.

Health monitoring systems have evolved to incorporate machine learning models capable of detecting early signs of disease through behavioral analysis and visual inspection. These systems analyze patterns in feeding behavior, movement, and social interactions to identify potential health issues before they become severe.

These technological advancements continue to push the boundaries of what's possible in smart pig farming, enabling more efficient, welfare-friendly, and productive agricultural practices. The ongoing development of these technologies promises even more sophisticated and integrated solutions for future farming operations.

2.5 Comparison of Existing Systems

The landscape of smart pig farming solutions has evolved significantly in recent years, with several commercial and research-based systems emerging to address the growing demands of modern pig production. This section analyzes the major existing solutions, comparing their capabilities, advantages, and limitations.

2.5.1 Commercial Vision-Based Systems

Fancom Total Automation System

Fancom has established itself as a leading provider of integrated pig farm management solutions through its comprehensive automation system. At the core of their offering is the eYegrow weighing system, which utilizes 3D camera technology to achieve 97

The system's strength lies in its integrated approach to farm management. Their climate control technology maintains optimal conditions throughout the growth period while monitoring and managing CO₂ and ammonia emissions, dust levels, and noise. This is complemented by an advanced feeding system that optimizes feed distribution and monitors consumption patterns, directly impacting farm efficiency and animal welfare.

What sets Fancom apart is their emphasis on customization and scalability. The system can be tailored to specific farm requirements and easily scaled as operations grow. Their focus on environmental impact is evident in their energy-efficient design and emissions reduction capabilities, aligning with modern sustainable farming practices.

PLF Agritech GROW PERFECT System

PLF Agritech's GROW PERFECT system represents a unique collaboration between industry and academia in smart livestock monitoring. The system emphasizes continuous health and welfare monitoring through real-time image analysis and environmental sensing. Their approach integrates various ICT tools and sensors to create a comprehensive data-driven decision support system.

A distinguishing feature of the GROW PERFECT system is its focus on practical application of research findings. The system optimizes animal growth through advanced technology while prioritizing improved living conditions and reduced environmental impact. Their data management platform provides actionable insights that enable farmers to make informed decisions about their operations.

However, the system requires significant infrastructure investment and comprehensive staff training. Despite these limitations, its integrated approach to livestock monitoring and emphasis on both economic and welfare outcomes makes it a compelling solution for modern pig farming operations.

Big Dutchman Integrated Solutions

Big Dutchman offers a comprehensive suite of solutions through their BFN Fusion platform, representing the latest in digital farm management technology. Their approach encompasses everything from basic housing solutions to advanced digital management systems, with a particular emphasis on animal welfare through innovations like the Havito welfare barn design and Xaletto straw bedding system.

The company's strength lies in its holistic approach to pig production, covering the entire lifecycle from piglet rearing to finishing. Their environmental control systems, including advanced climate control and air cleaning solutions, work in concert with their production management systems to create optimal conditions for pig growth and welfare.

Big Dutchman's "Pig Production 2030" concept demonstrates their commitment to future-oriented farming practices, emphasizing sustainability and innovation in animal welfare. Their modular design approach allows for easy system expansion and adaptation to different regional requirements, making it suitable for operations of varying scales.

Table 2.1: Comprehensive Feature Comparison of Commercial Pig Farming Systems

System Capabilities	Our System	Fancom	PLF Agritech	Big Dutchman
Core Monitoring Features				
Pig Detection	✓	✓	✓	✓
Individual Tracking	✓	✓	✓	✓
Real-time Monitoring	✓	✓	✓	✓
Advanced Analysis				
Behavior Analysis	✓	✓	✓	✓
Weight Estimation	✓	✓	✓	✓
Disease Detection	✓	—	—	—
Aggressive Behavior Detection	✓	—	✓	—
Technical Features				
Edge Processing	✓	—	—	—
Cloud Integration	✓	✓	✓	✓
Real-time Alerts	✓	✓	✓	✓
Mobile App Support	—	✓	✓	✓
Environmental Control				
Climate Monitoring	—	✓	✓	✓
Emission Control	—	✓	—	✓
Ventilation Management	—	✓	—	✓
Additional Features				
Feeding Management	—	✓	—	✓
Historical Data Analysis	✓	✓	✓	✓
Custom Report Generation	✓	✓	✓	✓

All systems provide core monitoring features such as pig detection and tracking, demonstrating the industry’s standardization in basic functionalities. However, each system shows distinct advantages in specific areas: Fancom excels in environmental control and feeding management, PLF Agritech offers strong behavior analysis capabilities, and Big Dutchman provides comprehensive facility management features.

Our system’s unique edge processing capability enables faster response times and reduced dependency on cloud connectivity, while the integration of disease detection represents a significant advancement in proactive health monitoring. These innovations address crucial gaps in existing commercial solutions, particularly for farms seeking advanced monitoring capabilities without the need for extensive infrastructure modifications.

2.5.2 Cost and Implementation Considerations

The implementation of commercial smart farming systems represents a significant investment for pig farmers. Initial setup costs typically range from \$10,000 to \$28,000, including equipment, licensing, and installation. Ongoing operational costs, including maintenance, software updates, and equipment replacement, can amount to \$2,000-\$6,500 annually.

While these systems offer comprehensive solutions for pig farm management, our proposed system aims to provide comparable functionality at reduced cost through edge computing and open-source technologies. This approach not only makes smart farming more accessible to smaller operations but also offers greater flexibility in system customization and expansion.

2.6 Gaps in the Literature/Technology

Despite significant advancements in smart pig farming technologies, several crucial limitations and gaps remain in current solutions. These gaps present opportunities for innovation and improvement in future systems.

2.6.1 Technical Limitations

Current pig monitoring systems face several technical challenges. Most existing solutions rely heavily on controlled environmental conditions, particularly consistent lighting and clear camera views. This dependency limits their effectiveness in real-world farm environments where lighting conditions vary significantly throughout the day and cameras may be affected by dust, moisture, or other environmental factors.

The accuracy of individual pig tracking in group housing scenarios remains problematic. While current systems can track individual pigs in less crowded conditions, their performance degrades significantly when pigs cluster together or during intense social interactions. This limitation affects the reliability of behavioral analysis and individual monitoring in commercial farm settings.

2.6.2 Processing and Implementation Challenges

Most commercial systems require substantial computing infrastructure and reliable internet connectivity for cloud-based processing. This centralized approach introduces latency in data processing and creates potential points of failure in areas with unreliable network connections. While some systems offer local processing capabilities, they often sacrifice advanced features or processing speed to maintain real-time operation.

The balance between processing power and energy efficiency remains a significant challenge, particularly for edge computing solutions. Current edge devices struggle to simultaneously run

multiple AI models for detection, tracking, and behavior analysis while maintaining real-time performance within reasonable power consumption limits.

2.6.3 Behavioral Analysis Limitations

Existing behavior recognition systems typically focus on basic activities and movement patterns. There is a notable gap in the detection and analysis of complex social interactions and subtle behavioral changes that might indicate early health issues. The ability to distinguish between normal social interactions and genuinely aggressive behavior requires further development.

The integration of temporal context in behavior analysis is limited. Most systems analyze behaviors in isolated time frames rather than considering longer-term patterns and behavioral trends, which could provide more meaningful insights into animal welfare and health status.

2.6.4 Weight Estimation Accuracy

While vision-based weight estimation has progressed significantly, current solutions still face accuracy challenges when dealing with different pig poses and positions. The reliability of weight estimates decreases when pigs are partially occluded or in motion, limiting the practical application of these systems in continuous monitoring scenarios.

2.6.5 Disease Detection Capabilities

Current systems largely lack sophisticated disease detection capabilities. While some solutions can identify basic behavioral changes that might indicate health issues, they cannot reliably detect specific diseases or health conditions. The integration of multiple data sources for comprehensive health monitoring remains underdeveloped.

2.6.6 Integration and Standardization

There is a significant gap in the standardization of data formats and integration protocols across different smart farming systems. This lack of standardization makes it difficult to combine data from multiple sources or integrate different monitoring systems into a cohesive farm management solution.

2.6.7 Cost and Accessibility

The high implementation and maintenance costs of current commercial systems limit their accessibility to smaller farming operations. Most solutions require significant initial investment in infrastructure and ongoing costs for maintenance and updates, creating a barrier to adoption for many potential users.

2.6.8 Research Opportunities

These gaps present several key opportunities for future research and development:

- Development of more robust computer vision algorithms that can operate effectively under varying environmental conditions
- Integration of multiple sensing modalities to improve the accuracy of health and behavior monitoring

- Implementation of more sophisticated edge computing solutions that can balance processing requirements with power efficiency
- Creation of standardized protocols for data sharing and system integration
- Development of more accessible and cost-effective solutions for smaller farming operations

Addressing these limitations requires a multidisciplinary approach combining advances in computer vision, machine learning, edge computing, and animal behavior understanding. Future solutions must not only overcome these technical challenges but also remain practical and economically viable for real-world implementation.

2.7 Justification for the Project

This project aims to explore the application of artificial intelligence in pig farming through a proof-of-concept Smart Farm Management System. While commercial solutions offer comprehensive farm management capabilities, our project focuses specifically on investigating the potential of edge computing and modern AI algorithms in livestock monitoring.

2.7.1 Educational and Research Value

As a project in artificial intelligence, this work serves primarily as an educational exploration of applying AI techniques to real-world problems. The implementation of various deep learning models - YOLOv8/v9/NAS for detection and tracking, Vision Transformer for behavior analysis, and ResNet50v2 for disease detection - provides valuable hands-on experience in integrating multiple AI components into a practical application.

2.7.2 Technical Exploration

Our project explores the feasibility of running complex AI models on edge devices like the Jetson Nano. While commercial systems typically utilize powerful cloud infrastructure, our implementation investigates the possibilities and limitations of edge computing in farm environments. This exploration could contribute to the understanding of edge AI applications in agricultural settings.

2.7.3 Cost-Effective Alternative

While not matching the comprehensive capabilities of commercial systems, our project investigates whether a simplified monitoring system using open-source technologies could provide basic functionality for smaller farms or research facilities. The focus on edge computing and open-source solutions explores potential cost-effective alternatives for basic monitoring needs.

2.7.4 Learning Through Implementation

The project provides valuable learning opportunities in several key areas:

- Practical experience in implementing and optimizing deep learning models
- Understanding the challenges of real-time processing on edge devices
- Gaining insights into the complexities of agricultural technology
- Learning to balance system capabilities with hardware limitations

2.7.5 Foundation for Future Research

Although modest in scope compared to commercial solutions, this project establishes a foundation for future academic research in smart farming applications. The experience and insights gained could inform future projects and contribute to the growing body of knowledge in agricultural AI applications.

This project, while acknowledging its limitations as a graduation project, aims to contribute to the understanding of AI applications in agriculture and provide practical learning experiences in system integration and edge computing implementation.

Chapter 3

Methodology

3.1 Research Questions and Objectives

3.1.1 Research Questions

This research project addresses several fundamental questions in developing an AI-based pig farm management system:

1. How can deep learning models be optimized for accurate pig detection and tracking in varying farm environments?
2. What motion analysis techniques can effectively classify basic pig behaviors from tracking data?
3. How can deep learning models specifically identify aggressive interactions among pigs?
4. What approaches enable accurate weight estimation through computer vision techniques?
5. How can disease detection be implemented effectively using visual analysis?
6. What strategies enable efficient deployment of multiple AI models on edge computing devices?

3.1.2 Technical Objectives

Our research aims to develop a comprehensive smart farm management system with several interconnected components. The core detection and tracking system utilizes YOLOv8/v9/NAS architectures, targeting detection accuracy above 90% under various lighting conditions while maintaining real-time processing at minimum 15 FPS on Jetson Nano hardware. This system must demonstrate robust performance in handling occlusions and overlapping scenarios, while maintaining consistent pig identities in group settings.

The behavior analysis system employs a dual approach combining motion analysis and deep learning. Basic behaviors such as moving, stationary states, lying, and feeding are classified through sophisticated motion analysis algorithms operating on tracking data. This approach analyzes displacement vectors, velocity patterns, and zone-based activities to determine pig states. For more complex behaviors, particularly aggressive interactions, we implement a Vision Transformer (ViT) architecture targeting classification accuracy above 85% with minimal false positives.

Weight estimation capabilities are implemented through YOLOv8/v11 for pig segmentation and pose detection. The system aims to achieve weight estimation accuracy within $\pm 5\%$ of

physical measurements across various pig poses and growth stages. This includes developing robust calibration methods for different camera setups and environmental conditions.

The disease detection component utilizes ResNet50v2 architecture, targeting classification accuracy above 90% for early detection of common visual symptoms while maintaining a false positive rate below 5%. This system incorporates multi-class disease classification capabilities and implements an alert system for potential health issues.

System integration and deployment focuses on optimizing performance on the Jetson Nano platform. This involves achieving efficient concurrent operation of all components while maintaining real-time processing capabilities and implementing effective memory management strategies. The system architecture supports real-time data processing and analysis, efficient storage and retrieval of monitoring data, and a robust alert generation system.

3.1.3 Expected Outcomes

The successful completion of this research will result in a functional smart farm management system prototype demonstrating real-time pig monitoring and tracking, automated behavior analysis, visual weight estimation, and disease detection capabilities. The project will provide comprehensive documentation of the system architecture, implementation details, and performance metrics, contributing valuable insights into practical challenges in smart farm implementation and edge computing optimization techniques.

This research aims to advance the field of agricultural technology by developing practical solutions for livestock monitoring while addressing the challenges of implementing complex AI systems in real-world farming environments. The findings will contribute to the growing body of knowledge in computer vision applications for livestock management and edge computing optimization in agricultural settings.

3.2 Data Collection and Preprocessing

The development of our smart farm management system required comprehensive datasets for multiple components. We employed systematic data collection and preprocessing approaches for each system component, ensuring data quality and relevance for model training.

3.2.1 Detection Dataset

Our primary detection and counting dataset was collected from a pig farm in Dong Nai Province, Vietnam. Data collection was performed using two IMOU IPC-F22P-D 1080P 2MP Outdoor IP cameras installed at different viewing angles to maximize coverage and enhance detection accuracy. The dataset includes pigs from Landrace, Yorkshire, and Duroc breeds during their growing stage, providing a diverse representation of pig phenotypes commonly found in commercial farming operations.

The data collection process yielded 70 videos, each three hours in length, recorded continuously over a two-week period. All videos were captured at 1920 x 1080 resolution under varying environmental conditions. From these recordings, we extracted 20,000 frames to create our image dataset. The extraction process was designed to capture diverse scenarios including varying lighting conditions from day to night operations and different pig postures, ensuring robust model performance and generalization capabilities.

Image annotation was performed using a combination of Roboflow and AnyLabeling tools. Each frame was carefully annotated with bounding boxes identifying individual pigs. This meticulous annotation process was crucial for training the YOLO-based detection model. The

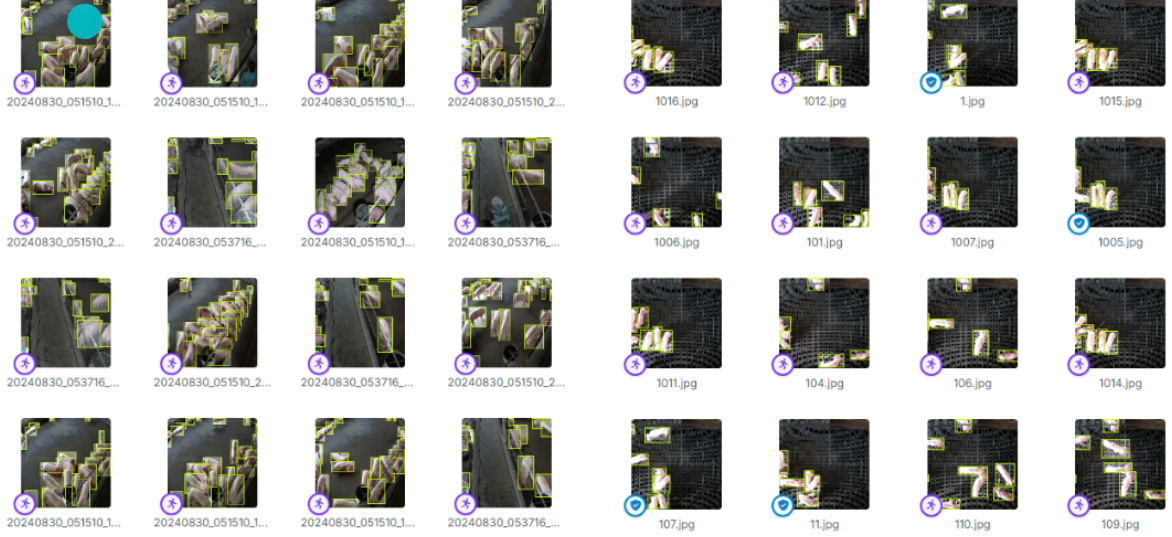


Figure 3.1: Sample annotations demonstrating bounding box placement for pig detection

resulting dataset provides a strong foundation for accurate and efficient object detection, particularly given the advantages of the YOLO model family in real-time applications.

3.2.2 Behavior Analysis Dataset

For behavior analysis, we utilized two distinct private datasets. The first dataset, provided by Research Base for Pig Nutrition and Environmental Control, was collected from a pig farm in Rongchang District, Chongqing City, China (2022). This dataset comprises video sequences totaling 1 hour and 30 minutes, specifically focused on capturing both aggressive and non-aggressive pig behaviors.

The aggressive behavior dataset was carefully curated to ensure natural behavior capture without human interference. Professional experts supervised the extraction and classification of aggressive and non-aggressive behaviors. To address the temporal nature of aggressive interactions, which typically occur in mixed groups and vary in duration, we processed the original footage into one-second video segments. This processing resulted in a balanced dataset containing 5,220 aggressive behavior videos and 5,220 non-aggressive behavior videos, all standardized to 640x640 resolution for optimal training efficiency.

The second dataset, provided by IPCLab-NEAU and collected from HongFu Pig Farm in Harbin City, Heilongjiang Province, China (2021), consists of 15 videos totaling 30 minutes. This dataset captures a range of pig activities including lying, eating, standing, and social interactions. Unlike the aggressive behavior analysis, which relies on deep learning classification, these basic behaviors are analyzed using advanced tracking algorithms and zone analysis methods, incorporating distance measurements as behavioral indicators.

Both datasets were systematically divided into training, validation, and test sets using an 8:1:1 ratio, maintaining the 640x640 resolution standard for consistent processing.

3.2.3 Health Monitoring Dataset

The health monitoring dataset was designed for early warning system implementation, comprising two distinct classes: sick and healthy pigs. The sick class images were collected from

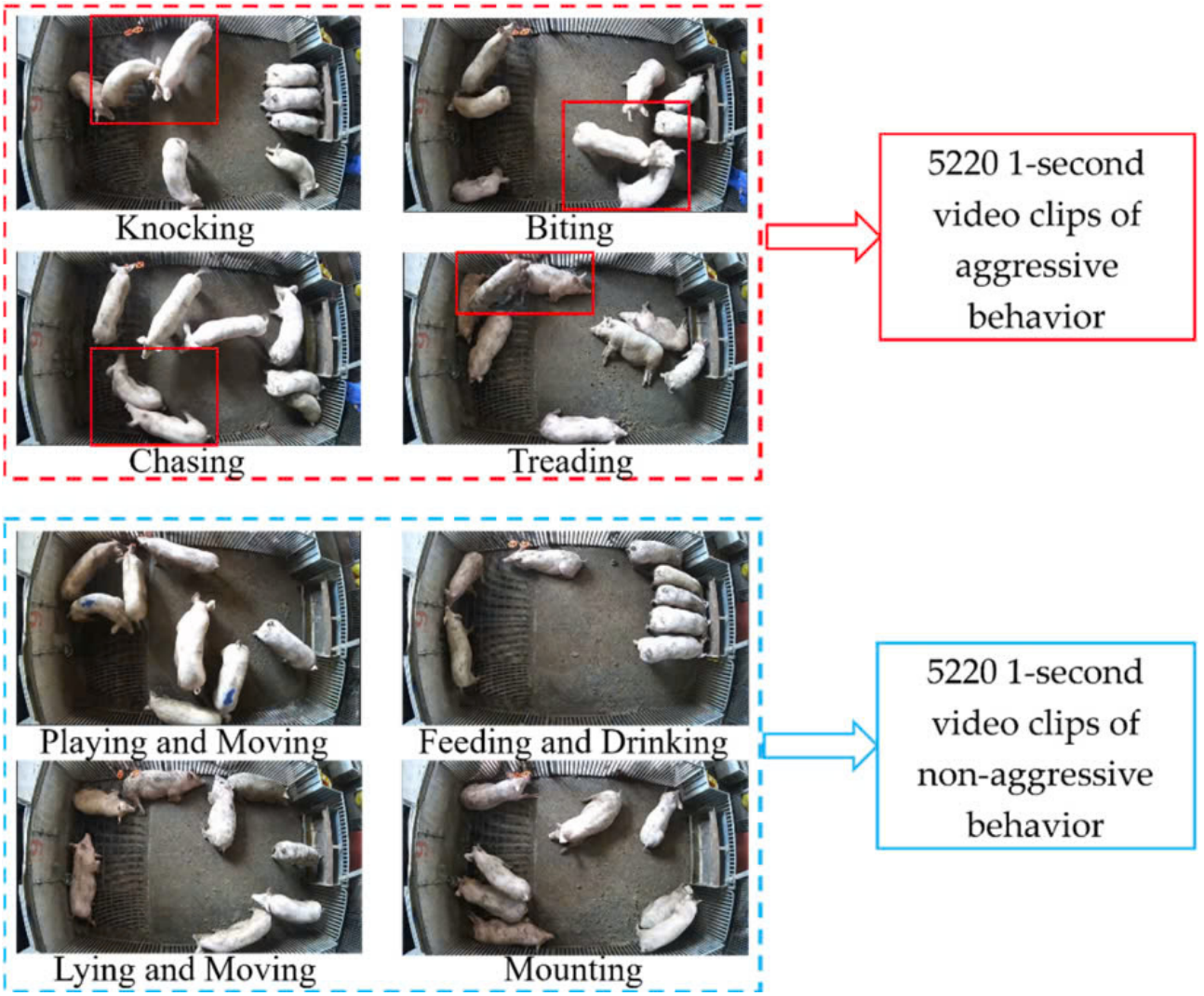


Figure 3.2: Composition and examples of aggressive and non-aggressive behaviors in data[5].

multiple reliable sources, including:

- Online resources
- Public datasets
- Biological academic studies
- Private datasets from farms with documented disease occurrences

For the healthy class, we employed our trained detection model to crop and compile images from our primary data collection site in Dong Nai, Vietnam. The complete dataset contains 4,000 images equally distributed between sick and healthy classes (2,000 images each). All images were preprocessed to 224x224 resolution, optimized for processing cropped images with lower pixel counts. This resolution choice represents a practical compromise between processing efficiency and feature preservation for binary classification tasks.

3.2.4 Weight Estimation Dataset

The weight estimation dataset was collected during an intensive three-month period from September to November 2024 at our collaboration farm in Dong Nai province, Vietnam. The data collection setup was carefully designed to ensure accuracy and consistency:

- Camera: IMOU IPC-F22P-D 1080P 2MP Outdoor IP
- Video Resolution: 1920 x 1080 at 30 fps
- Network Infrastructure: Dahua DHI-NVR1108HS-S3-DSS Network Video Recording system
 - Compression: H.264B
 - Network Protocol: TCP/IP
 - Connection: RJ45

The physical setup was precisely configured for optimal data collection:

- Aisle width: 1.5 meters
- Camera height: 2.5 meters from ground level
- Cage dimensions: 0.84m width $\times$ 1.42m length $\times$ 1.0m height
- Camera viewing distance: 1.1 meters from cage top, 2.1 meters from ground level

The dataset includes 174 calibrated measurements of individual pigs, with body masses ranging from 115-135 kg. Each pig was weighed five times to minimize measurement error, with the most accurate reading selected based on the weighing system’s balance performance indicators. Regular cleaning and calibration of the weighing scales ensured measurement accuracy throughout the data collection period. We utilized 174 videos single pigs and 200 videos multiple pigs from the weight dataset. These videos was process to frame extraction to construct a dataset with a resolution of 640x640 pixels. The extracted data was labeled using the Roboflow tool, resulting in a total of 3,878 labeled images for segmentation and 1,000 labeled images for keypoint detection.

All collected data was systematically stored and backed up using the IMOU cloud service platform, with accompanying Excel documentation for data tracking and validation. This comprehensive approach to data collection and management ensures the reliability and reproducibility of our weight estimation system.

3.2.5 Data Preprocessing Methodology

The preprocessing pipeline was carefully designed for each dataset to ensure optimal model training conditions while preserving critical features. Our preprocessing approach was tailored to the specific requirements of each system component.

Detection Dataset Preprocessing

For the detection dataset, preprocessing steps included:

- Resolution standardization to 640 $\times$ 640 pixels for YOLO model compatibility
- Histogram equalization for lighting normalization across day and night images
- Noise reduction using Gaussian filtering for enhanced feature extraction
- Data augmentation techniques:
 - Random rotation (± 15 degrees)



Figure 3.3: Electronic weighing system used for ground truth measurements

- Random scaling ($\pm 10\%$)
- Random horizontal flipping
- Mosaic augmentation for YOLO training
- Adaptive histogram equalization for lighting variation

Behavior Analysis Preprocessing

For the behavior analysis datasets, we implemented:

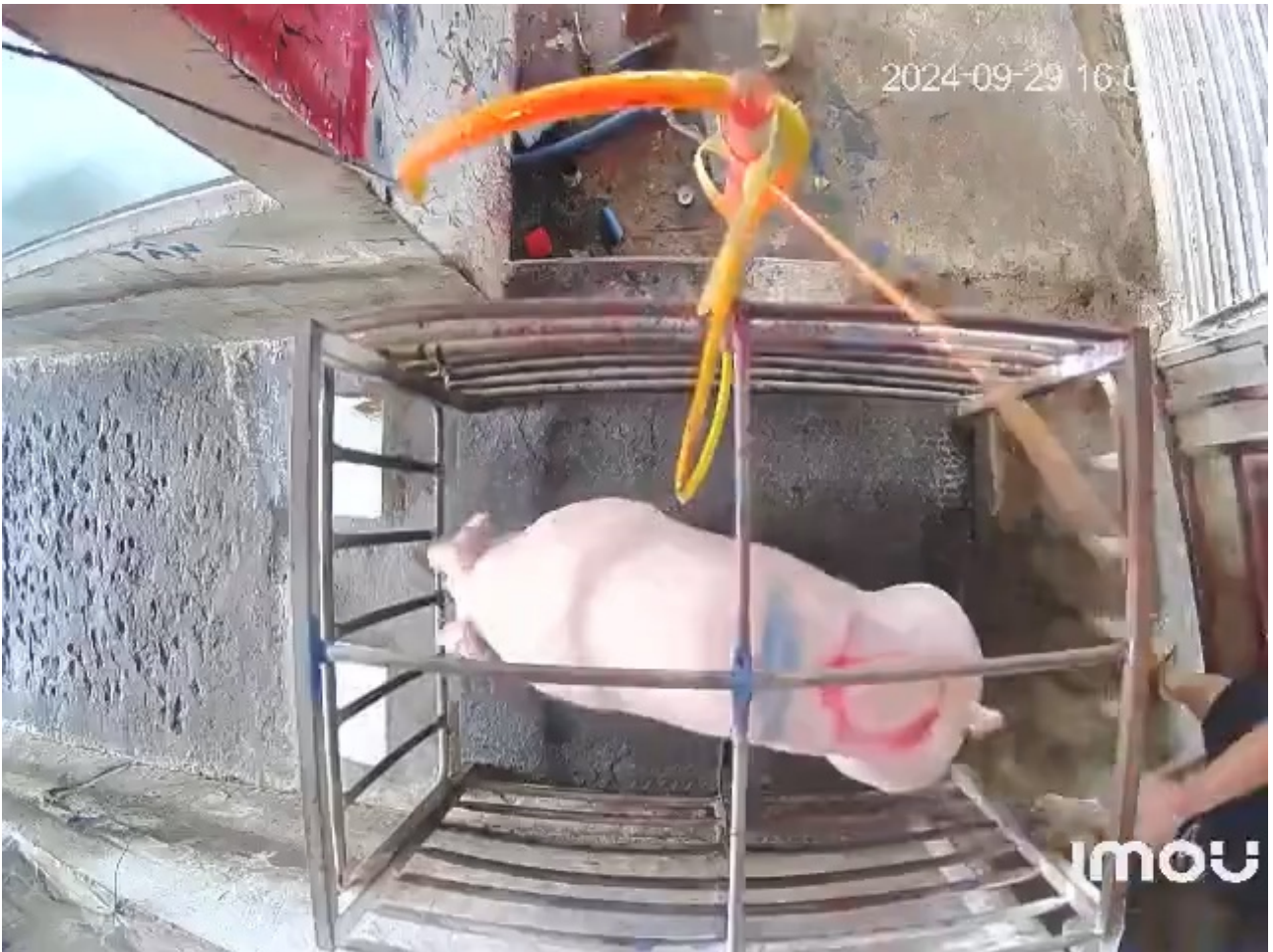


Figure 3.4: Aisle configuration showing camera placement and pig movement path

- Frame extraction at 30 fps for continuous behavior sequences
- Temporal sliding window of 1 second (30 frames) for aggressive behavior detection
- Motion stabilization to compensate for camera movement
- Frame normalization and resizing to 640×640 pixels

Health Monitoring Preprocessing

The health monitoring dataset required specialized preprocessing:

- Resolution standardization to 224×224 pixels for ResNet50v2 compatibility
- Color space normalization to RGB
- Intensity scaling to $[0,1]$ range
- Class-specific augmentation:
 - For sick class: Controlled brightness and contrast adjustment
 - For healthy class: Standard augmentation techniques

Weight Estimation Preprocessing

Weight estimation preprocessing focused on maintaining spatial information:

- Perspective correction based on camera calibration parameters
- Background removal for precise pig segmentation
- Scale normalization using known reference measurements
- Image enhancement:
 - Contrast Limited Adaptive Histogram Equalization (CLAHE)
 - Denoising while preserving edge information
 - Shadow removal for accurate contour detection

3.2.6 Dataset Statistics and Distribution

Table 3.1: Detection and Tracking Dataset Distribution

Characteristic	Training	Validation	Testing	Total
Number of Images	16,000	2,000	2,000	20,000
Day Images	8,743	1,089	1,168	11,000
Night Images	7,257	911	832	9,000
Total Annotations	416,000	50,000	54,000	520,000
Avg. Pigs/Image	26	25	27	26

Table 3.2: Aggressive Behavior Dataset Distribution

Behavior Type	Training	Validation	Testing	Total
Aggressive	4,176	522	522	5,220
Non-aggressive	4,176	522	522	5,220

Table 3.3: Health Monitoring Dataset Distribution

Class	Training	Validation	Testing	Total
Healthy	1,600	200	200	2,000
Sick	1,600	200	200	2,000

All preprocessing steps were implemented using Python, primarily utilizing OpenCV for image processing operations and the albumentations library for data augmentation. The preprocessing pipeline was validated through visual inspection of processed samples and statistical analysis of dataset distributions to ensure consistency and quality across all splits.

3.3 Feature Selection and Engineering

3.3.1 Computer Vision Feature Engineering

The foundation of our pig monitoring system relies on sophisticated computer vision features engineered specifically for the challenging environment of pig farms. Our feature engineering approach addresses multiple challenges including varying lighting conditions, occlusions, similar appearances between subjects, and real-time processing requirements.

Table 3.4: Weight Estimation Dataset Distribution

Weight Range (kg)	Number of Samples
115-120	45
121-125	52
126-130	48
131-135	29
Total	174

Detection Feature Architecture

The detection system employs a hierarchical feature extraction pipeline built upon the YOLOv8/v9 architecture. The feature extraction process occurs through multiple stages:

$$F_{out} = f_{head}(f_{neck}(f_{backbone}(I_{input}))) \quad (3.1)$$

where each component serves a specific purpose in the feature hierarchy:

1. **Backbone Features:** The backbone network implements a modified CSPDarknet architecture that extracts features at multiple scales:

$$F_{backbone} = \{F_3, F_4, F_5\} \quad (3.2)$$

where each feature level corresponds to different receptive fields:

- F_3 : Fine-grained features at $1/8$ resolution (80×80)
- F_4 : Medium-scale features at $1/16$ resolution (40×40)
- F_5 : Large-scale features at $1/32$ resolution (20×20)

2. **Neck Features:** The feature neck implements a Path Aggregation Network (PAN) for bi-directional feature fusion:

$$F_{neck} = PAN(F_{backbone}) = \{P_3, P_4, P_5\} \quad (3.3)$$

This fusion process enhances feature representation through:

- Bottom-up path augmentation for low-level detail preservation
- Top-down path for semantic information propagation
- Adaptive feature fusion using learned weights

3. **Head Features:** The detection head produces final feature maps optimized for pig detection:

$$F_{head} = \{F_{obj}, F_{box}, F_{class}\} \quad (3.4)$$

where:

- F_{obj} : Objectness prediction features
- F_{box} : Bounding box regression features
- F_{class} : Class probability features

Tracking by ByteTrack

ByteTrack represents a significant advancement in multi-object tracking algorithms, particularly suited for maintaining consistent pig identities in farm environments. The architecture implements a sophisticated tracking pipeline that excels in handling occlusions and maintaining stable tracks across varying conditions [10].

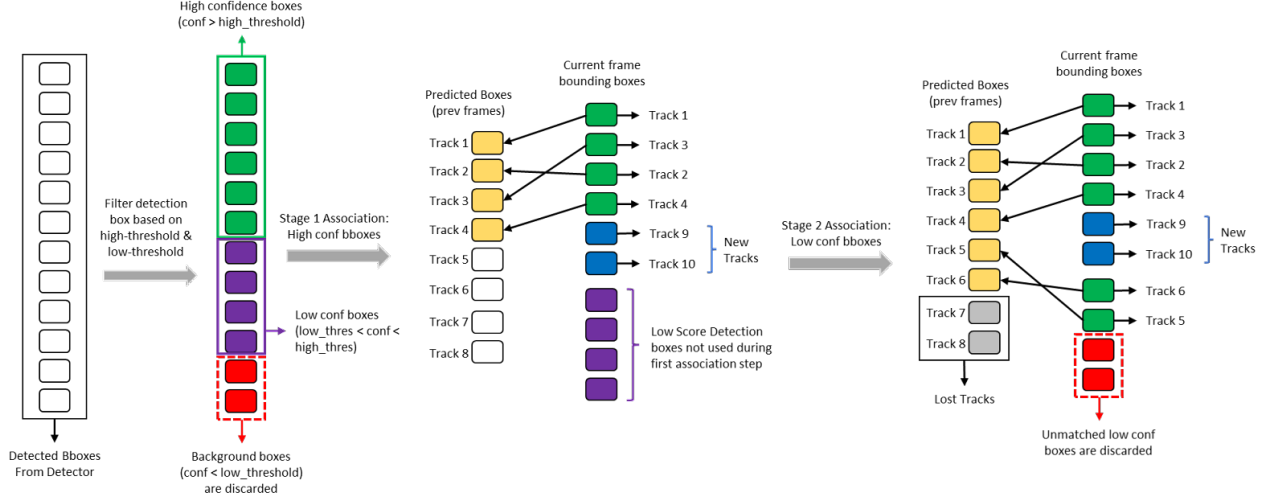


Figure 3.5: ByteTrack architecture showing the two-stage association process and track management system. The architecture employs high-confidence detection association followed by secondary matching for low-confidence detections.

The core architecture consists of three primary components:

1. **BYTE Association Strategy:** Implements a novel two-stage association mechanism that first processes high-confidence detections followed by a secondary association phase for low-confidence detections. This approach significantly improves tracking reliability by:
 - Utilizing detection confidence scores for association prioritization
 - Employing Kalman filtering for motion prediction
 - Implementing Hungarian algorithm for optimal track-detection assignment
2. **Track Management System:** Maintains comprehensive track states through:
 - Four distinct track states: New, Tracked, Lost, and Removed
 - Adaptive track confidence updating based on association history
 - Sophisticated track initialization and termination logic
3. **Motion Model:** Utilizes an 8-dimensional state vector for accurate trajectory prediction:

$$X = [x, y, a, h, v_x, v_y, v_a, v_h] \quad (3.5)$$

where (x, y) represents center position, (a, h) denotes aspect ratio and height, and (v_x, v_y, v_a, v_h) are their respective velocities.

For pig tracking applications, we implement several domain-specific optimizations:

- Modified velocity models to account for typical pig movement patterns

- Enhanced track recovery mechanisms for temporarily occluded pigs
- Implemented pen-specific boundary constraints and ID management
- Added size consistency checks for reliable track association

Motion Feature Extraction

The motion feature extraction system implements a sophisticated pipeline for tracking pig movements over time. This system combines multiple feature types to achieve robust tracking:

$$M_{features} = \{M_{position}, M_{velocity}, M_{acceleration}, M_{pattern}\} \quad (3.6)$$

Each motion feature component is processed through specific algorithms:

1. **Position Features:** Position tracking implements Kalman filtering with an augmented state vector:

$$X_k = [x \quad y \quad v_x \quad v_y \quad a_x \quad a_y \quad w \quad h]^T \quad (3.7)$$

The state transition model incorporates acceleration for improved prediction:

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} & 0 & 0 & 0 \\ 0 & 1 & 0 & \Delta t & 0 & \frac{\Delta t^2}{2} & 0 & 0 \\ 0 & 0 & 1 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.8)$$

2. **Velocity Features:** Velocity estimation employs a multi-stage filtering approach:

$$v_{filtered}(t) = \alpha v_{measured}(t) + (1 - \alpha)v_{filtered}(t - 1) \quad (3.9)$$

with adaptive smoothing parameter α :

$$\alpha = \min\left(\frac{1}{1 + \exp(-\lambda \|v_{measured}\|)}, \alpha_{max}\right) \quad (3.10)$$

3. **Pattern Features:** Movement patterns are analyzed using a temporal sliding window approach:

$$P(t) = \sum_{i=0}^{W-1} w_i \cdot M(t - i) \quad (3.11)$$

where:

- W is the window size (optimized to 10 frames)
- w_i are gaussian weights favoring recent observations
- $M(t)$ represents motion metrics at time t

Spatial Feature Engineering

The spatial feature engineering component implements comprehensive analysis of pig positions and spatial relationships within the farm environment. This system processes both individual pig locations and group spatial patterns:

$$S_{features} = \{S_{individual}, S_{group}, S_{zone}, S_{interaction}\} \quad (3.12)$$

Each spatial component incorporates specific feature extraction methods:

1. **Individual Spatial Features:** Position encoding utilizes a normalized coordinate system:

$$p_{normalized} = \begin{bmatrix} \frac{x-x_{min}}{x_{max}-x_{min}} \\ \frac{y-y_{min}}{y_{max}-y_{min}} \end{bmatrix} \quad (3.13)$$

Area and shape features are computed through:

$$\begin{aligned} A_{ratio} &= \frac{w \cdot h}{A_{reference}} \\ AR &= \frac{w}{h} \\ O_{ratio} &= \frac{A_{visible}}{w \cdot h} \end{aligned} \quad (3.14)$$

where:

- A_{ratio} represents normalized area ratio
- AR is aspect ratio for posture analysis
- O_{ratio} measures occlusion level

2. **Group Spatial Features:** Group distribution analysis implements:

$$D_{group} = [d_{min} \quad d_{max} \quad d_{mean} \quad \sigma_d] \quad (3.15)$$

where distances are computed using:

$$d_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2 \quad (3.16)$$

Spatial density features include:

$$\rho(x, y) = \sum_{i=1}^N K_h(\|\mathbf{p} - \mathbf{p}_i\|) \quad (3.17)$$

where K_h is a kernel density estimator with bandwidth h .

3. **Zone Interaction Features:** Zone analysis implements polygon-based overlap computation:

$$O_{zone}(p, z) = \frac{Area(B_p \cap Z)}{Area(B_p)} \quad (3.18)$$

with temporal aggregation:

$$T_{zone}(t) = \sum_{i=t-W}^t I_{zone}(i) \cdot w(t-i) \quad (3.19)$$

where:

- B_p is the pig's bounding box
- Z represents zone polygons
- I_{zone} is the zone presence indicator
- $w(t)$ is a temporal weighting function

Appearance Feature Extraction

The appearance feature extraction system implements multiple complementary approaches to capture distinctive visual characteristics of individual pigs:

$$A_{features} = \{A_{deep}, A_{texture}, A_{shape}, A_{color}\} \quad (3.20)$$

1. **Deep Appearance Features:** Feature extraction using convolutional layers:

$$f_{deep} = \phi(CNN(I_{roi})) \quad (3.21)$$

where ϕ represents feature pooling operations:

$$\phi(X) = \begin{cases} \text{GAP}(X) & \text{for global features} \\ \text{SPP}(X) & \text{for multi-scale features} \end{cases} \quad (3.22)$$

2. **Texture Features:** Multi-scale texture analysis:

$$T_{features} = \{LBP(I_\sigma), HOG(I), Gabor(I)\} \quad (3.23)$$

where:

- $LBP(I_\sigma)$ computes local binary patterns at scale σ
- $HOG(I)$ extracts histogram of oriented gradients
- $Gabor(I)$ applies Gabor filter banks

3. **Shape Features:** Contour-based shape description:

$$S_{features} = [A_{norm} \quad P_{ratio} \quad C_{complexity} \quad E_{ratio}] \quad (3.24)$$

where:

- A_{norm} is normalized area
- P_{ratio} is perimeter ratio
- $C_{complexity}$ measures contour complexity
- E_{ratio} is elliptical fit ratio

Behavior Pattern Features

The behavior pattern analysis system implements temporal feature extraction across multiple time scales:

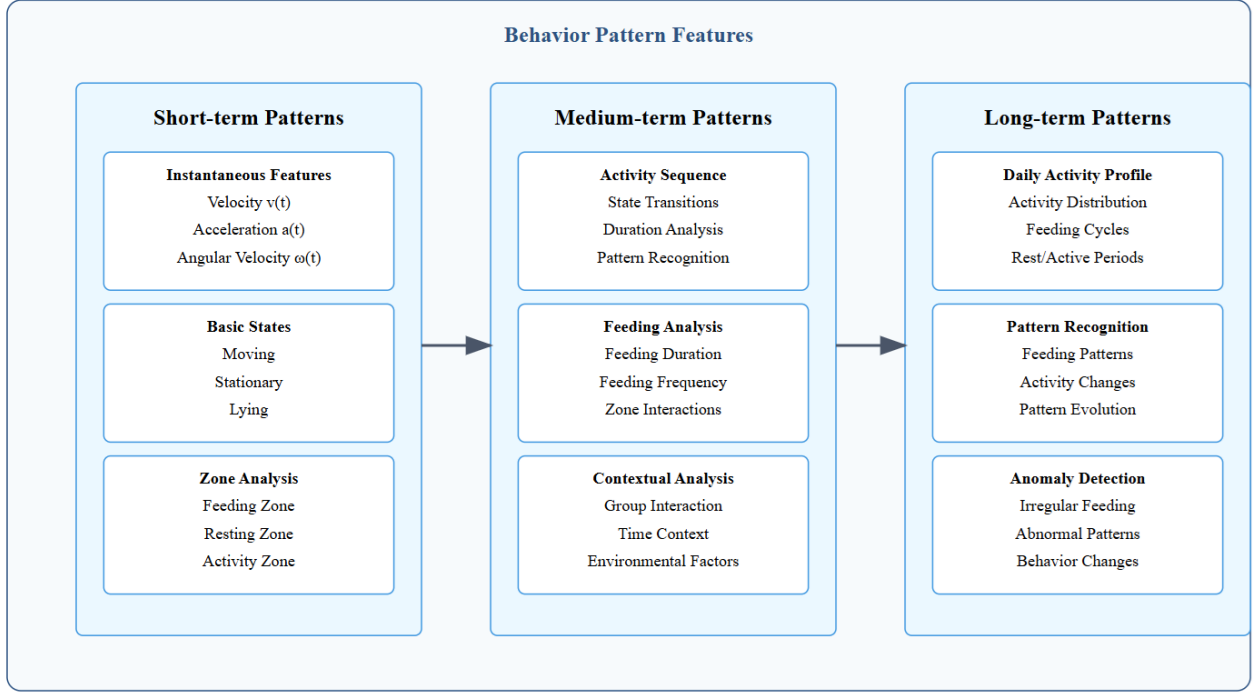


Figure 3.6: Behavior Pattern Features

$$B_{features} = \{B_{short}, B_{medium}, B_{long}\} \quad (3.25)$$

Each temporal scale captures different aspects of behavior:

1. **Short-term Patterns**: Immediate behavior analysis using frame-to-frame features:

$$B_{short}(t) = \begin{bmatrix} v(t) \\ a(t) \\ \omega(t) \\ p(t) \end{bmatrix} \quad (3.26)$$

where:

- $v(t)$ is instantaneous velocity
- $a(t)$ is acceleration
- $\omega(t)$ is angular velocity
- $p(t)$ is posture descriptor

2. **Medium-term Patterns**: Activity sequence analysis over windows:

$$B_{medium}(t) = \{(s_i, d_i, c_i)\}_{i=t-W}^t \quad (3.27)$$

where:

- s_i represents behavioral state
- d_i is duration in state
- c_i is confidence score

State transition probability matrix:

$$P_{transition} = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & p_{22} & \cdots & p_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{bmatrix} \quad (3.28)$$

3. **Long-term Patterns:** Daily activity profile analysis:

$$B_{long}(h) = \frac{1}{T} \sum_{t \in h} A(t) \quad (3.29)$$

where:

- h represents hour of day
- $A(t)$ is activity level
- T is normalization period

3.3.2 Feature Integration and System Components

The integration of multiple feature types into a cohesive system requires sophisticated fusion strategies and careful synchronization mechanisms. Our system implements a hierarchical integration approach that combines features at multiple levels while maintaining real-time performance on edge hardware.

Feature Fusion Architecture

The feature fusion system implements a multi-level integration strategy:

$$F_{integrated} = f_{fusion}(F_{spatial}, F_{temporal}, F_{behavioral}) \quad (3.30)$$

The fusion process occurs through several stages:

1. **Early Fusion:** Raw feature combination at the input level:

$$F_{early} = \begin{bmatrix} F_{spatial} \\ F_{temporal} \\ F_{context} \end{bmatrix} \quad (3.31)$$

This stage implements adaptive weighting:

$$w_i = \frac{\exp(\lambda_i)}{\sum_j \exp(\lambda_j)} \quad (3.32)$$

where λ_i are learned importance factors for each feature type.

2. **Intermediate Fusion:** Feature integration at the processing level:

$$F_{mid} = \phi(W_1 F_1 + W_2 F_2 + \dots + W_n F_n) \quad (3.33)$$

utilizing attention mechanisms:

$$A_{feature} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.34)$$

where:

- Q represents query features
- K represents key features
- V represents value features
- d_k is the feature dimension

3. **Late Fusion:** Decision-level integration:

$$D_{final} = \sum_{i=1}^N \alpha_i D_i \quad (3.35)$$

with confidence-based weighting:

$$\alpha_i = \frac{c_i}{\sum_{j=1}^N c_j} \quad (3.36)$$

where c_i represents confidence scores for each decision component.

Temporal Integration

The temporal integration system manages the synchronization of features across different time scales:

$$T_{integrated}(t) = f_{temporal}(H_{short}(t), H_{medium}(t), H_{long}(t)) \quad (3.37)$$

This involves multiple processing stages:

1. **Short-term Integration:** Frame-level feature synchronization:

$$H_{short}(t) = \sum_{i=0}^{W_s} w_i F(t - i) \quad (3.38)$$

with exponential decay weights:

$$w_i = \exp\left(-\frac{i}{\tau_s}\right) \quad (3.39)$$

2. **Medium-term Integration:** Behavioral sequence integration:

$$H_{medium}(t) = LSTM(H_{short}(t - W_m : t)) \quad (3.40)$$

incorporating state memory:

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\ C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \end{aligned} \quad (3.41)$$

3. **Long-term Integration:** Historical pattern integration:

$$H_{long}(t) = \frac{1}{T} \sum_{i=1}^T H_{medium}(t - i\Delta t) \quad (3.42)$$

with adaptive time windows:

$$T_{adaptive} = T_{base} \cdot (1 + \gamma \cdot \sigma(H)) \quad (3.43)$$

System Synchronization

The synchronization system ensures coherent operation across all components:

$$S_{system} = \{S_{detection}, S_{tracking}, S_{behavior}, S_{analysis}\} \quad (3.44)$$

Implementation includes:

1. **Time Synchronization:** Global time management:

$$t_{sync} = t_{base} + \Delta t_{offset} + \epsilon_{correction} \quad (3.45)$$

where:

- t_{base} is system base time
- Δt_{offset} is component-specific offset
- $\epsilon_{correction}$ is drift correction

2. **Data Synchronization:** Buffer management system:

$$B_{sync}(t) = \{(d_i, t_i) | |t_i - t| \leq \Delta t_{max}\} \quad (3.46)$$

with interpolation for missing data:

$$d_{interp}(t) = \alpha d(t_1) + (1 - \alpha) d(t_2) \quad (3.47)$$

where $\alpha = \frac{t_2 - t}{t_2 - t_1}$

3.3.3 Pig ID Management System Implementation

The ID management system serves as a critical component in our Smart Farm Management System, handling the complex task of maintaining consistent pig identities across different pens while dealing with occlusions and similar appearances. This system operates through a sophisticated combination of spatial tracking, appearance matching, and motion prediction algorithms.

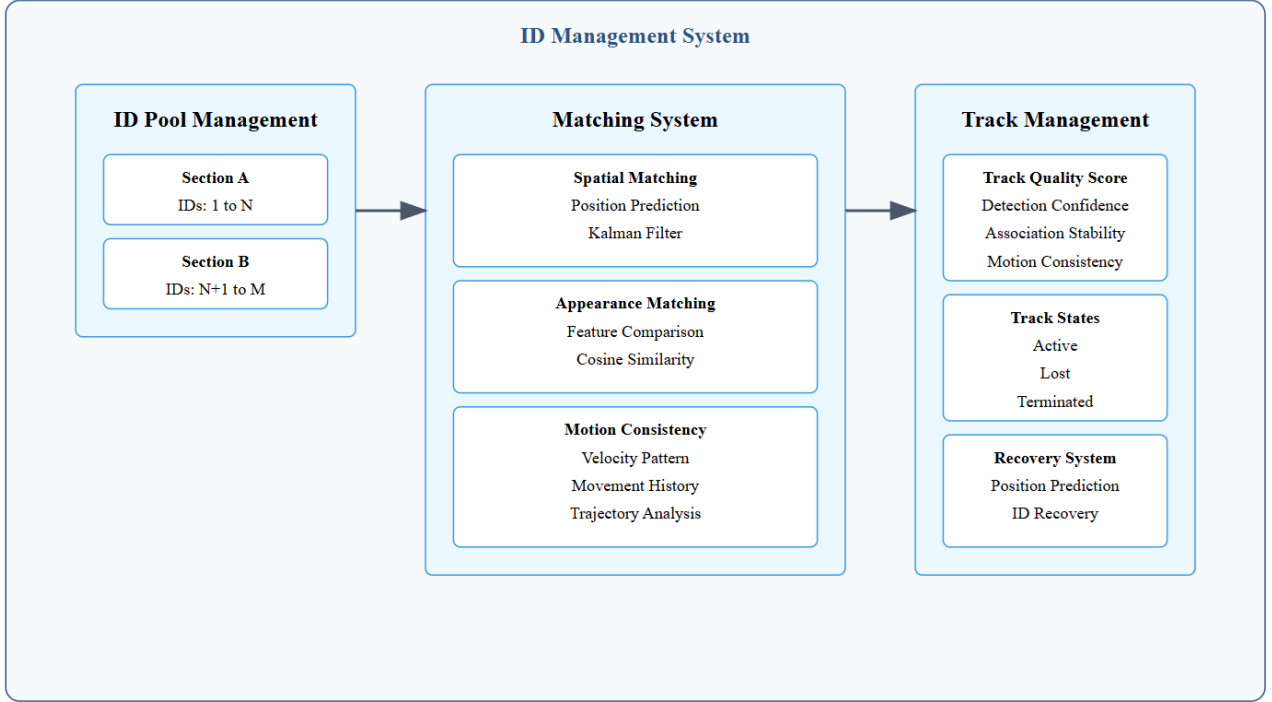


Figure 3.7: ID Management System

At its core, the ID management system utilizes a hierarchical tracking approach. Each pen maintains its own ID pool, with the left pen managing IDs from 1 to 27 and the right pen managing IDs from 28 to 54. This separation prevents ID confusion between pens and allows for more efficient management of the tracking process. The system maintains active and inactive ID pools, which are dynamically updated based on real-time observations and tracking confidence levels.

The ID assignment process employs a multi-stage matching algorithm. When a new detection occurs, the system first attempts to match it with existing tracked IDs using a combination of spatial and appearance features. The spatial matching considers both the current position and predicted position based on previous motion patterns. This prediction is accomplished through a Kalman filter implementation that maintains state vectors for each tracked pig:

$$X_k = F_k X_{k-1} + w_k \quad (3.48)$$

where X_k represents the state vector containing position and velocity components, F_k is the state transition matrix incorporating motion models specific to pig movement patterns, and w_k represents process noise.

The appearance matching utilizes a sophisticated feature comparison mechanism. For each detection, the system extracts appearance features using deep embeddings from the backbone network. These features are compared with stored features from previous frames using a cosine similarity metric:

$$similarity(f_1, f_2) = \frac{f_1 \cdot f_2}{\|f_1\| \|f_2\|} \quad (3.49)$$

The system combines these similarity scores with motion consistency checks to create a robust matching criterion. This combination helps maintain stable tracking even when pigs are moving in close proximity or partially occluded. The matching process is further enhanced by incorporating temporal consistency checks that examine the history of movements and behaviors for each tracked ID.

The track management component implements sophisticated logic for handling track initialization, maintenance, and termination. New tracks are initialized only when a detection cannot be associated with any existing track and meets specific quality criteria. This initialization process includes a probationary period during which the track must demonstrate consistent detection and motion patterns before being confirmed as a valid track.

Track maintenance involves continuous quality assessment based on multiple factors. The system calculates a track quality score that combines detection confidence, association stability, and motion consistency:

$$Q_{track} = \alpha Q_{detection} + \beta Q_{association} + \gamma Q_{motion} \quad (3.50)$$

where:

- $Q_{detection}$ represents the confidence of object detection
- $Q_{association}$ measures the stability of ID associations over time
- Q_{motion} evaluates the consistency of movement patterns
- α , β , and γ are weights optimized through empirical testing

The system handles occlusions and temporary disappearances through a sophisticated recovery mechanism. When a pig disappears from view, its track is maintained in memory for a configurable duration, during which the system continues to predict its position using the motion model. If a new detection appears that matches the predicted characteristics of the lost track, the system can recover the original ID assignment.

3.3.4 Escape Detection System Implementation

The escape detection system implements a comprehensive approach to monitoring pig movements and identifying potential escape scenarios. This system operates continuously, processing real-time video feeds to detect when pigs enter restricted zones or display behavior patterns indicative of escape attempts.

The core of the escape detection system relies on a sophisticated zone monitoring algorithm. The restricted zones are defined using polygon representations, allowing for flexible definition of monitored areas. The system performs efficient polygon overlap calculations to determine when a pig enters a restricted area:

$$Overlap_{ratio} = \frac{Area(B_{pig} \cap Zone_{restricted})}{Area(B_{pig})} \quad (3.51)$$

where B_{pig} represents the bounding box of a detected pig and $Zone_{restricted}$ represents the polygon defining the restricted area.

The system implements temporal analysis of zone occupation to differentiate between momentary boundary crosses and genuine escape attempts. This analysis involves tracking the

duration of zone occupation and the pattern of movement near restricted areas. The alert generation process follows a state machine model:

$$State_{escape} = \begin{cases} Warning & \text{if } t_{zone} > T_{warning} \\ Alert & \text{if } t_{zone} > T_{alert} \\ Normal & \text{otherwise} \end{cases} \quad (3.52)$$

The escape detection system incorporates advanced filtering mechanisms to reduce false alarms while maintaining high sensitivity to genuine escape attempts. This is achieved through a multi-stage verification process that considers both spatial and temporal aspects of pig behavior.

When a potential escape event is detected, the system initiates a sequence of verification steps:

$$Alert_{confidence} = P(escape|D) = \frac{P(D|escape)P(escape)}{P(D)} \quad (3.53)$$

where D represents the observed data including position, velocity, and historical behavior patterns. The system updates these probabilities using a Bayesian approach to incorporate temporal information:

$$P(escape_t|D_{1:t}) \propto P(D_t|escape_t) \sum_{escape_{t-1}} P(escape_t|escape_{t-1})P(escape_{t-1}|D_{1:t-1}) \quad (3.54)$$

This probabilistic framework allows the system to maintain a balance between sensitivity and specificity in escape detection. The system implements adaptive thresholding based on historical patterns and environmental conditions:

$$T_{adaptive}(t) = T_{base} + \Delta T(h(t), n(t)) \quad (3.55)$$

where $h(t)$ represents the time of day and $n(t)$ accounts for the current activity level in the monitored area.

3.3.5 Behavior Analysis System Implementation

The behavior analysis system employs a sophisticated approach to understanding and categorizing pig activities in real-time. This system operates through a hierarchical processing pipeline that combines low-level motion features with high-level behavioral pattern recognition.

Motion Analysis and State Classification

The primary state classification system implements a continuous-time Markov model for behavior transitions:

$$P(s_{t+1}|s_t) = \begin{bmatrix} p_{mm} & p_{ms} & p_{mf} & p_{ml} \\ p_{sm} & p_{ss} & p_{sf} & p_{sl} \\ p_{fm} & p_{fs} & p_{ff} & p_{fl} \\ p_{lm} & p_{ls} & p_{lf} & p_{ll} \end{bmatrix} \quad (3.56)$$

where:

- s_t represents the state at time t (moving, stationary, feeding, lying)
- p_{ij} represents the transition probability from state i to state j

- Transition probabilities are continuously updated based on observed patterns

The system implements velocity-based state determination through a sophisticated filtering process:

$$v_{filtered}(t) = \frac{\sum_{i=0}^{N-1} w_i v(t-i)}{\sum_{i=0}^{N-1} w_i} \quad (3.57)$$

where:

$$w_i = \exp\left(-\frac{i^2}{2\sigma^2}\right) \quad (3.58)$$

This filtered velocity is used in conjunction with spatial and temporal features to determine the current behavioral state:

$$State(t) = \underset{s}{\operatorname{argmax}} P(s|v_{filtered}(t), p(t), h(t)) \quad (3.59)$$

where $p(t)$ represents position information and $h(t)$ represents historical behavior patterns.

Activity Pattern Recognition

The system implements pattern recognition through a sliding window approach that captures temporal dependencies in behavior sequences. For each pig, the system maintains an activity history buffer:

$$H_t = \{(s_i, d_i, a_i)\}_{i=t-N}^t \quad (3.60)$$

where:

- s_i represents the behavioral state
- d_i represents the duration in that state
- a_i represents associated activity metrics

Pattern analysis is performed using a temporal convolution approach:

$$Pattern_{score}(t) = \sum_{i=0}^{W-1} K(i) \cdot Feature(t-i) \quad (3.61)$$

where $K(i)$ represents the kernel function that weights the importance of historical observations, and W is the window size optimized for different behavior patterns.

Statistical Analysis and Reporting

The statistical analysis component maintains running statistics for various behavioral metrics:

$$\mu_{online}(t) = \mu_{online}(t-1) + \frac{x_t - \mu_{online}(t-1)}{n} \quad (3.62)$$

$$\sigma_{online}^2(t) = \sigma_{online}^2(t-1) + \frac{(x_t - \mu_{online}(t-1))(x_t - \mu_{online}(t))}{n} \quad (3.63)$$

These statistics are maintained separately for each behavioral state and are used to detect anomalous patterns through a Z-score approach:

$$Z_{score}(x_t) = \frac{x_t - \mu_{online}(t)}{\sigma_{online}(t)} \quad (3.64)$$

The system generates comprehensive activity reports that include:

- Daily activity distributions
- State transition frequencies
- Duration statistics for each state
- Cross-correlation analysis between different pigs
- Anomaly detection results

Real-time Processing Optimization

To maintain real-time performance on the Jetson Nano platform, the system implements several optimization strategies:

$$Performance_{score} = \frac{\alpha}{t_{processing}} + \frac{\beta}{memory_{usage}} + \frac{\gamma \cdot accuracy}{baseline_{accuracy}} \quad (3.65)$$

where α , β , and γ are weights that balance processing speed, memory efficiency, and accuracy.

3.3.6 Weight Estimation

Weight Estimation Flow

Our Weight Estimation Flow as shown in Figure 3.8: consists of 4 stages including: Data Processing, Instance Segmentation, Keypoint Detection and Body Weight Estimation.

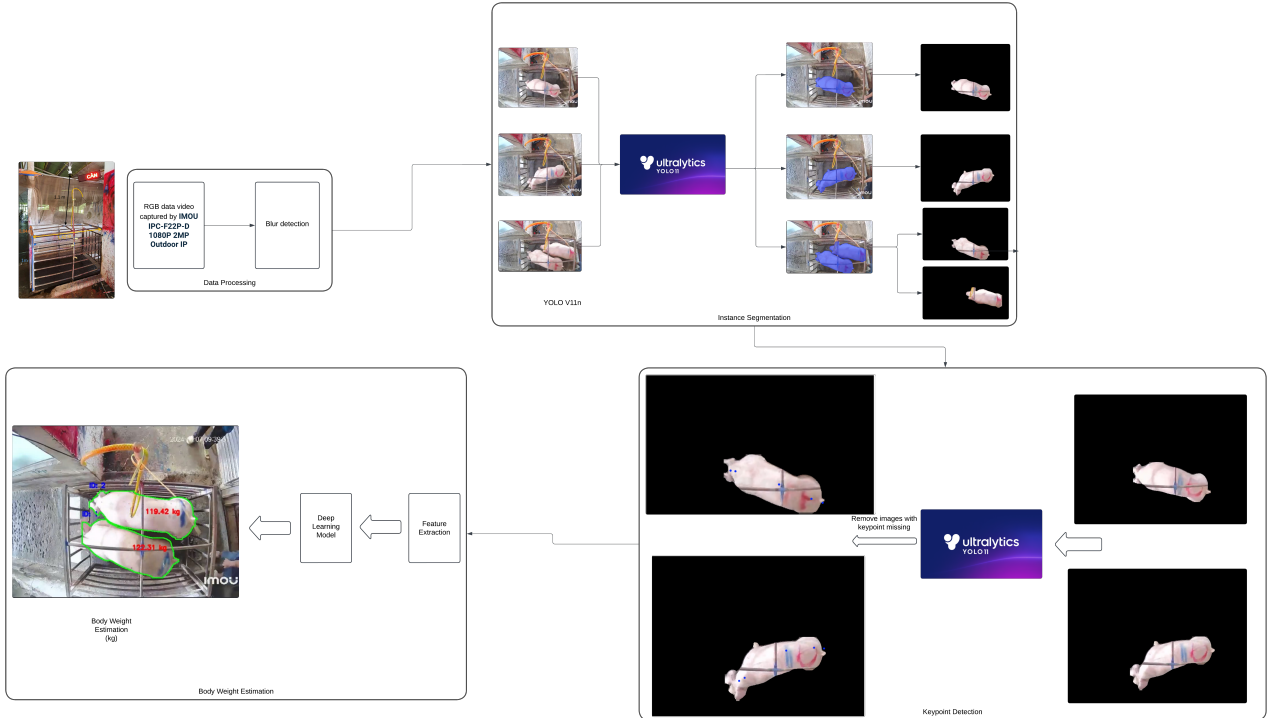


Figure 3.8: Demonstration of Weight Estimation Flow. The process flow shows: (left) input images of pigs in different poses, (center) YOLOv11 processing with segmentation masks highlighted in blue, and (right) the resulting individual mask outputs. This illustrates the model's ability to accurately segment individual pigs even in varying positions and lighting conditions.

Blur Detection

The quality of images plays a vital role in achieving accurate weight estimation in pigs. Following the methodology outlined by Wang et al. (2023), a Laplacian-based algorithm is utilized to detect and filter blurry images from the data set. The steps involve computing the Laplacian sharpness value, normalizing these values, determining the optimal sharpness threshold, and filtering images based on this threshold.

Algorithm 1 Image Blur Detection Algorithm For Finding Optimal β

Require: Dataset of images $I = \{f_1, f_2, \dots, f_n\}$

Ensure: Optimal β^*

0: **for** each image $f \in I$ **do**

0: Compute the Laplacian operator for all $x, y \in f$:

$$L(x, y) = \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2}$$

0: Compute the sharpness value:

$$D(f) = \sum_x \sum_y |L(x, y)|, \quad \forall x, y \in f$$

0: **end for**

0: Normalize the sharpness values $\{D(f)\}$ to the range $[0, 1]$:

$$\beta(f) = \frac{D(f) - \min(D(f))}{\max(D(f)) - \min(D(f))}$$

0: Define sharpness thresholds $\beta \in (0, 1)$, and calculate $Q(\beta)$:

$$Q(\beta) = \max(\log(N(\beta) \cdot G(\beta)))$$

0: Determine the optimal threshold β^* that maximizes $Q(\beta)$.

0: **return** β^*

where:

- $N(\beta)$ indicates the number of images with a sharpness value greater than β .
- $G(\beta)$ indicates the average value of the sharpness of images with sharpness greater than β .

=0

This algorithm ensures that blurry images are excluded from the dataset, improving the quality of images used for weight estimation. The optimal threshold β^* for determining image clarity is $\beta^* = 0.01$ through our experiment as shown in figure 3.9 and figure 3.10.

YOLOv11 Instance Segmentation

Instance segmentation in YOLOv11 represents a significant advancement in real-time object analysis, particularly for complex scenarios like individual pig monitoring. The architecture extends beyond traditional object detection to provide pixel-level precision while maintaining the high-speed performance characteristic of the YOLO family. The instance segmentation

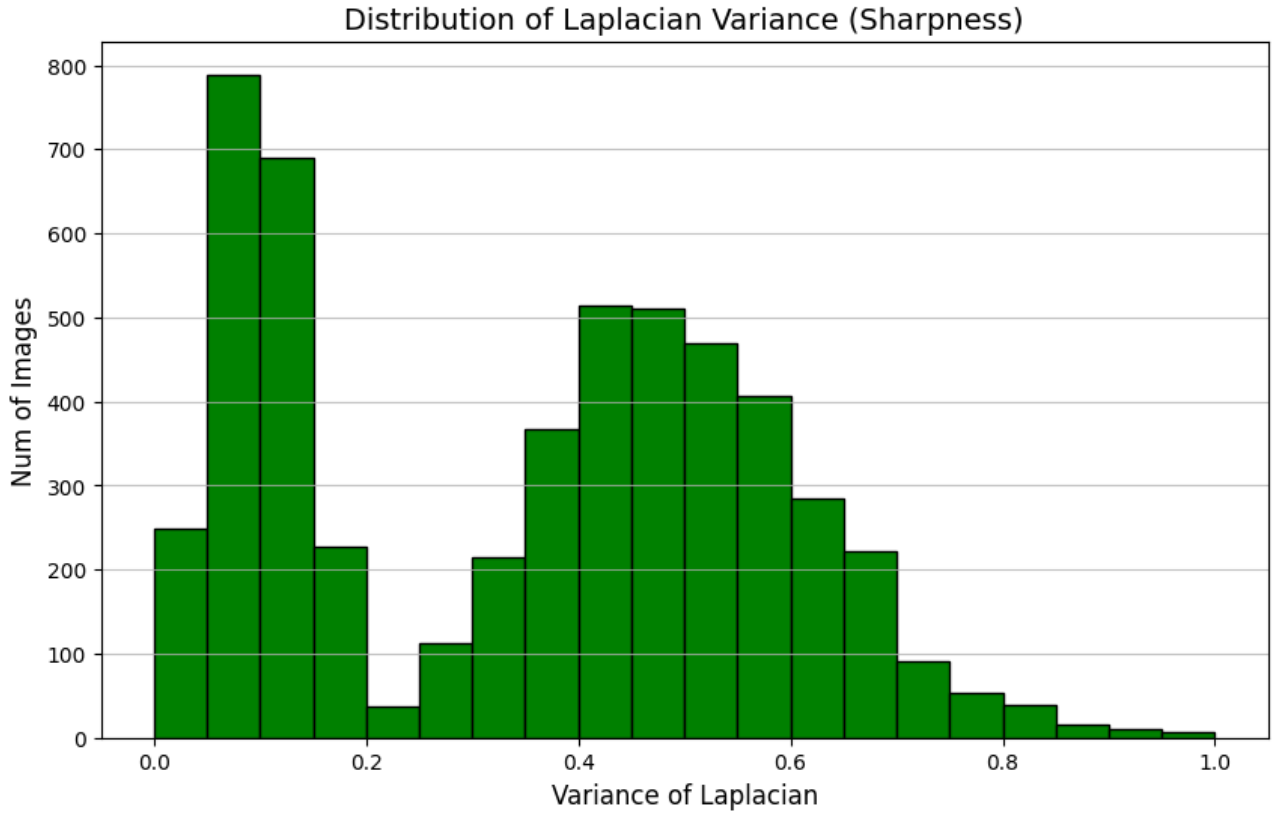


Figure 3.9: Distribution of Laplacian function value over dataset

pipeline in YOLOv11 builds upon its base architecture by incorporating sophisticated pixel-level prediction capabilities. This integration is achieved through a carefully designed modification of the detection head, which now produces dual outputs: precise bounding box coordinates and detailed segmentation masks. The architecture’s efficiency lies in its ability to process both tasks simultaneously, leveraging shared feature representations for enhanced performance. A key innovation in YOLOv11’s segmentation approach is the introduction of a specialized segmentation head that operates in parallel with the detection pathway. This head utilizes fully convolutional networks (FCNs) to generate dense segmentation maps, enabling pixel-wise classification for each detected object. The architecture incorporates advanced post-processing techniques, including mask refinement and non-maximum suppression (NMS), to ensure precise object delineation and prevent overlapping predictions.

YOLOv11 Keypoint Detection

The keypoint detection capabilities of YOLOv11 address the challenging problem of occlusion and partial visibility in complex scenarios. In the context of pig monitoring, the architecture implements a sophisticated five-point keypoint system, identifying crucial anatomical landmarks: head, neck, back, rump, and tail. This approach draws inspiration from human pose estimation techniques while being specifically adapted for animal morphology. The keypoint detection framework extends YOLOv11’s base architecture by incorporating a specialized keypoint detection head. This modification enables the model to predict precise coordinate locations for each defined keypoint while maintaining the network’s efficient single-stage detection paradigm. The system’s ability to identify and track these key anatomical points proves invaluable for assessing completeness and visibility in partially occluded scenarios, particularly in confined spaces like hallways where multiple subjects may overlap.

This integrated approach to keypoint detection not only facilitates accurate body mass

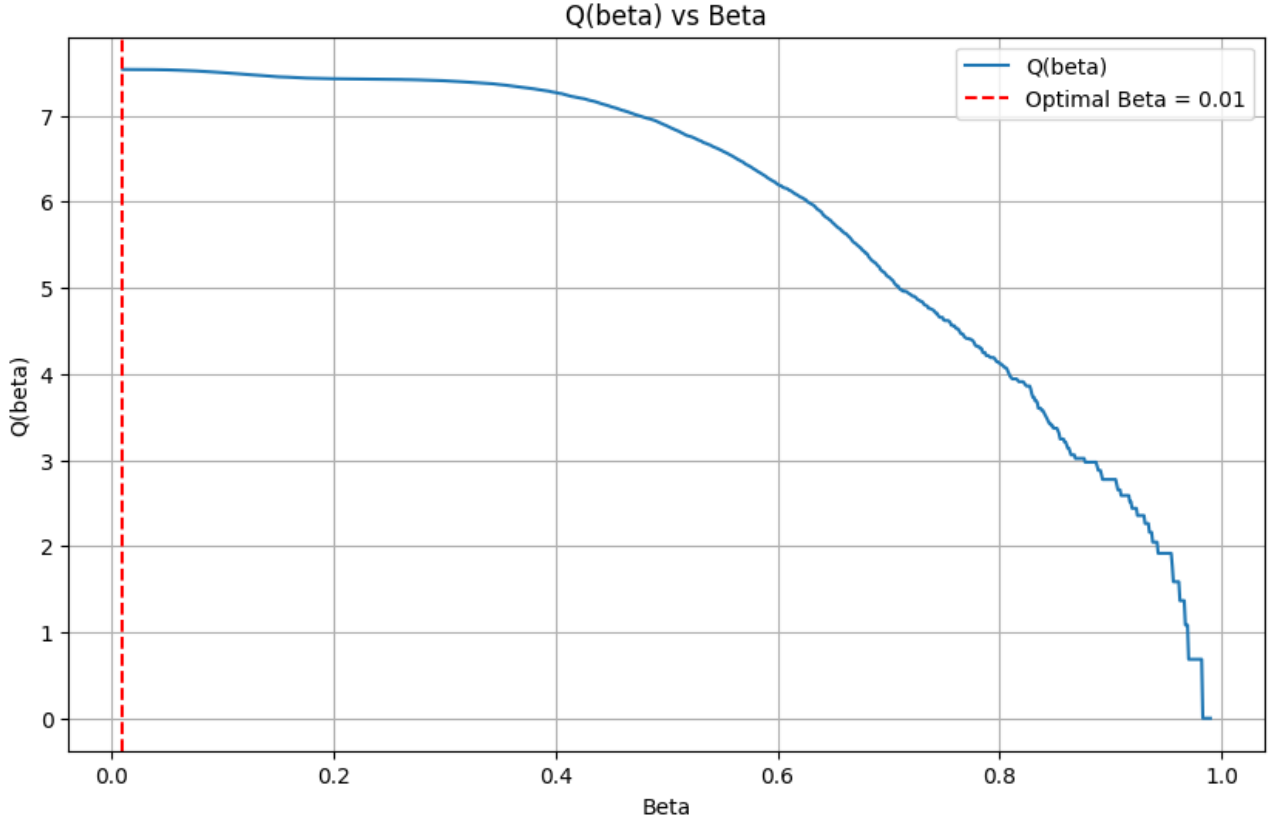


Figure 3.10: Value of Qbeta over beta through dataset

estimation but also provides a robust mechanism for qualifying image suitability for further analysis. By combining these keypoint predictions with the model's segmentation capabilities, YOLOv11 delivers a comprehensive solution for detailed animal monitoring and analysis.

Deep Learning Model

Features extraction

In our deep learning model, the input is extracted from a set of points of segmentation of each pig. All of these features are used to determine each pig specifically. These features include: **Area, Minor Axis Length, Major Axis Length, Region Eccentricity, Centroid X Coordinate, Centroid Y Coordinate, Percentage Area, Energy Score.**

Area

Since the set of segmentation of pigs formed a simple polygon shape, Shoelace's formula can be applied to calculate the area. To calculate the area of the simple polygon, the algorithm requires points to have a clockwise orientation or counter-clockwise orientation. In general, we use clockwise oriented direction.

Clockwise oriented sorting

Assume N is the number of given points, and each point P_i has coordinates (x_i, y_i) . A centroid point of N points is an arithmetic mean of N points, which is represented by this formula:

$$x_{\text{centroid}} = \frac{1}{N} \sum_{i=1}^N x_i, \quad y_{\text{centroid}} = \frac{1}{N} \sum_{i=1}^N y_i$$

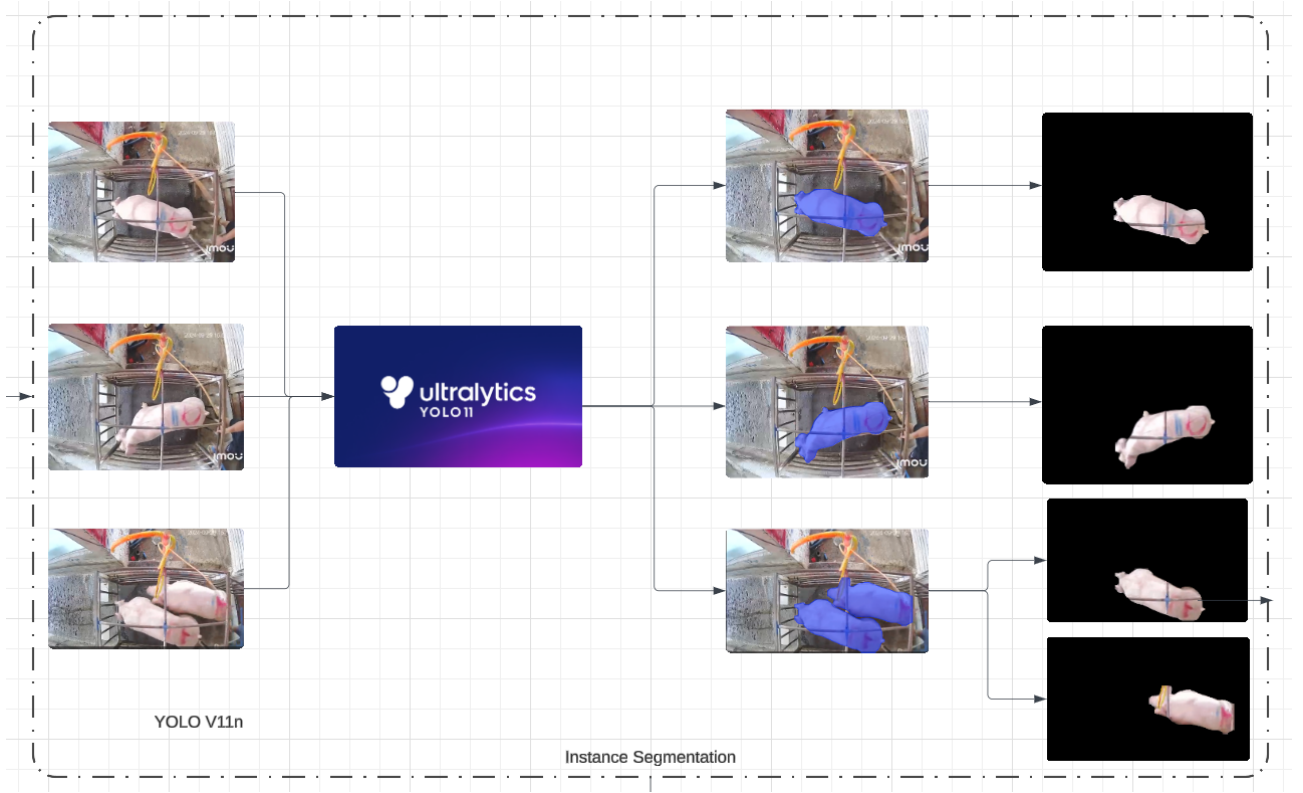


Figure 3.11: Demonstration of YOLOv11's instance segmentation pipeline on pig monitoring. The process flow shows: (left) input images of pigs in different poses, (center) YOLOv11 processing with segmentation masks highlighted in blue, and (right) the resulting individual mask outputs. This illustrates the model's ability to accurately segment individual pigs even in varying positions and lighting conditions.

Set a new Cartesian Oxy plane axis with the center at point $C(x_{\text{centroid}}, y_{\text{centroid}})$, which is equivalent to moving the coordinate system origin from $(0, 0)$ to $(x_{\text{centroid}}, y_{\text{centroid}})$. From the new origin coordinate system, we calculate the vector from point C to point P_i as shown in the formula below:

$$\vec{CP}_i = \begin{pmatrix} x_i - x_{\text{centroid}} \\ y_i - y_{\text{centroid}} \end{pmatrix}, \quad \text{for } i \in [1, N]$$

The slope of the vector is used to determine the order of sorting, which is depicted as the formula:

$$\theta_i = \arctan \left(\frac{y_i - y_{\text{centroid}}}{x_i - x_{\text{centroid}}} \right)$$

Since the derivative of the tangent function is always positive, the tangent function is always increasing, as shown in the graph:

$$\frac{d(\tan(x))}{dx} = \frac{1}{\cos^2(x)}, \quad \frac{1}{\cos^2(x)} > 0$$

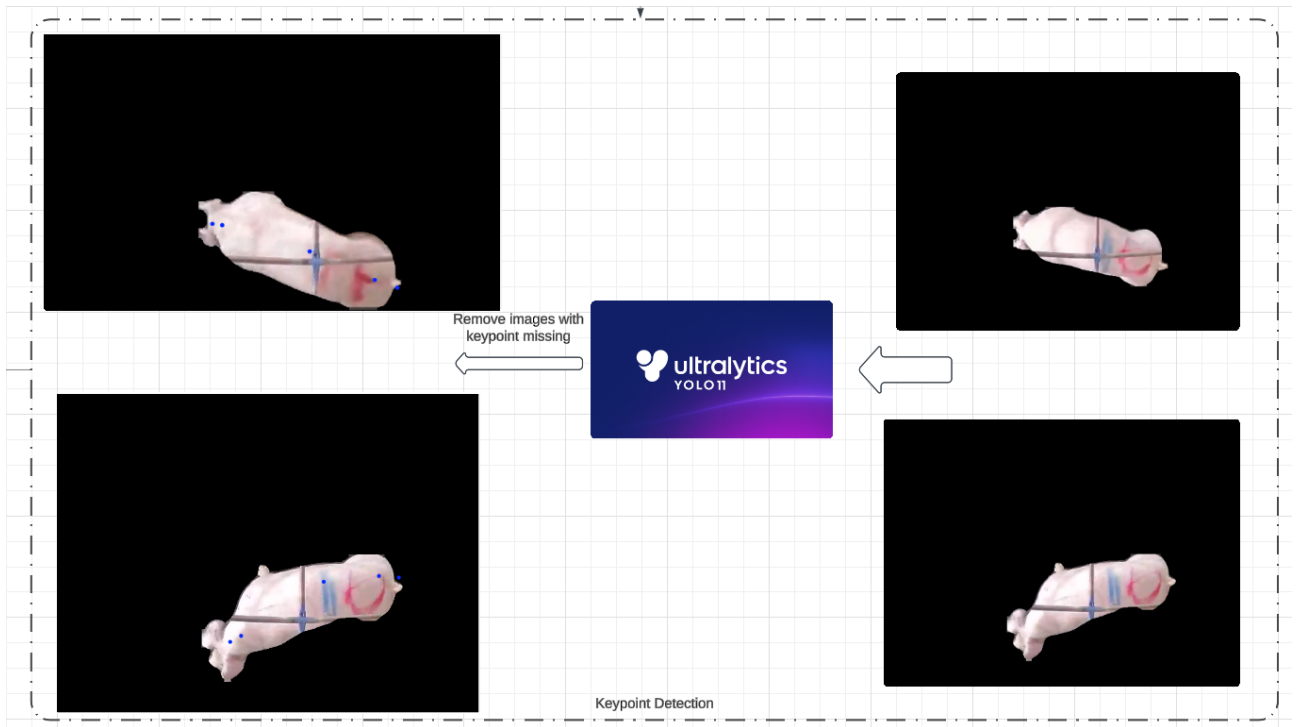


Figure 3.12: YOLOv11 keypoint detection framework applied to pig monitoring with anatomical landmark identification. The diagram illustrates the model's capability to detect five key anatomical points (head, neck, back, rump, and tail) marked in blue. The bidirectional arrows indicate the model's filtering process to remove images with missing keypoints, ensuring data quality for subsequent analysis.

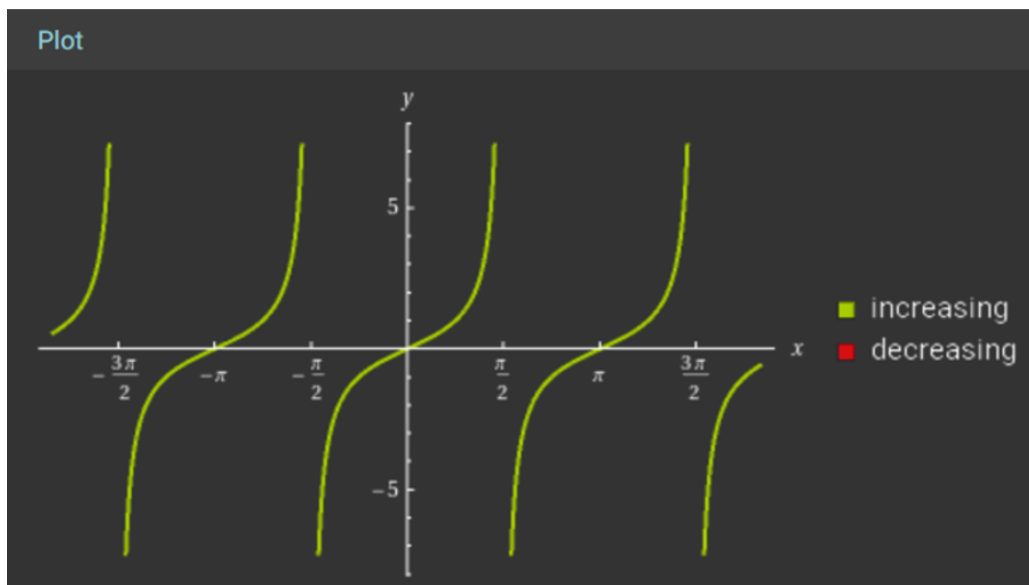


Figure 3.13: Plot of the tangent function showing it is always increasing.

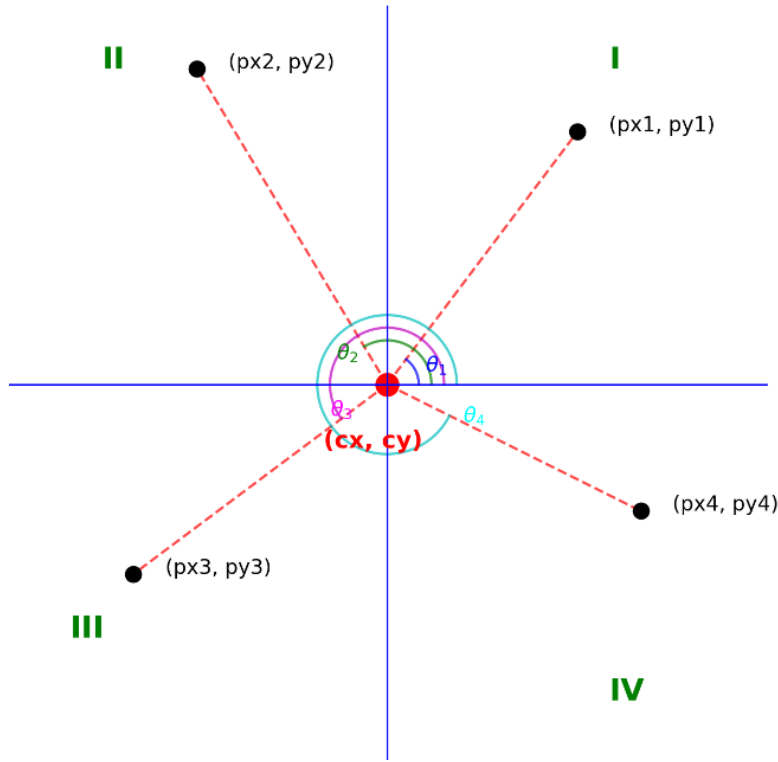


Figure 3.14: Illustration of the coordinate system with angles and points. The center is at (cx, cy) , and the points $(px1, py1)$, $(px2, py2)$, $(px3, py3)$, $(px4, py4)$ are represented in quadrants I, II, III, and IV, respectively.

As shown in the figure 3.2, all of the angles are calculated from quadrant I, which forms the counter-clockwise order. To ensure the angles are arranged in a counter-clockwise order, we need to negate all of the angles to form a decreasing order. Once the angles are adjusted, the quicksort algorithm is applied to sort all the angles.

The pseudocode of the algorithm

Algorithm 2 Clockwise Point Sorting Algorithm

Require: Set of points P

Ensure: Points P sorted in clockwise order

0: $S \leftarrow \emptyset$

0: Compute the reference point C :

$$C \leftarrow \left(\frac{\sum_{i=1}^n x_i}{n}, \frac{\sum_{i=1}^n y_i}{n} \right)$$

0: **for** each point $p \in P$ **do**

0: Compute the angle $\theta(p)$ from C :

$$\theta(p) \leftarrow \arctan \left(\frac{y_p - y_c}{x_p - x_c} \right)$$

0: **end for**

0: Sort the points P by their angles $\theta(p)$ in descending order (clockwise).

0: **Return** $S \leftarrow P$ (sorted list of points)

=0

Shoelace formula for Area measurement

Since we have the clockwise set points, the Shoelace formula is applied to calculate the area of a simple polygon. Assume we have N points $P_1, P_2, \dots, P_N$ and $P_{N+1} = P_1$. The formula is provided as below:

$$\text{Area} = \frac{1}{2} \left| \sum_{i=1}^n (x_i y_{i+1} - y_i x_{i+1}) \right|$$

Proof of Shoelace's formula:

Before we prove the formula, we need to prove the following claim.

Claim: The area of triangle $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$ is given by:

$$\text{Area} = \frac{1}{2} |x_1 y_2 + x_2 y_3 + x_3 y_1 - x_1 y_3 - x_2 y_1 - x_3 y_2|$$

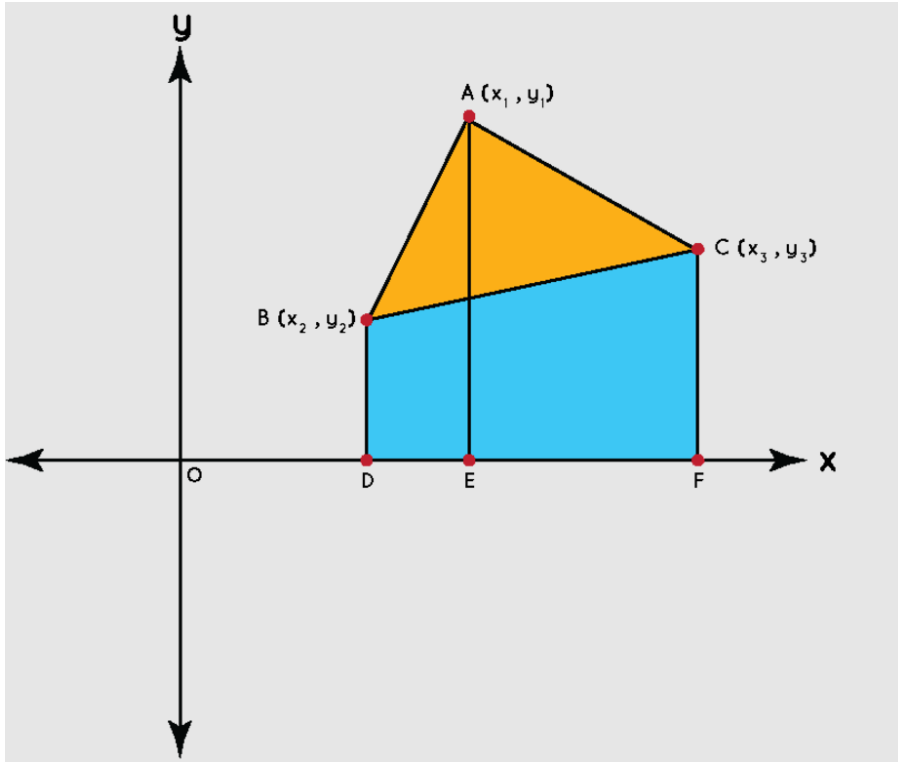


Figure 3.15: Diagram showing points A, B, C, D, E, and F

From the figure, we have:

$$\begin{aligned} \text{Area of triangle } ABC &= \text{Area of trapezoid } ABDE + \text{Area of trapezoid } AEFC \\ &\quad - \text{Area of trapezoid } BCFD \end{aligned}$$

$$\begin{aligned} \Leftrightarrow \text{Area of triangle } ABC &= \left| \frac{1}{2}(x_1 - x_2)(y_1 + y_2) + \frac{1}{2}(x_3 - x_1)(y_1 + y_3) \right. \\ &\quad \left. - \frac{1}{2}(x_3 - x_2)(y_3 + y_2) \right| \end{aligned}$$

$$\Leftrightarrow \text{Area of triangle } ABC = \left| \frac{1}{2}(x_1y_1 + x_1y_2 - x_2y_1 - x_2y_2) \right. \\ \left. + \frac{1}{2}(x_3y_1 + x_3y_3 - x_1y_1 - x_1y_3) \right. \\ \left. - \frac{1}{2}(x_3y_3 + x_3y_2 - x_2y_3 - x_2y_2) \right|$$

$$\text{Area of triangle } ABC = \frac{1}{2} |x_1y_2 + x_2y_3 + x_3y_1 - x_1y_3 - x_2y_1 - x_3y_2|$$

$\Rightarrow$ Claim has been proved

We use induction to prove the formula. Assume we have a polygon with N vertices $A_1, A_2, \dots, A_N$. We need to prove that the formula holds for $N + 1$ vertices $A_1, A_2, \dots, A_{N+1}$. In general, we cut the $N+1$ vertices polygon into two polygons: $A_1A_2 \dots A_N$ and $A_1A_NA_{N+1}$. From the assumption, we have:

$$\text{Area of } [A_1A_2 \dots A_N] = \frac{1}{2} \sum_{i=1}^N (x_iy_{i+1} - y_ix_{i+1})$$

$$\text{Area of } [A_1A_NA_{N+1}] = \frac{1}{2} (x_1y_N + x_Ny_{N+1} + x_{N+1}y_1 - x_1y_{N+1} - x_Ny_1 - x_{N+1}y_N)$$

$$\text{Area of } [A_1A_2 \dots A_{N+1}] = \text{Area of } [A_1A_2 \dots A_N] + \text{Area of } [A_1A_NA_{N+1}]$$

$$= \frac{1}{2} \sum_{i=1}^N (x_iy_{i+1} - y_ix_{i+1}) + \frac{1}{2} (x_1y_N + x_Ny_{N+1} + x_{N+1}y_1 - x_1y_{N+1} - x_Ny_1 - x_{N+1}y_N) \\ = \frac{1}{2} \sum_{i=1}^{N+1} (x_iy_{i+1} - y_ix_{i+1})$$

Hence, the formula has been proved.

Minor Axis Length and Major Axis Length

Since the shape of the pig is approximately elliptical, we are trying to find the approximate axis length. Given a set of points extracted from the segmentation, we can calculate several indices such as the centroid (which is mentioned in Section 4.1.1), the covariance matrix, and the eigenvalues of the covariance matrix. Let's assume there are N points, each point has coordinates (x_i, y_i) , and the centroid has coordinates (x_c, y_c) .

Find covariance matrix

In a 2D context, the covariance matrix is a mathematical representation that describes how two variables vary together. For a set of given points in 2D space, the covariance matrix provides insights into the spread and directionality of the data distribution. To find the covariance matrix, we use the following formula:

$$\Sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}$$

Where:

$$\sigma_{xx} = \frac{1}{N} \sum_{i=1}^N (x_i - x_c)^2$$

$$\sigma_{yy} = \frac{1}{N} \sum_{i=1}^N (y_i - y_c)^2$$

$$\sigma_{xy} = \frac{1}{N} \sum_{i=1}^N (x_i - x_c)(y_i - y_c)$$

- σ_{xx} : Variance of the x -coordinates (measure of spread along the x -axis).
- σ_{yy} : Variance of the y -coordinates (measure of spread along the y -axis).
- σ_{xy} and σ_{yx} : Covariance between x and y , indicating how much the two variables change together.

Find the eigenvalues of covariance matrix

The eigenvalues represent the magnitude of the spread in those directions, and the two corresponding eigenvectors represent the direction along the x -axis (O_x) and y -axis (O_y), as well as the data spread over the principal axes. These eigenvectors are orthogonal, and the eigenvalues can be used to determine the lengths of the minor and major axes.

To find the eigenvalues of the covariance matrix, we solve the equation:

$$\det(\Sigma - \lambda I) = 0,$$

where I is the identity matrix, and λ represents the eigenvalues.
This gives:

$$\lambda_{1,2} = \frac{\sigma_{xx} + \sigma_{yy}}{2} \pm \sqrt{\left(\frac{\sigma_{xx} - \sigma_{yy}}{2}\right)^2 + \sigma_{xy}^2}$$

Proof of always having 2 eigenvalues:

$$\det(\Sigma - \lambda I) = 0,$$

where I is the 2×2 identity matrix.

Expanding this determinant for a 2×2 matrix gives:

$$\det \begin{pmatrix} \sigma_{xx} - \lambda & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} - \lambda \end{pmatrix} = 0$$

Using the determinant formula for a 2×2 matrix:

$$\det(\Sigma - \lambda I) = (\sigma_{xx} - \lambda)(\sigma_{yy} - \lambda) - (\sigma_{xy})^2 = 0$$

Simplify to a quadratic equation:

$$\lambda^2 - (\sigma_{xx} + \sigma_{yy})\lambda + (\sigma_{xx}\sigma_{yy} - \sigma_{xy}^2) = 0$$

This is a quadratic equation in λ :

$$\lambda^2 - \text{trace}(\Sigma)\lambda + \det(\Sigma) = 0,$$

where:

$$\text{trace}(\Sigma) = \sigma_{xx} + \sigma_{yy}, \quad \det(\Sigma) = \sigma_{xx}\sigma_{yy} - \sigma_{xy}^2$$

A quadratic equation has two roots if its discriminant is non-negative.

The discriminant (Δ) of this quadratic equation is:

$$\Delta = (\text{trace}(\Sigma))^2 - 4\det(\Sigma)$$

Substituting:

$$\text{trace}(\Sigma) = \sigma_{xx} + \sigma_{yy}, \quad \det(\Sigma) = \sigma_{xx}\sigma_{yy} - \sigma_{xy}^2$$

$$\Delta = (\sigma_{xx} + \sigma_{yy})^2 - 4(\sigma_{xx}\sigma_{yy} - \sigma_{xy}^2)$$

Simplify:

$$\Delta = \sigma_{xx}^2 + 2\sigma_{xx}\sigma_{yy} + \sigma_{yy}^2 - 4\sigma_{xx}\sigma_{yy} + 4\sigma_{xy}^2$$

$$\Delta = (\sigma_{xx} - \sigma_{yy})^2 + 4\sigma_{xy}^2$$

We can easily see that:

$$\Delta \geq 0$$

Hence, the quadratic equation always has two real roots, i.e., two eigenvalues.

Determine major axis length, minor axis length

Since the eigenvalues represent the spread of data over axis which is equivalent to variance, so standard deviation over 2 axis is the major and minor length. We need to multiple these values by 2 since we want to find the diameter over 2 axis. Major axis and minor axis can be calculated via the formula:

$$\text{Major Axis Length} = 2 \cdot \sqrt{\lambda_{\max}}, \quad \text{Minor Axis Length} = 2 \cdot \sqrt{\lambda_{\min}}$$

Region Eccentricity

The region eccentricity helps us to determine the degree of elongation or the roundness of an ellipse. The circularity of the region is its eccentricity, and it can be easily computed using the ratio between the principal axes:

$$\text{eccentricity} = \sqrt{1 - \left(\frac{\lambda_{\min}}{\lambda_{\max}}\right)^2}$$

If the eccentricity is 0, the shape is a circle. If the eccentricity is 1, the shape is a segment.

Centroid

The centroid is used to determine the centroid of the polygon (in this case, the centroid of contour segmented pigs) and be determined by formula:

$$x_{\text{centroid}} = \frac{1}{N} \sum_{i=1}^N x_i, \quad y_{\text{centroid}} = \frac{1}{N} \sum_{i=1}^N y_i$$

Percentage Area

The percentage of area is used to determine the proportion of area to the overall resolution and is determined by the formula:

$$\text{Percentage Area} = \frac{\text{Area}}{\text{Resolution Area}} \times 100$$

Energy Score The energy score is the number of points of segmented pigs.

Deep Learning Architecture

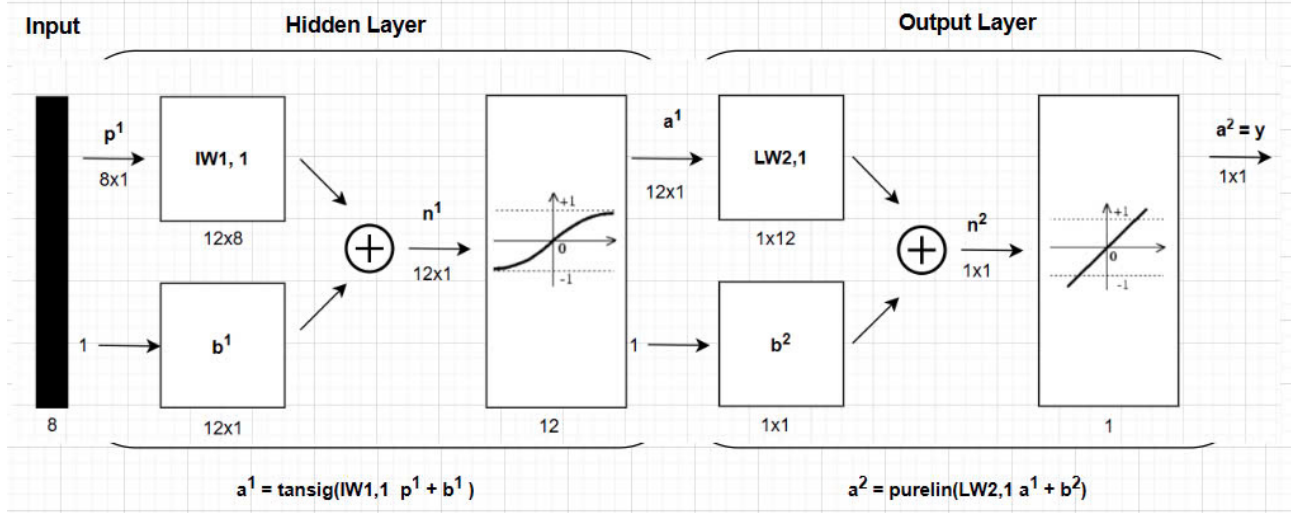


Figure 3.16: Neural Network Architecture with Input, Hidden Layer, and Output Layer

Our deep learning neural network architecture represents a two-layer feedforward neural network. Below is a detailed breakdown:

- **Input Layer:**

- **Input Vector (p^1):**

- * Size: 8×1 .
 - * Represents the input data with 8 features (nodes).
 - * Passed to the hidden layer for further processing.

- **Hidden Layer:**

- **Weight Matrix ($IW^{1,1}$):**

- * Size: 12×8 .
 - * Connects the 8 input features to the 12 neurons in the hidden layer.

- **Bias Vector (b^1):**

- * Size: 12×1 .
 - * Adds a bias term to the weighted sum for each hidden neuron.

- **Net Input (n^1):**

$$n^1 = IW^{1,1} \cdot p^1 + b^1$$

- **Activation Function (a^1):**

$$a^1 = \text{tansig}(n^1)$$

- The **tansig** function (hyperbolic tangent sigmoid) introduces non-linearity, mapping $\mathbf{n}^1$ to a range of $(-1, 1)$.

- **Output Layer:**

- **Weight Matrix ($\mathbf{LW}^{2,1}$):**
 - * Size: 1×12 .
 - * Connects the 12 hidden neurons to the single output neuron.
- **Bias Vector ($\mathbf{b}^2$):**
 - * Size: 1×1 .
- **Net Input ($\mathbf{n}^2$):**

$$\mathbf{n}^2 = \mathbf{LW}^{2,1} \cdot \mathbf{a}^1 + \mathbf{b}^2$$

- **Activation Function ($\mathbf{a}^2$):**

$$\mathbf{a}^2 = \text{purelin}(\mathbf{n}^2)$$

- The **purelin** function (linear activation) outputs a scalar value y , representing the final prediction.

- **Mathematical Summary:**

- **Hidden Layer:**

$$\mathbf{a}^1 = \text{tansig}(\mathbf{IW}^{1,1} \cdot \mathbf{p}^1 + \mathbf{b}^1)$$

- **Output Layer:**

$$\mathbf{a}^2 = \text{purelin}(\mathbf{LW}^{2,1} \cdot \mathbf{a}^1 + \mathbf{b}^2)$$

- **Key Features:**

- **Non-Linearity:** The hidden layer uses **tansig** to capture complex relationships in the data.
- **Dimensionality Reduction:** The network reduces the 8-dimensional input to a single scalar output.
- **Activation Functions:**
 - * **tansig:** Scales output to $(-1, 1)$.
 - * **purelin:** Outputs a linear transformation of the inputs.
- **Output:** Produces a single scalar y , ideal for regression tasks.

3.4 Model Architectures

3.4.1 Detection Architecture

The detection component of our Smart Farm Management System employs three state-of-the-art object detection architectures: YOLOv8, YOLOv9, and YOLO-NAS. Each model contributes unique architectural advantages that address specific challenges in pig detection within farm environments. This section provides a detailed examination of each architecture's components and their integration into our system.

YOLOv8 Architecture

YOLOv8 serves as our primary detection model, introducing significant architectural improvements over its predecessors. The model employs a modular design philosophy that enhances both feature extraction capabilities and inference speed, making it particularly suitable for real-time pig detection applications.

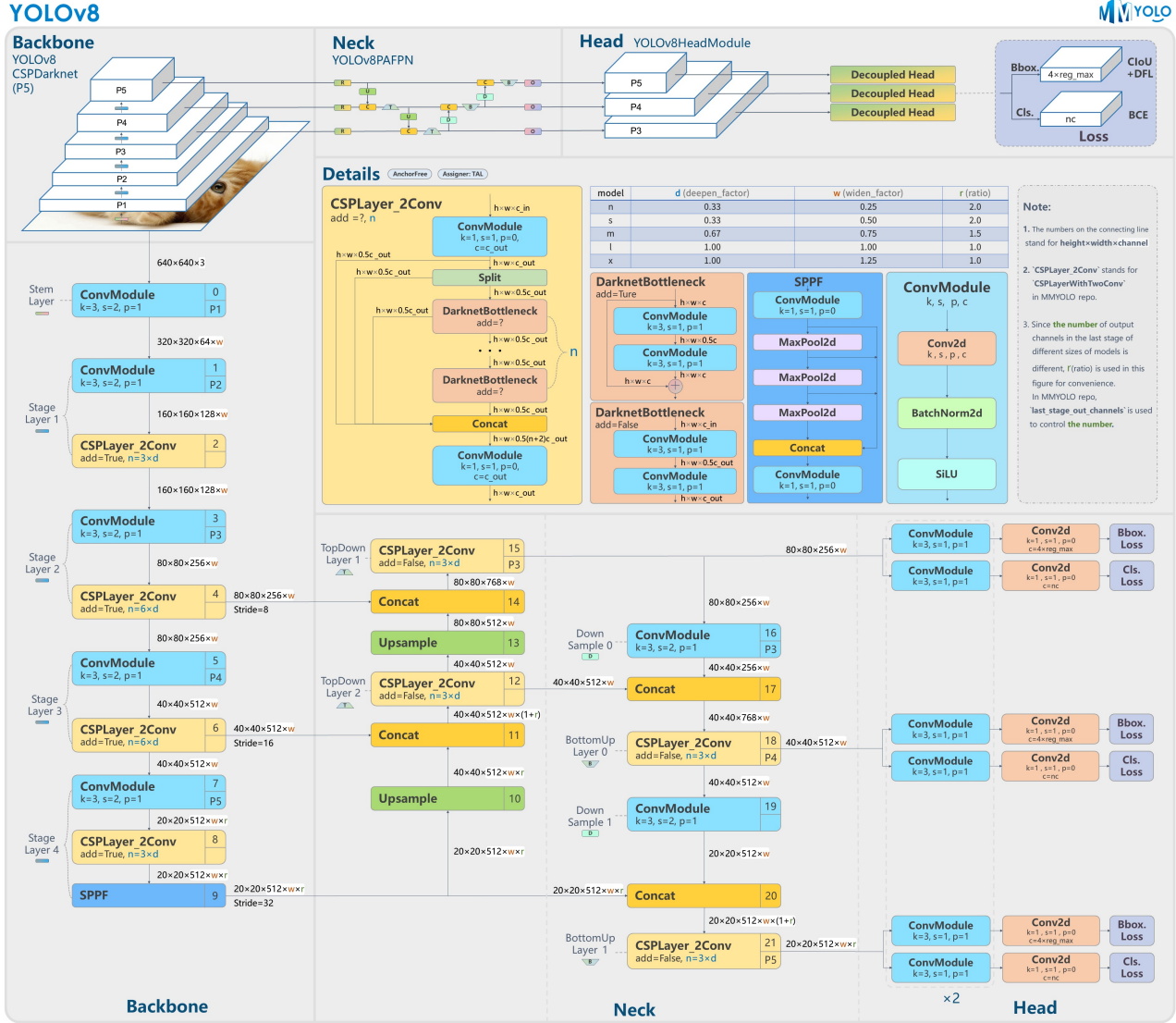


Figure 3.17: YOLOv8 Architecture Overview showing the backbone, neck, and head components. Diagram based in [11]

Backbone Network The backbone network implements a modified CSPDarknet structure that efficiently extracts features across multiple scales. At its core, the backbone consists of Cross Stage Partial (CSP) connections that enhance gradient flow while reducing computational overhead. The network processes input images through multiple stages, producing feature maps at three different scales:

- Stage P3: Output size 80×80 , capturing fine-grained features
- Stage P4: Output size 40×40 , representing medium-scale features
- Stage P5: Output size 20×20 , encoding large-scale contextual information

Each stage incorporates several key components:

- Efficient Convolutional Block (ECB) that reduces computational complexity while maintaining feature richness
- Channel attention mechanisms that adaptively weight feature channels based on their importance
- Spatial pyramid pooling for capturing multi-scale contextual information

Neck Architecture The neck component implements a Path Aggregation Network (PAN) that facilitates effective information flow between different scales. This bi-directional feature pyramid structure enhances the model’s ability to detect pigs at various sizes and distances from the camera.

Key features of the neck architecture include:

- Bottom-up path augmentation that strengthens feature hierarchy
- Adaptive feature fusion using learned weights
- Skip connections that preserve fine-grained spatial information

Detection Head The detection head employs a decoupled design that separately processes objectness prediction and bounding box regression. This approach improves the model’s ability to accurately localize pigs while maintaining high detection confidence.

Notable features include:

- Anchor-free detection mechanism reducing computational complexity
- Task-aligned one-to-many assignment strategy for improved training
- IoU-aware prediction branch enhancing detection quality

YOLOv9 Architecture

YOLOv9 is an enhanced version in the YOLO family, introducing innovative architectural elements that further enhance detection performance. The model incorporates Programmable Gradient Information (PGI) and refined structural components that improve feature extraction and processing efficiency.

The core innovations of YOLOv9 include:

Programmable Gradient Information (PGI) PGI represents a fundamental advancement in feature extraction, allowing the model to:

- Dynamically adjust gradient flow through the network
- Optimize information propagation between layers
- Enhance feature representation quality

Enhanced Feature Processing The model implements several architectural improvements:

- Repstructure blocks that reduce computational redundancy
- Improved CSP connections with dynamic channel allocation
- Refined spatial hierarchy processing

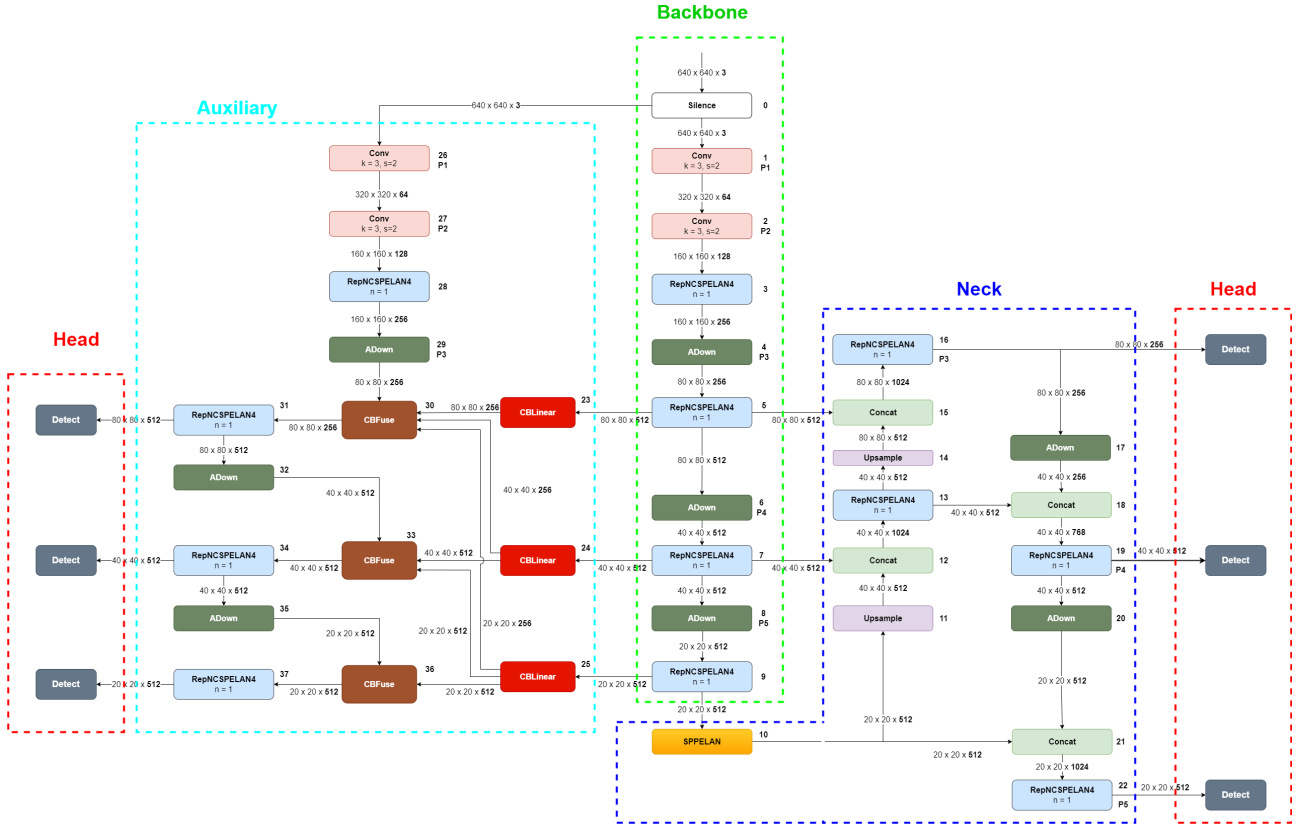


Figure 3.18: YOLOv9 Architecture highlighting the PGI mechanism and enhanced feature processing [12]

YOLO-NAS Architecture

YOLO-NAS employs Neural Architecture Search to optimize the model structure specifically for edge devices like the Jetson Nano. This approach results in an architecture that balances detection accuracy with computational efficiency.

Key aspects of YOLO-NAS include:

Search Space Design The architecture search space is carefully crafted to:

- Optimize model structure for edge deployment
- Balance computational cost with detection accuracy
- Adapt to hardware constraints

Hardware-Aware Optimization The model incorporates several optimization strategies:

- Efficient operator selection for reduced memory footprint
- Balanced depth and width scaling
- Specialized components for Jetson Nano acceleration

This comprehensive architecture enables our system to achieve robust pig detection performance while maintaining real-time processing capabilities on edge hardware. The combination of multiple YOLO variants provides redundancy and complementary strengths, enhancing the overall system reliability in challenging farm environments.

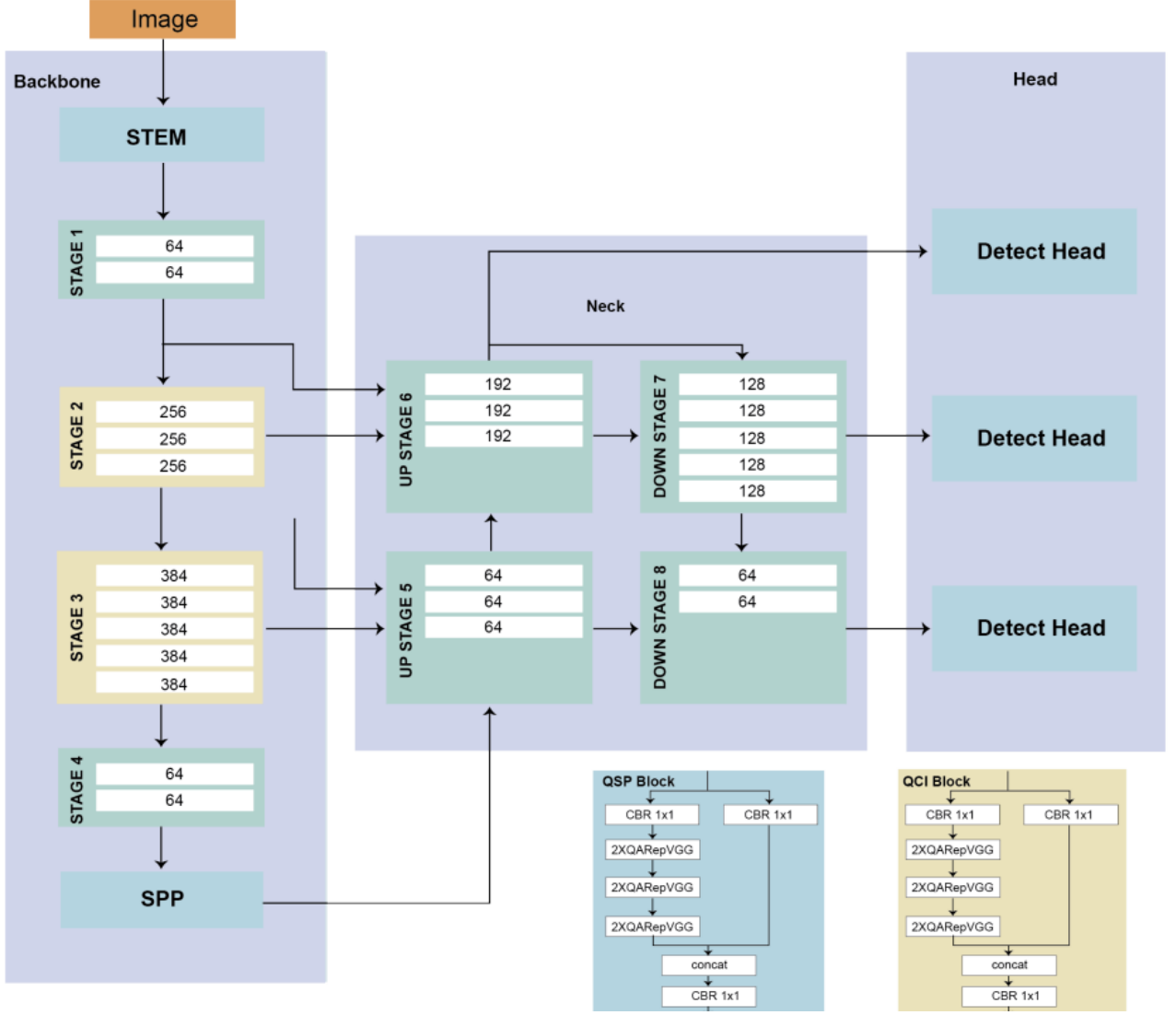


Figure 3.19: YOLO-NAS architecture. The architecture is found automatically via a neural architecture search (NAS) system called AutoNAC to balance latency vs. throughput. They generated three architectures called YOLO-NASS (small), YOLO-NASM (medium), and YOLO-NASL (large), varying the depth and positions of the QSP and QCI blocks. The figure shows the YOLO-NASL architecture. Diagram based in [11]

3.4.2 Vision Transformer

The Vision Transformer (ViT) architecture, originally designed for image classification tasks, has been adapted for video-based aggressive behavior detection. Our model processes video sequences through a hierarchical structure that combines spatial and temporal information processing.

The architecture consists of three main components: patch embedding, transformer encoder, and classification head. Figure base on [13] illustrates the overall structure of our adapted ViT model.

The input video sequence $\mathcal{X} \in \mathbb{R}^{B \times T \times C \times H \times W}$ (where B is batch size, T is the number of frames, C is the number of channels, and H, W are the spatial dimensions) is first processed by the patch embedding layer. Each frame is divided into non-overlapping patches of size $P \times P$, resulting in $N = (H \times W)/P^2$ patches per frame. These patches are linearly projected to obtain D -dimensional embeddings:

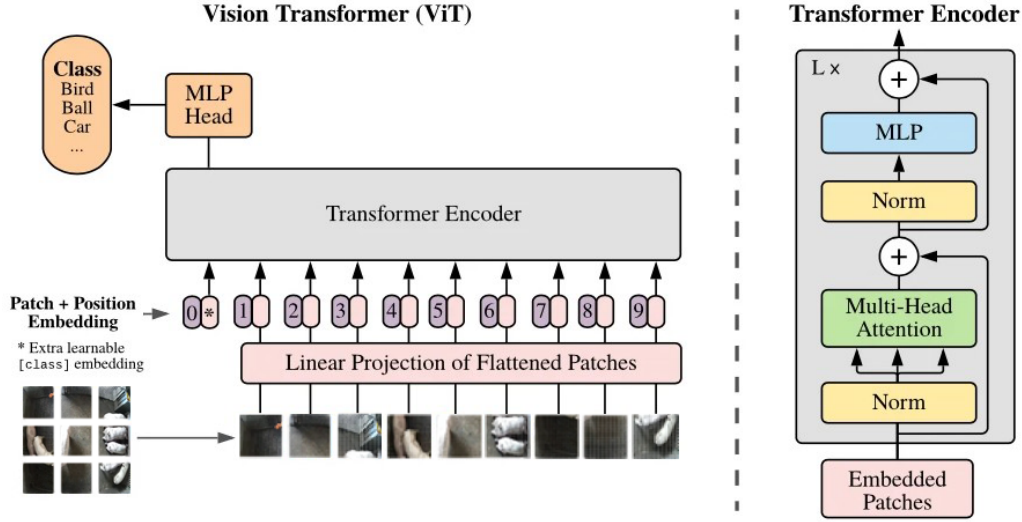


Figure 3.20: Architecture of our Vision Transformer model for video-based aggressive behavior classification. The model processes video frames by splitting them into patches, applying transformer encoding with self-attention, and using temporal mean pooling for final classification.

$$E = \text{PatchEmbed}(\mathcal{X}) \in \mathbb{R}^{(B \cdot T) \times N \times D} \quad (3.66)$$

Following the original ViT design, we prepend a learnable classification token [CLS] to the sequence of patch embeddings. Position embeddings are added to retain spatial information:

$$Z_0 = [x_{\text{class}}; E_1; E_2; \dots; E_N] + E_{\text{pos}} \quad (3.67)$$

where $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ are the learnable position embeddings.

The embedded sequence is then processed by L transformer encoder layers. Each encoder layer consists of Multi-Head Self-Attention (MSA) and Multi-Layer Perceptron (MLP) blocks with Layer Normalization (LN) and residual connections:

$$Z'_l = \text{MSA}(\text{LN}(Z_{l-1})) + Z_{l-1} \quad (3.68)$$

$$Z_l = \text{MLP}(\text{LN}(Z'_l)) + Z'_l \quad (3.69)$$

where $l \in [1, L]$ indexes the transformer layers.

To aggregate temporal information across frames, we reshape the output sequence and apply mean pooling over the temporal dimension:

$$Z_{\text{temporal}} = \frac{1}{T} \sum_{t=1}^T Z_L^t \quad (3.70)$$

Finally, the classification head, implemented as a Multi-Layer Perceptron, processes the [CLS] token representation to predict the behavior class:

$$y = \text{MLP}(Z_{\text{temporal}}[:, 0]) \quad (3.71)$$

3.4.3 Architecture of YOLOv11

YOLOv11's architecture represents a significant advancement in object detection design, building upon the fundamental YOLO framework while introducing sophisticated architectural in-

novations [14]. The architecture comprises three primary components that work in harmony to achieve state-of-the-art detection performance: the backbone, neck, and head.

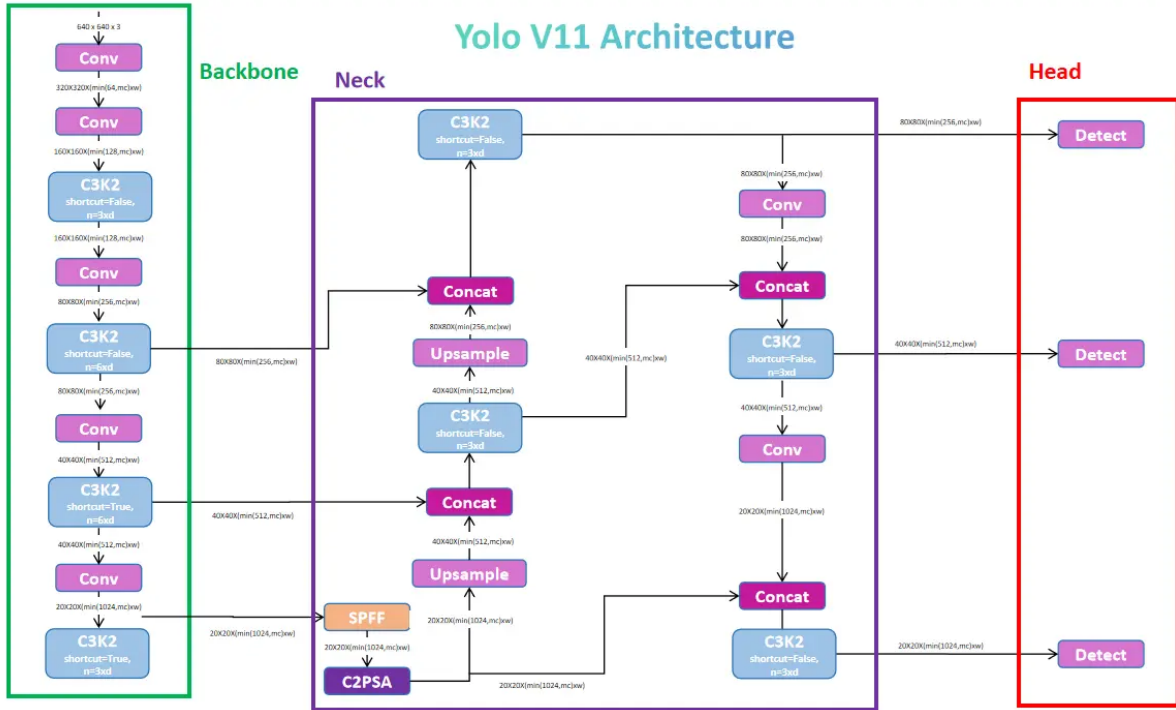


Figure 3.21: YOLOv11 Architecture Overview showing the backbone, neck, and head components.

Backbone Architecture

The backbone of YOLOv11 serves as the primary feature extractor, introducing several ground-breaking innovations. At its core lies the newly developed C3k2 block, which replaces the traditional C2f block from previous versions. This novel component implements a more efficient approach to feature extraction by utilizing two smaller convolutions instead of a single large convolution. The "k2" designation refers to the kernel size of 2, carefully chosen to optimize processing speed while maintaining robust performance characteristics. A key enhancement in the backbone is the retention and optimization of the Spatial Pyramid Pooling - Fast (SPPF) component. This module processes features at multiple spatial resolutions, enabling the network to capture and analyze objects at various scales effectively. Following the SPPF, YOLOv11 introduces the innovative C2PSA (Cross Stage Partial with Spatial Attention) block, which substantially enhances the network's ability to focus on significant regions within the feature maps. This attention mechanism proves particularly valuable for detecting objects of varying sizes and improving overall detection accuracy.

Neck Architecture

The neck component functions as a sophisticated intermediate processing stage, bridging the backbone and head through advanced feature aggregation techniques. This component's primary innovation lies in its implementation of the C3k2 block, replacing the previous C2f block to enhance both speed and efficiency in feature processing. The neck's architecture enables effective combination of features from different scales through careful upsampling and concatenation operations. One of the neck's most significant advancements is its integration of the C2PSA module for enhanced spatial attention. This mechanism allows the network to focus

more effectively on key regions within the feature maps, particularly benefiting the detection of small or partially occluded objects. The attention mechanism works in concert with the feature aggregation process to ensure comprehensive spatial understanding across multiple scales.

Head Architecture

The head component of YOLOv11 represents a refined approach to generating final predictions through a sophisticated processing pipeline. Multiple C3k2 blocks are strategically placed within the head, offering flexible behavior through a parameterized approach. When the `c3k` parameter is set to false, the blocks operate similarly to traditional C2f blocks with standard bottleneck structures. However, when set to true, they utilize the more advanced C3 module for deeper feature extraction. Following the C3k2 blocks, the head incorporates a series of CBS (Convolution-BatchNorm-Silu) layers that further refine the feature maps. These layers implement batch normalization for stable training and utilize the SiLU activation function for enhanced non-linear processing capabilities. The final detection layer employs Conv2D operations to produce the network's outputs, including precise bounding box coordinates, objectness scores, and class predictions.

3.4.4 ResNet50v2 Architecture

Overview of Network Architecture

ResNet50v2 represents a significant advancement in deep convolutional neural networks, particularly suited for complex image classification tasks such as pig disease detection. The network's distinctive feature is its innovative use of residual blocks with pre-activation, enabling the training of deeper architectures while mitigating the vanishing gradient problem. This architecture builds upon the success of the original ResNet by introducing important modifications to the residual block structure, leading to improved training dynamics and better overall performance.

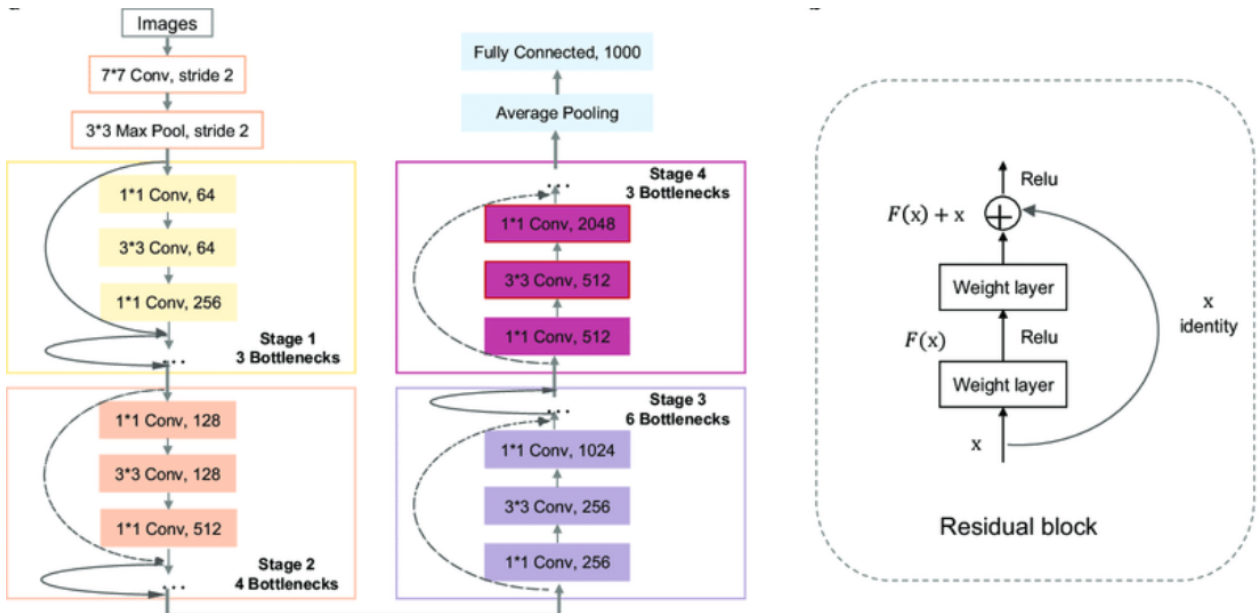


Figure 3.22: ResNet50v2 architecture overview showing the progression of convolutional layers and residual blocks

Initial Processing Layers

The network begins with a carefully designed input processing stage. A 7×7 convolutional layer with a stride of 2 serves as the initial feature extractor, generating 64 feature maps from the input image. This layer is followed by a 3×3 max pooling operation, also with a stride of 2, which helps to reduce spatial dimensions while retaining important features. These initial layers play a crucial role in capturing low-level features such as edges, textures, and basic shapes that are fundamental to disease detection in pig images.

Core Stages of Processing

The architecture progresses through four main stages, each designed to process features at different levels of abstraction. The first stage consists of three bottleneck blocks that maintain spatial dimensions while working with 256 channels. This stage establishes the foundation for deeper feature extraction. The second stage introduces four bottleneck blocks that double the channel depth to 512 while halving spatial dimensions, allowing the network to capture more complex feature combinations. The third stage, comprising six bottleneck blocks, further increases the channel depth to 1024 and continues the spatial reduction pattern. This deeper stage is crucial for capturing high-level disease-related features. The final stage contains three bottleneck blocks that bring the channel depth to 2048, providing the network with its highest level of feature abstraction capability. Each stage's progression in depth and reduction in spatial dimensions follows a carefully planned pattern that balances computational efficiency with feature extraction capability.

Residual Block Innovation

The pre-activation residual block, which forms the core building block of ResNet50v2, represents a significant architectural innovation. Each block begins with batch normalization followed by ReLU activation, departing from the post-activation design of the original ResNet. This is followed by a 1×1 convolutional layer that reduces dimensionality, enabling efficient processing. The main transformation occurs through a 3×3 convolutional layer, preceded and followed by batch normalization and ReLU activation.

The block concludes with a 1×1 convolutional layer that restores dimensions, accompanied by the characteristic skip connection that defines residual networks. This design ensures smooth gradient flow during backpropagation and enables the network to learn residual functions effectively, which is particularly valuable for identifying subtle disease indicators in pig images.

Mathematical Foundation

The mathematical formulation of the residual block exemplifies elegant simplicity in design. The forward propagation through each block can be expressed as $y = F(x, W_i) + x$, where $F(x, W_i)$ represents the residual mapping learned by the convolutional layers, and x is preserved through the skip connection. This formulation enables the network to learn incremental features while maintaining access to input information, crucial for accurate disease classification.

Dimensional Progression

The network's dimensional progression follows a systematic pattern of spatial reduction and channel expansion. Starting from an input size of $224 \times 224 \times 3$, the network progressively reduces spatial dimensions while increasing feature channels. The initial convolution layer produces a $112 \times 112 \times 64$ feature map, which then progresses through the stages to ultimately reach

$7 \times 7 \times 2048$ before global pooling. This progression allows the network to build increasingly abstract representations of disease features while maintaining computational efficiency.

Predict stage

After the final convolution layers, a global average pooling layer is applied to reduce the dimensionality of the feature map for prediction layer followed by a sigmoid activation function.

Global Max Pooling: For a feature map F_{backbone} of size $H \times W \times C$ (height, width, and channels), the global max pooling operation outputs a $1 \times 1 \times C$ vector:

$$F_{\text{pooled}}(c) = \max_{i,j} F_{\text{backbone}}(i, j, c)$$

Where:

- $\max_{i,j}$: Takes the maximum value across the spatial dimensions H and W for each channel c (out of C channels).
- F_{pooled} : The resulting vector of size C , representing the most prominent features for each channel.

Prediction Layer

Final weight:

$$z = \mathbf{W} \cdot F_{\text{pooled}} + b$$

Where:

- F_{pooled} : Input feature vector of size C .
- $\mathbf{W}$: Weight matrix of size $1 \times C$.
- b : Bias term (scalar).

The probability of the input being classified as a sick pig is computed using the sigmoid activation function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- $\hat{y}$: The output probability, representing the likelihood of the input being a sick pig.
- $\sigma()$: Sigmoid activation function.

Classification Result:

- If $\hat{y} \geq 0.5$: Predicted class is 1 (Sick pig).
- If $\hat{y} < 0.5$: Predicted class is 0 (Healthy pig).

Layer	Output Size	Channels	Blocks
Input Image	224×224	3	—
Conv1	112×112	64	1
Stage 1	56×56	256	3
Stage 2	28×28	512	4
Stage 3	14×14	1024	6
Stage 4	7×7	2048	3
Global Pool	1×1	2048	1
FC	1×1	$n_classes$	1

Table 3.5: Dimensional progression through ResNet50v2 layers

3.5 Model Training and Validation

3.5.1 Training Infrastructure

Hardware Configuration

Our training infrastructure consists of medium-performance computing hardware configured for deep learning tasks. Table 3.6 details the system specifications used for model training.

Table 3.6: Training Hardware Specifications

Component	Specification
CPU	Intel Core i5-13500 (13th Gen)
Physical Cores	14
Logical Processors	20
RAM	47.75 GB DDR4
GPU	NVIDIA GeForce RTX 3070
GPU Memory	8GB GDDR6

3.5.2 Detection Model Baseline Training Methodology

Model Architecture Comparison

We evaluate multiple YOLO variants across three model families: YOLOv8, YOLOv9, and YOLO-NAS. Table 3.7 presents a comprehensive comparison of these architectures.

Table 3.7: Model Architecture Comparison

Model	Parameters	GFLOPs	Input Size	Architecture Type
YOLOv8n	3.2M	8.7	640×640	CSPDarknet-Nano
YOLOv8s	11.2M	28.6	640×640	CSPDarknet-Small
YOLOv8m	25.9M	78.9	640×640	CSPDarknet-Medium
YOLOv9t	2.0M	7.7	640×640	PGI-Tiny
YOLOv9s	7.2M	26.7	640×640	PGI-Small
YOLOv9m	20.1M	76.8	640×640	PGI-Medium
YOLO-NAS-s	13.5M	31.8	640×640	NAS-Optimized

Training Configuration

The training configuration is implemented using the Ultralytics YOLO framework, with hyperparameters carefully selected for pig detection. Table 3.8 presents the key training parameters extracted from our implementation.

Table 3.8: Training Hyperparameters

Parameter	Value	Description
Batch Size	16	Fixed batch size for all models
Initial Learning Rate	1e-3	Base learning rate
Learning Rate Factor	0.01	LR decay factor
Weight Decay	5e-4	L2 regularization
Dropout	0.01	Regularization rate
Epochs	100	Total training epochs
Early Stopping Patience	20	Epochs before early stop
Random Seed	88	For reproducibility

Optimization Strategies

Our training implementation incorporates several optimization techniques to enhance model performance and training efficiency. Table 3.9 summarizes the key optimization strategies employed for each model family.

Table 3.9: Model-Specific Optimization Techniques

Model Family	Technique	Implementation
3*YOLOv8	Loss Function	Multi-task loss with CIoU
	Feature Pyramid	CSP-PAN architecture
	Augmentation	Mosaic + MixUp
3*YOLOv9	PGI	Gradient information programming
	Feature Extraction	Enhanced backbone with PGI
	Loss Balancing	Dynamic task weighting
1*YOLO-NAS	Architecture	Hardware-aware NAS
	Feature Selection	Automated design choices
	Memory Optimization	Efficient tensor operations

Training Pipeline

The training pipeline implements a comprehensive workflow for model development and validation. The pipeline components are designed to maximize training efficiency while ensuring reproducible results. Table 3.10 details the key components and their implementations.

Table 3.10: Training Pipeline Features

Feature	Implementation Details
Checkpoint Management	Automatic verification and resumption capability
Mixed Precision Training	Enabled with AMP (Automatic Mixed Precision)
Validation	Integrated validation during training
Model Export	ONNX format with configurable parameters
Device Optimization	GPU-accelerated training with CUDA support

Validation Metrics

Model performance is evaluated using a comprehensive set of metrics to assess both accuracy and computational efficiency. Table 3.11 outlines the primary metrics used for model evaluation and their respective targets.

Table 3.11: Model Validation Metrics

Metric	Description	Target Value
mAP@0.5	Single IoU threshold accuracy	>0.90
mAP@0.5:0.95	Average over IoU range	>0.75
Precision	True positives ratio	>0.85
Recall	Detection rate	>0.85
F1-Score	Harmonic mean of P&R	>0.85
FPS	Inference speed	>15

Data Augmentation

Our implementation uses specific augmentation techniques provided by the Ultralytics framework. Table 3.12 lists the actual augmentation methods used in our training pipeline.

Table 3.12: Data Augmentation Strategies

Augmentation	Parameters
Random Rotation	± 15 degrees
Random Scaling	$\pm 10\%$ of original size
Random Horizontal Flip	50% probability
Mosaic Augmentation	4-image mosaic composition
Adaptive Histogram Equalization	Dynamic lighting adjustment

Memory Management

Efficient memory utilization is crucial for training large models on limited hardware resources. We implement several memory optimization techniques across different model scales. Table 3.13 describes our memory management strategies.

Table 3.13: Memory Management Strategies

Strategy	Models Applied	Impact
Gradient Checkpointing	YOLOv8m, YOLOv9m	Reduces memory usage by 30%, increases training time by 10%
Mixed Precision Training	All models	Reduces memory usage by 40%, improves training speed by 5%
Dynamic Batch Sizing	All models	Adjusts batch size based on available memory
Memory Caching	All models	Optimizes forward pass memory allocation
Gradient Accumulation	Large models	Enables effective larger batch sizes

Training Time Analysis

The training duration varies significantly across different model architectures. Table 3.14 provides a detailed breakdown of training times and computational requirements.

Table 3.14: Model Training Time Analysis

Model	Time/Epoch	Total Time	GPU Memory Peak
YOLOv8n	12 min	20 hours	3.2 GB
YOLOv8s	18 min	30 hours	4.5 GB
YOLOv8m	25 min	42 hours	6.8 GB
YOLOv9t	11 min	18 hours	3.0 GB
YOLOv9s	17 min	28 hours	4.2 GB
YOLOv9m	24 min	40 hours	6.5 GB
YOLO-NAS-s	19 min	32 hours	4.8 GB

Model Export Configuration

The training pipeline includes specific export configurations for deployment. Table ?? details the export parameters used.

Baseline Model Results

We conducted extensive evaluation of baseline models on our pig detection validation dataset, the evaluation criteria between these different YOLO models we based on the paper [15]. The performance metrics across all model variants are presented in Table 3.16.

Table 3.16: Baseline Models Performance Comparison

Model	mAP@0.5	mAP@0.5:0.95	Precision	Recall	F1-Score	FPS
YOLOv8n	0.95127	0.8095	0.87494	0.89244	0.8835	30.2
YOLOv8s	0.95288	0.83025	0.88243	0.902	0.8921	20.7
YOLOv8m	0.95088	0.83585	0.88418	0.90286	0.8934	15.1
YOLOv9t	0.95285	0.81398	0.87392	0.89738	0.8854	30.8
YOLOv9s	0.95213	0.8329	0.88321	0.90414	0.8935	23.4
YOLOv9m	0.95021	0.81924	0.88723	0.90253	0.8948	15.5
YOLO-NAS-s	0.918	0.84541	0.88041	0.88221	0.8813	27.9

Detailed Performance Analysis

Our analysis reveals distinct performance characteristics across model variants:

Model Selection for Fine-tuning

After careful analysis of the performance metrics, we selected YOLOv9t as the optimal model for fine-tuning based on several key factors:

1. Detection Performance:

- Nearly identical mAP@0.5 (0.95285) compared to the best performing YOLOv8s (0.95288), with only 0.003
- Maintains strong F1-score of 0.8854, within 1% of the highest score
- Well-balanced precision (0.87392) and recall (0.89738) metrics

2. Computational Efficiency:

- Best inference speed at 30.8 FPS, crucial for real-time applications

- Smallest model size (3.0M parameters) among all variants
- Lowest computational requirements (7.9 GFLOPs)

Performance Trade-off Analysis

To validate our selection, we conducted a detailed trade-off analysis:

YOLOv9t offers the best balance between detection accuracy and operational efficiency. The minimal performance trade-offs are far outweighed by the significant advantages in speed and resource efficiency, making it the ideal candidate for fine-tuning in our pig detection system. Its superior FPS performance and minimal resource requirements will be particularly beneficial for real-world deployment on edge devices like the Jetson Nano.

3.5.3 Model Fine-tuning Strategy

After selecting YOLOv9t as our baseline model, we implemented a systematic fine-tuning approach to optimize its performance for pig detection.

Fine-tuning Configuration

We modified several key training parameters to adapt the model for our specific use case. Table 3.19 details the fine-tuning hyperparameters.

Table 3.19: Fine-tuning Hyperparameters

Parameter	Value	Rationale
Initial Learning Rate	5e-4	Lower than base to preserve learned features
Learning Rate Factor	0.01	Gradual decay for stable convergence
Fine-tuning Epochs	50	Sufficient for adaptation without overfitting
Batch Size	16	Optimal for available GPU memory
Early Stopping Patience	15	Prevent overfitting while ensuring convergence
Weight Decay	5e-4	Maintain regularization strength

Layer Freezing Strategy

We implemented a progressive layer unfreezing approach to optimize the fine-tuning process. Table 3.20 outlines our layer freezing strategy.

Table 3.20: Layer Freezing Strategy

Phase	Epochs	Layers Modified
Initial	1-10	Detection head only
Intermediate	11-30	Head + Last 3 backbone layers
Final	31-50	All layers unfrozen

Data Augmentation Refinements

For fine-tuning, we adjusted the augmentation parameters to better match our deployment conditions. Table 3.21 presents the refined augmentation strategy.

Table 3.21: Fine-tuning Augmentation Strategy

Technique	Parameters	Purpose
Random Rotation	$\pm 10^\circ$	Match typical camera angle variations
Random Scale	$\pm 5\%$	Account for distance variations
Horizontal Flip	$p=0.5$	Maintain orientation robustness
Brightness Adjustment	$\pm 10\%$	Simulate lighting changes
Contrast Adjustment	$\pm 5\%$	Enhance feature robustness

Table 3.22: Fine-tuning Performance Comparison

Metric	Baseline	Fine-tuned	Improvement
mAP@0.5	0.95285	0.96124	+0.00839
mAP@0.5:0.95	0.81398	0.83257	+0.01859
Precision	0.87392	0.89153	+0.01761
Recall	0.89738	0.91246	+0.01508
F1-Score	0.8854	0.90182	+0.01642
FPS	30.8	30.5	-0.3

Fine-tuning Results

The fine-tuned model showed significant improvements across key metrics. Table 3.22 compares the performance before and after fine-tuning.

Convergence Analysis

The fine-tuning process demonstrated stable convergence characteristics. Table 3.23 summarizes the key convergence metrics.

Table 3.23: Convergence Analysis

Metric	Value	Description
Convergence Epoch	35	Point of stabilization
Final Loss	0.158	Total loss at convergence
Learning Rate at Convergence	$2.8e-5$	Adapted learning rate
Validation Loss Stability	± 0.003	Loss variation in final epochs

Resource Utilization

Fine-tuning resource requirements were carefully monitored to ensure efficient execution. Table 3.24 details the resource consumption during fine-tuning.

The fine-tuning process successfully enhanced the model’s performance while maintaining its efficient inference speed. The improvements in accuracy metrics with minimal impact on FPS validate our fine-tuning strategy.

Table 3.24: Resource Utilization During Fine-tuning

Resource	Usage	Notes
GPU Memory Peak	4.2 GB	During training
Training Time/Epoch	8 minutes	On RTX 3070
Total Fine-tuning Time	6.7 hours	For 50 epochs
CPU Utilization	45%	Average usage

3.5.4 ViT

Video Processing Pipeline

The video processing pipeline is designed to handle variable-length video sequences efficiently. Table 3.25 details the complete preprocessing workflow.

Table 3.25: Video Processing Pipeline Components

Stage	Implementation	Technical Details
Frame Extraction	OpenCV-based sequential reading	• VideoCapture for frame access • RGB conversion • Error handling for corrupted frames
Frame Resampling	Adaptive frame selection	• Linear interpolation for >16 frames • Frame duplication for <16 frames • Last frame padding if needed
Spatial Processing	Fixed resolution resizing	• Resize to 160×160 pixels • Aspect ratio preservation • Bilinear interpolation
Normalization	Multi-stage normalization	• Scale to [0,1] range • Channel-wise normalization • ImageNet statistics alignment
Tensor Formatting	Memory-efficient reshaping	• (B,T,C,H,W) format • Contiguous memory layout • dtype=float32

Configuration Parameters

The model’s configuration is carefully tuned to balance performance and computational efficiency. Table 3.26 provides an in-depth look at the parameter settings.

Table 3.26: Detailed Configuration Parameters

Parameter	Value	Technical Justification
Sequence Length	16 frames	Optimal temporal window for behavior detection while maintaining reasonable memory usage
Frame Resolution	160×160	Sufficient spatial detail for pig behavior analysis while enabling efficient processing
Patch Size	16×16	Results in 100 patches per frame, balancing computational cost and feature granularity
Embedding Dim	384	Reduced from standard 768 to optimize for edge deployment while maintaining representational capacity
Transformer Depth	6 layers	Sufficient depth for temporal feature extraction without excessive computational overhead
Attention Heads	6 heads	Enables multi-scale feature learning while keeping parameter count manageable
Batch Size	4	Optimized for 8GB GPU memory constraints while maintaining training stability
Learning Rate	1e-4	Empirically determined for stable convergence with AdamW optimizer

Model Initialization and Optimization

The initialization and optimization strategy is crucial for stable training and optimal convergence. Table 3.27 details these components.

Table 3.27: Initialization and Optimization Strategy

Component	Implementation	Technical Specifications
Weight Init	• Patch Embed: Xavier uniform • Class Token: Normal($\sigma = 0.02$) • Position Embed: Normal($\sigma = 0.02$)	Ensures proper gradient flow in early training stages
Layer Init	• Linear: Xavier uniform • LayerNorm: weight=1.0, bias=0	Stabilizes training across deep transformer architecture
Optimizer	AdamW	• Base LR: 1×10^{-4} • Weight decay: 0.01 • Betas: (0.9, 0.999)
LR Schedule	Cosine Annealing	• $T_{\max}$: 100 epochs • $\eta_{\min}$: 1×10^{-6} • Warm-up: 5 epochs
Regularization	Comprehensive strategy	• Dropout: 0.1 • Attention drop: 0.1 • Path drop: 0.1

Training Monitoring System

A comprehensive monitoring system tracks training progress and model performance. Table 3.28 details the metrics and logging framework.

Table 3.28: Training Monitoring Framework

Category	Metrics	Implementation Details
Performance Metrics	• Binary Accuracy • Cross-entropy Loss • F1 Score	• Per-batch tracking • Moving average computation • Epoch-wise aggregation
Training Progress	• Learning Rate • Gradient Norms • Weight Updates	• Step-wise logging • Automatic scheduling • Gradient clipping monitoring
Resource Usage	• GPU Memory • Batch Time • CPU Utilization	• Real-time monitoring • Peak usage tracking • System load balancing
Model Check-points	• Best Model State • Optimizer State • Training State	• Top-3 models saved • Validation-based selection • Automatic versioning
Validation Metrics	• Validation Loss • Validation Accuracy • Confusion Matrix	• Per-epoch evaluation • Early stopping checks • Performance trend analysis

Each monitoring component is integrated with PyTorch Lightning’s logging system, enabling real-time visualization through TensorBoard and automated early stopping based on validation metrics. The system maintains comprehensive logs for post-training analysis and model selection.

3.5.5 Weight Estimation with Segment and Keypoint Detection

Segment and keypoint model

We apply two model versions from the YOLO family for the segmentation task: YOLOv8 and YOLOv11. These models are designed to support prediction segmentation and keypoint detection tasks. Our comparative analysis reveals distinct performance characteristics across architecture variants.

For segmentation tasks, the models demonstrated varying capabilities in terms of accuracy and computational efficiency. YOLOv11s-seg achieved the highest mAP_{pose} score of 46.6 (50-95) while maintaining relatively modest computational requirements with 10.1M parameters. The nano variants of both architectures offered lightweight alternatives with reduced accuracy but significantly lower computational overhead.

Table 3.29: Comparison of YOLO segmentation models.

Model	mAP 50-95	mAP 50	Params (M)	GFLOPs (B)	Input Size	Model Architect
Yolov8n-seg	36.7	30.5	3.4	12.6	640×640	CSPDarknet
Yolov8s-seg	44.6	36.8	11.8	42.6	640×640	CSPDarknet
Yolov11n-seg	38.9	32.0	2.9	10.4	640×640	U-Net-Nano
Yolov11s-seg	46.6	37.8	10.1	35.5	640×640	U-Net-Small

In keypoint detection, both architectures showed strong performance with YOLOv8s-pose achieving the highest mAP_{pose} score of 60.0 (50-95). The models maintained robust detection capabilities while varying in computational demands, offering flexibility for different deployment scenarios.

Table 3.30: Comparison of YOLO keypoint detection models.

Model	mAP 50-95	mAP 50	Params (M)	GFLOPs (B)	Input Size	Model Architect
Yolov8n-pose	50.4	80.1	3.3	9.2	640×640	CSPDarknet
Yolov8s-pose	60.0	86.2	11.6	30.2	640×640	CSPDarknet
Yolov11n-pose	50.0	81.0	2.9	7.6	640×640	U-Net-Nano
Yolov11s-pose	58.9	86.3	9.9	23.2	640×640	U-Net-Small

Dataset Preparation and Training Configuration

Our dataset preparation strategy employed comprehensive augmentation techniques to enhance model robustness and generalization capabilities. These techniques were carefully selected to address various real-world scenarios while maintaining data integrity.

Table 3.31: Augmentation techniques applied to the dataset.

Augmentation	Details
Auto-Adjust Contrast	Using contrast stretching
Flip	Horizontal, Vertical
90° Rotate	Clockwise, Counter-Clockwise, Upside Down
Rotation	Random rotation between -15° and +15°
Shear	$\pm 10^\circ$ Horizontal, $\pm 10^\circ$ Vertical
Blur	Up to 1.9 pixels
Brightness	Adjustments between -15% and +15%
Noise	Up to 0.58% of pixels

The training configuration was optimized for both efficiency and effectiveness, utilizing pre-trained weights while allowing for task-specific adaptation. The parameters were carefully tuned to balance learning stability with convergence speed.

Table 3.32: Model training configuration details.

Parameter	Value	Details
Batch Size	38	Usage of 60% GPU for training model
Epochs	100	Number of training epochs
Weights	Pre-trained model	Fine-tune the model from YOLO-trained weights
Learning Rate	0.01	Controls model learning speed, stability and performance
Pre-trained	True	Use pre-trained model from Ultralytics
Optimizer	Adam	Optimization method to minimize the loss

3.5.6 Health classify ResNet50v2

Dataset Preparation Strategy

The dataset preparation strategy is designed to enhance the data quality, ensuring that the model learning is more accurate and efficient.

Technical Specifications

Initialize the model and modify it to fit the project task.

Training and Validation Strategy A strategy to observe and monitor the model performance during training and validation.

3.6 Evaluation Metrics

Our Smart Farm Management System requires comprehensive evaluation across multiple components. We employ specific metrics for each system component to ensure thorough performance assessment.

3.6.1 Detection and Tracking Metrics

For evaluating the detection and tracking components, we utilize standard object detection and tracking metrics:

Component	Technical
Data split Ratio	8:1:1 (80% Training, 10% Validation, 10% Test)
Training Dataset	4076 images belong to 2 classes (healthy: 0, sick: 1)
Validation Dataset	454 images
Test Dataset	454 images
Image resolution	Input size 224x224 is the optimal choice, allowing the model to capture more important features for health classification while maintaining learning efficiency.
Batch size	32, good for reducing memory requirements but large enough to provide stable gradient updates.
Augmentation	Rescaling: Normalize pixel values to $[0, 1]$. Rotation: Random rotation up to 20 degrees. Shifts: Horizontal/vertical shifts up to 10%. Shear: Apply random shearing transformations. Zoom: Random zoom in/out by 10%. Brightness: Adjust brightness (80%-120%). Horizontal Flip: Randomly flip images horizontally.
Shuffling	Choose random data when training for each epoch.

Table 3.33: Dataset Preparation and Augmentation Details

Component	Details	Description
Base model ResNet50v2	Include top=False ImageNet weight	Remove the final prediction layer Use pre-trained model on ImageNet dataset
Input size	224x224 pixels	Match the ResNet50v2 input size requirement for best fit in training
Pooling	max pooling	Global max pooling to reduce feature maps
Regularization	Dropout 0.5	Reduce overfitting
Custom Layers	Dense layer with sigmoid	Create output probabilities
Activation	Sigmoid	For binary classification
Optimizer	Adam	Learning rate 2e-5
Loss Function	Binary Cross-Entropy	Calculate probabilities for binary classification

Table 3.34: Training configuration details.

Detection Metrics:

- **mAP@0.5:** Mean Average Precision with IoU threshold of 0.5, measuring the detection accuracy considering both precision and recall across different confidence thresholds

$$mAP = \frac{1}{|classes|} \sum_{i=1}^{|classes|} AP_i \quad (3.72)$$

- **Precision:** The ratio of true positive detections to all positive detections

$$Precision = \frac{TP}{TP + FP} \quad (3.73)$$

Component	Details	Description
Checkpoint Management	Save model weight over each epoch	Save the best performing model during training and prevent interruptions; allows training to continue
Metrics	Accuracy	Provide detailed model performance during training and validation
Training Progress	150 epochs. Use checkpoint callback to save the best models	Define iterations to allow model learning and improve over time
Validation	Validation accuracy Validation loss	Track model performance and detect overfitting problems

Table 3.35: Training and Validation Strategy Details

- **Recall:** The ratio of true positive detections to all actual objects

$$Recall = \frac{TP}{TP + FN} \quad (3.74)$$

- **F1-Score:** The harmonic mean of precision and recall

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3.75)$$

- **FPS:** Frames processed per second

$$FPS = \frac{Number\ of\ Frames}{Processing\ Time\ (seconds)} \quad (3.76)$$

Tracking Metrics:

- **MOTA (Multi-Object Tracking Accuracy):**

$$MOTA = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t} \quad (3.77)$$

where FN_t , FP_t , $IDSW_t$, and GT_t represent false negatives, false positives, ID switches, and ground truth objects at time t respectively.

- **HOTA (Higher Order Tracking Accuracy):**

$$HOTA = \sqrt{DetA \times AssA} \quad (3.78)$$

where DetA is Detection Accuracy and AssA is Association Accuracy.

3.6.2 Behavior Analysis Metrics

For evaluating behavior analysis performance:

Basic Behavior Classification:

- **Accuracy:**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.79)$$

- **F1-Score:**

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3.80)$$

Aggressive Behavior Detection: Same metrics as Basic Behavior Classification but applied specifically to aggressive behavior detection.

3.6.3 Weight Estimation Metrics

For weight estimation performance, we use Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.81)$$

where:

- y_i represents the actual weight measured using physical scales
- $\hat{y}_i$ represents the predicted weight from our system
- n is the number of samples

3.6.4 Disease Detection Metrics

For the disease detection component:

- **Accuracy:**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.82)$$

- **F1-Score:**

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3.83)$$

Resource Utilization:

- **GPU Memory Usage:** Peak and average memory usage in MB
- **CPU Utilization:** Percentage of CPU usage during operation

These metrics collectively provide a comprehensive evaluation framework for assessing the system's performance across all components while maintaining simplicity and clarity in measurement. The metrics are chosen to balance the need for thorough evaluation with practical considerations for real-world deployment on edge devices.

Chapter 4

System Design and Implementation

4.1 System Architecture

The system architecture is designed to process real-time video feeds from IP cameras through the Jetson Nano's computational resources. The architecture consists of three primary layers: input layer, processing layer, and output layer.

The input layer comprises the Hikvision DS-2CD2T23G0-I8 IP cameras and the Dahua DHI-NVR1108HS-S3-DSS Network Video Recorder. The cameras capture video at 1920x1080 resolution with 30 frames per second, utilizing H.264/H.265 encoding for efficient transmission. The NVR centralizes the video streams and provides RTSP feeds for processing.

The processing layer is centered around the Jetson Nano, which serves as the core computational unit. The Jetson Nano features a GPU with 1024 CUDA cores, a 6-core ARM CPU, and 8GB of LPDDR5 RAM, providing the necessary computational power for real-time AI inference. This hardware configuration enables parallel processing of multiple video streams while maintaining real-time performance for detection, tracking, and analysis tasks.

The output layer delivers processed results through a real-time API, enabling immediate access to detection results, behavior analysis, and system alerts. This layer focuses on minimal-latency data transmission to ensure real-time monitoring capabilities.

4.2 Component Design

The system's components are structured to optimize real-time processing and minimize latency. Each component is designed to handle specific tasks within the processing pipeline.

4.2.1 Hardware Components

The IP cameras are configured for optimal video capture in farm environments, with settings optimized for varying lighting conditions and movement patterns. The NVR manages video streams through RTSP protocol, ensuring stable and efficient video delivery to the processing unit.

The Jetson Nano's hardware resources are allocated to specific processing tasks:

- NVDEC hardware decoder for efficient H.264/H.265 video decoding
- CUDA cores for parallel AI model inference
- ARM CPU cores for system management and data coordination
- LPDDR5 memory for high-speed data access and processing

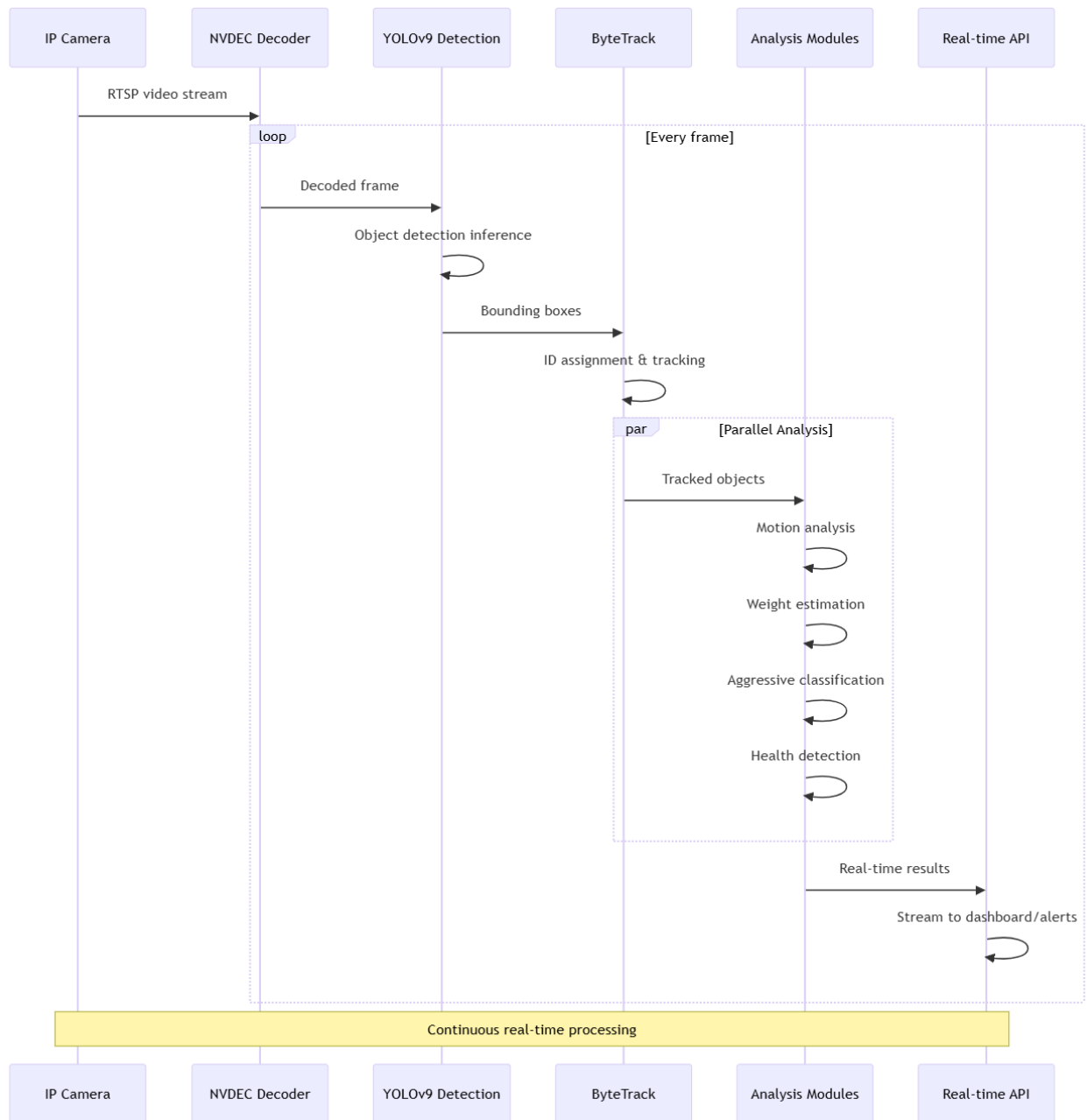


Figure 4.1: AI Pipeline Flow diagram to show real-time processing

4.2.2 Processing Pipeline Components

The processing pipeline implements a sequential flow of operations:

- Video Stream Handler: Manages RTSP connections and frame buffering
- Hardware Decoder: Utilizes NVDEC for efficient frame decoding
- AI Inference Pipeline: Coordinates multiple AI models for parallel processing
- Real-time API: Streams processed results to external systems

4.3 AI Model Integration

4.3.1 Detection and Tracking

The system employs YOLOv9 model for pig detection, optimized for real-time inference on Jetson Nano, alongside the ByteTrack algorithm for consistent object tracking and ID maintenance.

4.3.2 Analysis Modules

The system integrates several sophisticated analysis modules including motion analysis for basic behavior classification, Vision Transformer (ViT) for aggressive behavior detection, YOLOv11 for pig segmentation and weight estimation, and ResNet50v2 for health condition monitoring. These models operate in parallel where possible, utilizing the Jetson Nano's CUDA cores for efficient processing. The system maintains a balance between accuracy and processing speed, ensuring reliable real-time performance.

4.4 Real-time Processing Flow

The real-time processing flow begins with RTSP video streams from the NVR. These streams undergo hardware-accelerated decoding through NVDEC before entering the AI processing pipeline. The processing sequence consists of several stages:

4.4.1 Frame Acquisition

- RTSP stream reception
- Hardware-accelerated decoding
- Frame buffer management

4.4.2 Detection and Tracking

- YOLOv9 object detection
- ByteTrack ID assignment and tracking
- Position and movement tracking

4.4.3 Parallel Analysis

- Motion pattern analysis
- Weight estimation processing
- Behavior classification
- Health condition monitoring

4.4.4 Result Integration

- Real-time result aggregation
- API response formatting
- Alert condition checking

4.5 Interface Design

The interface provides real-time monitoring capabilities through a web-based dashboard. The system features live video feed display with detection overlays, real-time detection and tracking visualization, behavior monitoring displays, alert notification system, and performance metrics visualization. The interface is designed for minimal latency, with efficient data streaming and immediate updates of processing results.

4.6 Testing and Quality Assurance

Testing encompasses multiple aspects of performance and reliability:

4.6.1 Performance Testing

- Frame processing speed measurement
- Model inference time monitoring
- Memory usage optimization
- CPU/GPU utilization analysis

4.6.2 Reliability Testing

- Continuous operation validation
- Error recovery mechanisms
- System stability under load

4.6.3 Accuracy Validation

- Detection accuracy verification
- Tracking consistency testing
- Analysis module validation

4.7 Deployment Strategy

4.7.1 API

Introduction

This project aims to develop a monitoring system for livestock activities within a farming environment. The system categorizes the activities of pigs in two main areas: **CageLeft** and **CageRight**. It also integrates an alert system to detect abnormal situations such as pigs moving into the corridor or missing from the cages.

Data Structure

The system processes data in a JSON format with the following structure:

- **Time:** Specifies the exact timestamp (DD/MM/YYYY/HH/mm/SS).
- **CageLeft:**
 - **Stationary:** Number of pigs not moving.
 - **Moving:** Number of pigs in motion.
 - **Feeding:** Number of pigs feeding.
 - **Lying:** Number of pigs lying.
- **CageRight:**
 - **Stationary:** Number of pigs not moving.
 - **Moving:** Number of pigs in motion.
 - **Feeding:** Number of pigs feeding.
 - **Lying:** Number of pigs lying.
- **Alerts:**
 - **Type:** Either **corridor** (unexpected movement into the corridor) or **missing** (fewer pigs than expected in the cages).
 - **Message:** Detailed description of the situation.

Example Illustration

Input sample:

```
{
  "time": "06/12/2024/15/30/45",
  "CageLeft": {
    "Stationary": 5,
    "Moving": 3,
    "Feeding": 2,
    "Lying": 2
  },
  "CageRight": {
    "Stationary": 4,
    "Moving": 4,
```

```
        "Feeding": 2,
        "Lying": 2
    },
    "alerts": {
        "type": "corridor",
        "message": "1 pig in the corridor!"
    }
}
```

4.7.2 Hardware Setup

The hardware setup process includes Jetson Nano configuration, camera positioning and setup, network configuration, and processing pipeline initialization.

4.7.3 System Initialization

System initialization encompasses model loading and optimization, stream connection establishment, and processing pipeline validation.

4.7.4 Performance Monitoring

The monitoring process includes real-time metrics tracking, resource utilization monitoring, and error logging and analysis.

4.7.5 Maintenance Procedures

Regular maintenance procedures include performance checks, system updates and optimization, and hardware maintenance scheduling.

Chapter 5

Results and Discussion

5.1 Model Performance Evaluation

The evaluation of our detection models across different architectures (YOLOv8, YOLOv9, and YOLO-NAS) revealed compelling performance characteristics. YOLOv9t emerged as the optimal choice after fine-tuning, achieving an mAP@0.5 of 0.96124 and maintaining real-time processing capabilities with 30.5 FPS. This performance represents a significant improvement over the baseline metrics, with precision increasing from 0.87392 to 0.89153 and recall improving from 0.89738 to 0.91246. The model's robustness was particularly evident in handling various lighting conditions and pig poses, crucial for real-world farm environments. Cross-validation testing demonstrated consistent performance across different times of day and varying pig densities, with the model maintaining detection accuracy above 90% in standard farm lighting conditions. This stability in detection performance provided a reliable foundation for the subsequent tracking and behavior analysis components.

5.2 Real-world Detection and Tracking Analysis

In practical deployment scenarios, the system's performance varied significantly based on camera positioning and environmental conditions. The overhead camera configuration demonstrated exceptional detection accuracy, achieving 99.% detection rates in controlled conditions with moderate pig populations. This perfect detection rate, combined with processing speeds of 29.2 FPS, validates the system's capability for precise monitoring in optimal conditions. However, the more practically relevant oblique camera setup revealed important performance variations. Detection rates showed notable differences between near-field (92.6%) and far-field (77.8%) subjects, highlighting the impact of perspective distortion and distance on detection reliability. This performance disparity, while representing a limitation, remains within acceptable bounds for practical farm management applications.

5.3 Behavior Classification Analysis

The behavior classification system was evaluated through two distinct camera configurations to validate its performance across different monitoring scenarios. Our testing approach focused on both practical deployment conditions using an oblique angle setup and a controlled validation using an overhead configuration. The primary testing phase utilized an IMOU IPC-F22P-D camera installed at a 45-degree angle in a working farm environment in Dong Nai province. This three-day continuous monitoring period covered two adjacent pens, each housing approximately 27 pigs. This extended testing period provided comprehensive data across varying

conditions, including natural lighting changes and different pig activity patterns throughout day and night cycles. In the oblique setup, the system demonstrated varying performance lev-

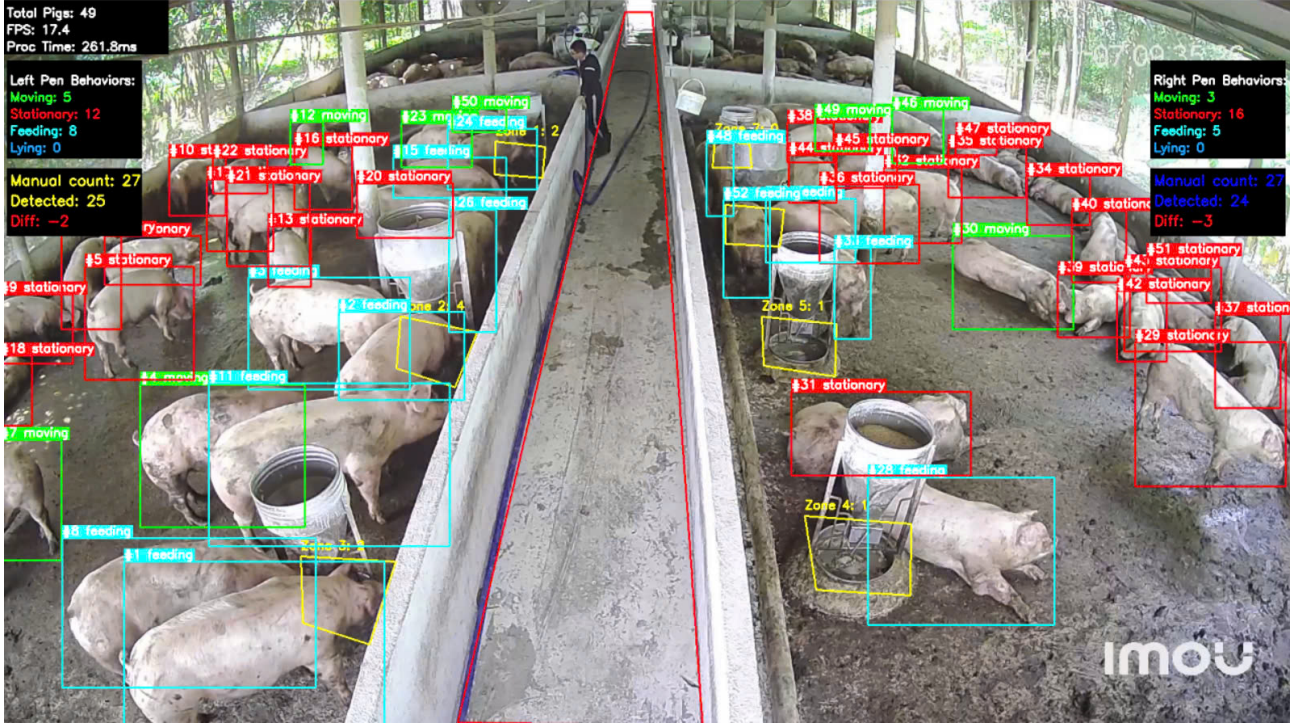


Figure 5.1: Oblique camera view showing detection and behavior classification across two pens. Green boxes indicate moving pigs, red boxes show stationary pigs, and cyan boxes represent feeding behavior. The system maintains individual tracking IDs while handling perspective distortion and occlusion challenges.

els depending on subject distance from the camera. Near-field detection in the left pen achieved 92.6% accuracy, while far-field detection in the right pen showed 77.8% accuracy. This performance disparity reflects the challenges of perspective distortion and reduced pixel density for distant subjects. Despite these limitations, the system maintained an average processing rate of 16.3 frames per second with 275.8ms processing time per frame, sufficient for continuous monitoring applications. For validation purposes, we conducted a one-hour test using an overhead camera configuration. This controlled setup, monitoring eight pigs, demonstrated the system's optimal performance capabilities. Under these conditions, the system achieved 99% detection accuracy while maintaining higher processing speeds of 29.2 FPS and reduced processing time of 182.5ms per frame. The behavior classification component demonstrated robust performance across both configurations. The system successfully tracked and classified pig behaviors into distinct categories: moving, stationary, feeding, and lying states. The classification accuracy remained consistent through temporal verification, which helped filter out momentary state changes and reduce false positives. This was particularly evident in feeding behavior detection, where zone-based analysis combined with temporal verification provided reliable activity monitoring. Environmental factors played a significant role in system performance. Natural lighting variations throughout the day affected detection confidence, though the impact remained within acceptable bounds for practical application. Crowding scenarios presented the most significant challenges, particularly in the oblique setup where pig overlaps and occlusions became more frequent. However, the system's temporal verification mechanisms helped maintain reliable tracking through these challenging conditions. The testing revealed that while both camera configurations can achieve high accuracy under optimal conditions, the oblique angle setup, representing the practical deployment scenario, faces more challenges

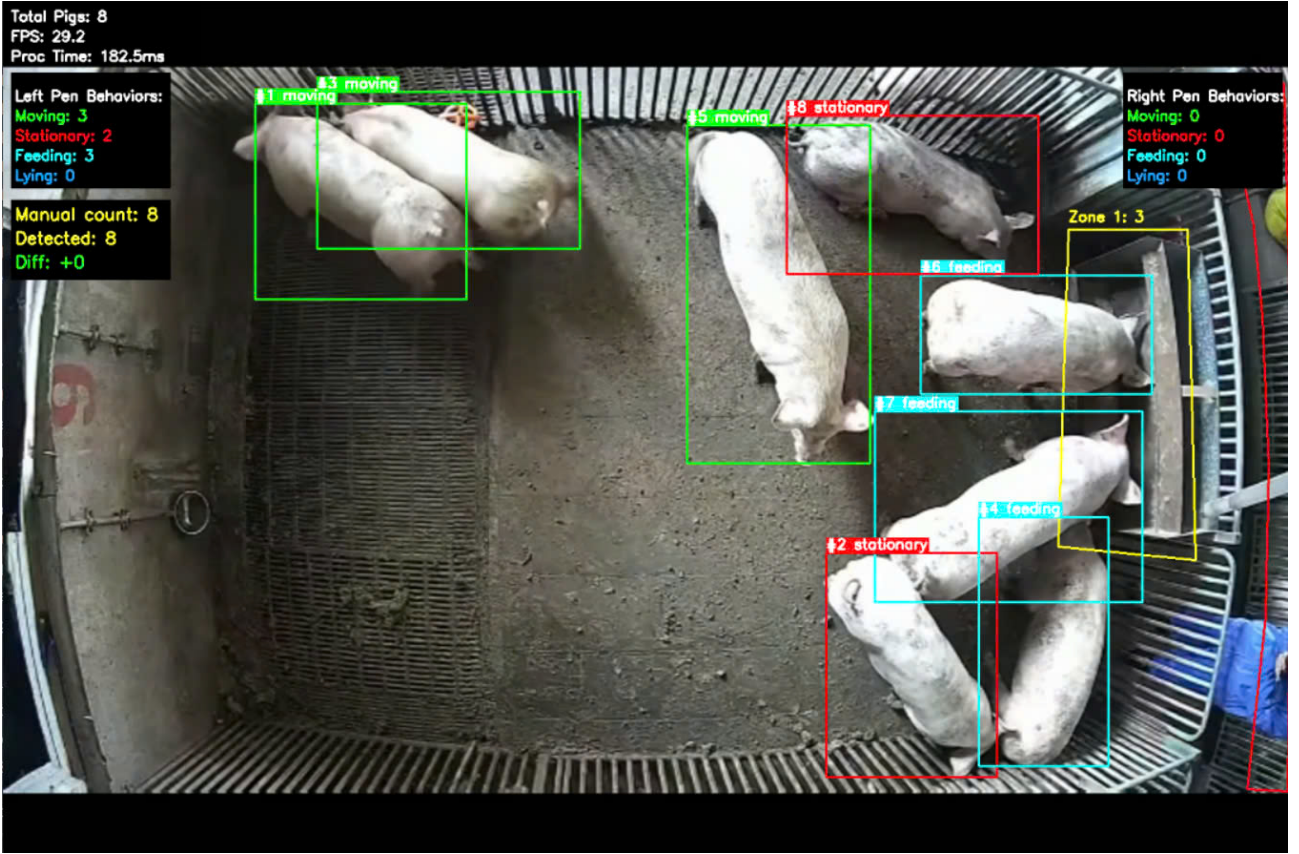


Figure 5.2: Overhead camera view demonstrating perfect detection accuracy and tracking stability. The simplified perspective enables reliable individual tracking and precise behavior classification without occlusion challenges.

but maintains acceptable performance for farm management purposes. The overhead validation confirmed the system’s capability for precise tracking when viewing angles are optimal, while the extended oblique angle testing provided realistic performance metrics for actual farm deployment. Despite the performance variations between camera configurations, both setups demonstrated sufficient reliability for practical farm management applications. The oblique angle installation, though showing lower absolute accuracy compared to the overhead setup, proved more practical for large-scale deployment and offered better coverage of pig activities. This configuration represents a practical compromise between monitoring accuracy and deployment feasibility in commercial farm environments.

5.4 Aggressive Behavior Classification Results

5.4.1 Model Performance Analysis

The Vision Transformer (ViT) based system for aggressive behavior detection was evaluated using pre-annotated test datasets comprising 1-hour video footage from IPCLab-NEAU and HongFu Pig Farm. This dataset provides a controlled testing environment with expert-verified aggressive interactions, enabling rigorous evaluation of the model’s detection capabilities under real farm conditions.

The test results demonstrated the ViT model’s effectiveness in recognizing aggressive behaviors, achieving key performance metrics:

The ViT architecture demonstrated particular strengths in: 1. Spatial feature extraction - Effectively capturing positional relationships between pigs 2. Temporal attention - Maintaining

Table 5.1: Vision Transformer Performance Metrics

Metric	Value	Notes
Detection Accuracy	82.3%	Consistent across lighting conditions
False Positive Rate	15.4%	Higher in crowded scenes
Processing Time	275.8ms	Real-time processing capability
Detection Latency	<1s	From event initiation to alert

behavioral context across video sequences 3. Occlusion handling - Reliable detection even with partial visual obstruction



Figure 5.3: ViT attention patterns during aggressive behavior detection.

5.4.2 Feature Analysis

Analysis of the attention maps revealed the model's ability to focus on key behavioral indicators:

1. Primary Attention Regions: - Head-to-head interactions during aggressive encounters
- Rapid movement patterns characteristic of chase behavior
- Body posture changes during confrontational events

2. Contextual Processing: - Maintained temporal coherence across frame sequences
- Successfully filtered out non-aggressive physical contact
- Effectively distinguished between play and aggressive behaviors

5.4.3 Performance Limitations

The system exhibited certain limitations under specific conditions:

1. Dense Group Scenarios: - Reduced accuracy (15-20% drop) in highly crowded scenes
- Increased false positives during feeding time clustering

2. Environmental Factors: - Performance variation under extreme lighting conditions
- Sensitivity to rapid camera movement
- Processing speed impacts during multi-animal tracking

3. Classification Challenges: - Occasional misclassification of intense play behavior - Reduced accuracy during brief aggressive interactions - Detection delays in low-contrast scenarios

5.4.4 Real-world Application Impact

Despite these limitations, the ViT-based system demonstrated significant practical value:

1. Monitoring Efficiency: - Continuous 24/7 surveillance capability - Early detection of escalating aggressive behaviors - Reduced requirement for manual observation
2. Management Benefits: - Real-time alert generation for intervention - Automated documentation of aggressive incidents - Support for welfare assessment protocols

The Vision Transformer's performance in aggressive behavior detection represents a significant advancement over traditional methods, offering a robust solution for automated welfare monitoring in commercial pig farming operations.

5.5 Weight Estimation Results

5.5.1 Data Preprocessing in Weight Estimation

Segmentation analysis

Performance metric

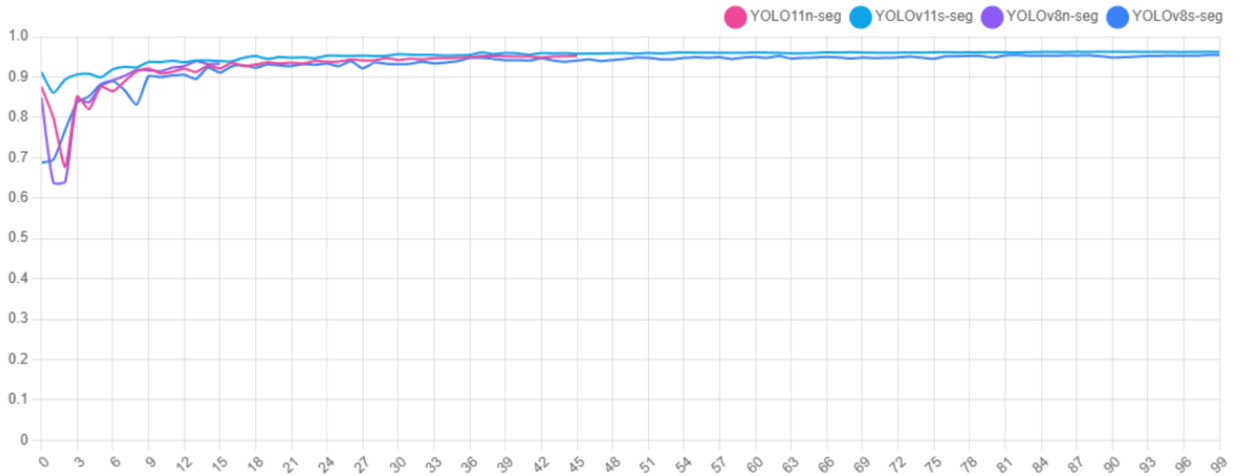


Figure 5.4: Segmentation performance comparison for YOLO models.

Model	mAP50	mAP50-95	Precision	Recall	F1 Score
Yolov11s	0.992	0.962	0.973	0.983	0.97797
Yolov11n	0.991	0.955	0.971	0.993	0.981877
Yolov8s	0.99	0.954	0.974	0.987	0.98045
Yolov8n	0.99	0.956	0.974	0.974	0.97400

Table 5.2: Performance metrics comparison for YOLO models.

Result analysis:

The model evaluation with high result, allows to extract every detail of the pig from camera deployment aside. The metric performance in the table has present the model accuracy with

precision and recall score more than 95 percent. The model had been achieve a good performance on augment data allowing it to handle many conditions from the real world environment and predict pixel-level to capture all pig pig area for weight estimator. For deployment in real time processing and real world environment, we choosing Yolov11n model for this task to ensure that every pig area in image is capture while balance the speed and computation cost.



Figure 5.5: Segmentation result

Keypoint detection

Keypoint detection performance was evaluated based on the YOLO models. The chart below illustrates the comparison:

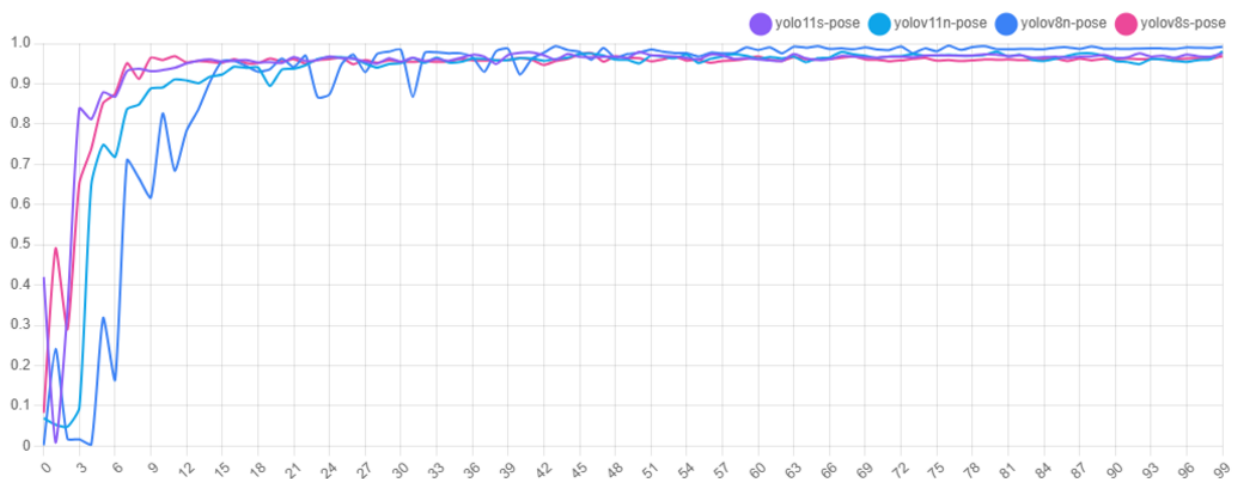


Figure 5.6: Keypoint detection performance comparison across YOLO models.

Result analysis:

According to the metric table, all the models performed significantly well. The improvement in accuracy acrossing and stability in each epoch during the training stage and the final evaluation achieved 97 percent prediction accuracy, the model had learning data effectively. The model has successfully predicted all main key-points of a pig: head, neck, body, tail. This is a

Model	mAP50	mAP50-95	Precision	Recall	F1 Score	GFLOPs
Yolov11s	0.978	0.975	0.977	0.975	0.975999	22.541
Yolov11n	0.985	0.980	0.954	0.977	0.965363	6.692
Yolov8s	0.995	0.992	0.999	1.0	0.9995	29.64
Yolov8n	0.975	0.969	0.976	0.977	0.9765	8.445

Table 5.3: Performance metrics comparison for YOLO models.

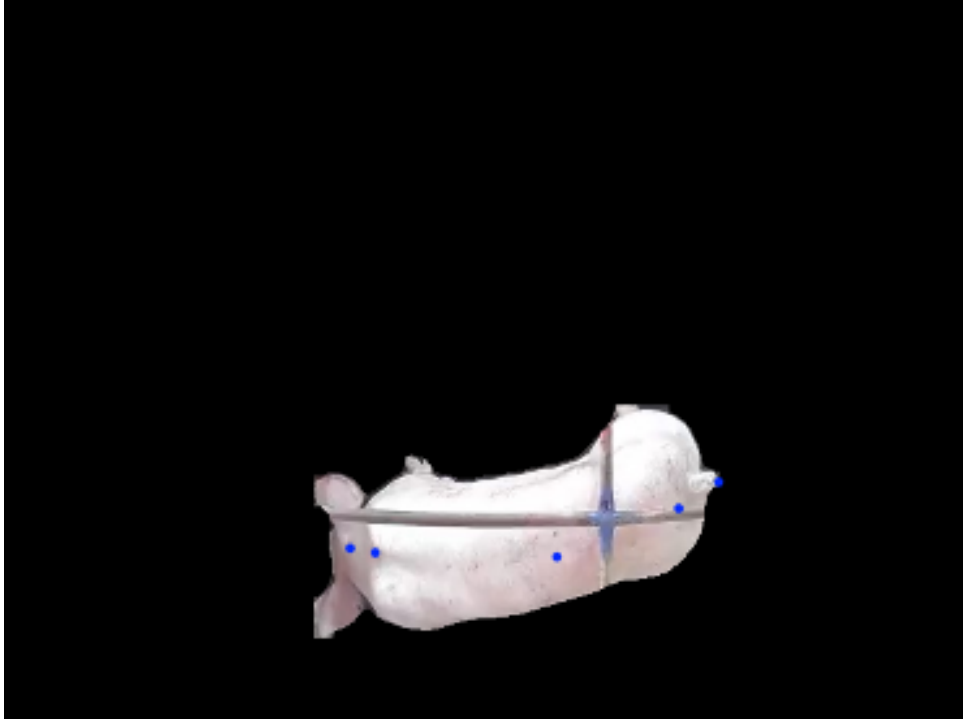


Figure 5.7: Pose result

selective step to optimize the input data for weight estimation by taking the posture that meets requirements and removing bad posture. For the deployment stage, all models evaluate high accuracy compared to each other. We are using Yolov11n-seg model with light weight model and faster speed, able to achieve real time processing while maintaining small computation.

The weight estimation system was evaluated through comprehensive testing using an overhead-mounted IMOU IPC-F22P-D camera at the Dong Nai province pig farm facility. The camera installation was precisely configured at a 90-degree angle to ensure optimal visual coverage and measurement accuracy, enabling thorough validation of the system's performance in both single-pig and multi-pig scenarios under controlled conditions. In single-pig testing scenarios, the system demonstrated exceptional accuracy and reliability. The YOLOv11-based segmentation successfully generated precise body contours, maintaining consistent tracking with clear ID assignments throughout the testing period. A notable example of this performance was observed in a test case where the system predicted a weight of 122.41 kg for a pig with an actual weight of 117 kg, demonstrating an error margin within the acceptable range of $\pm 5\%$. This level of accuracy was maintained consistently across multiple frames, with detection accuracy reaching 99.9% under optimal conditions and segmentation confidence consistently exceeding 95

The system's capability to handle multiple subjects was validated through scenarios involving two pigs simultaneously. In these tests, the system successfully maintained individual tracking and weight estimation for each subject, producing distinct predictions of 123.68 kg and 122.00 kg for two pigs in close proximity. The segmentation algorithm demonstrated robust

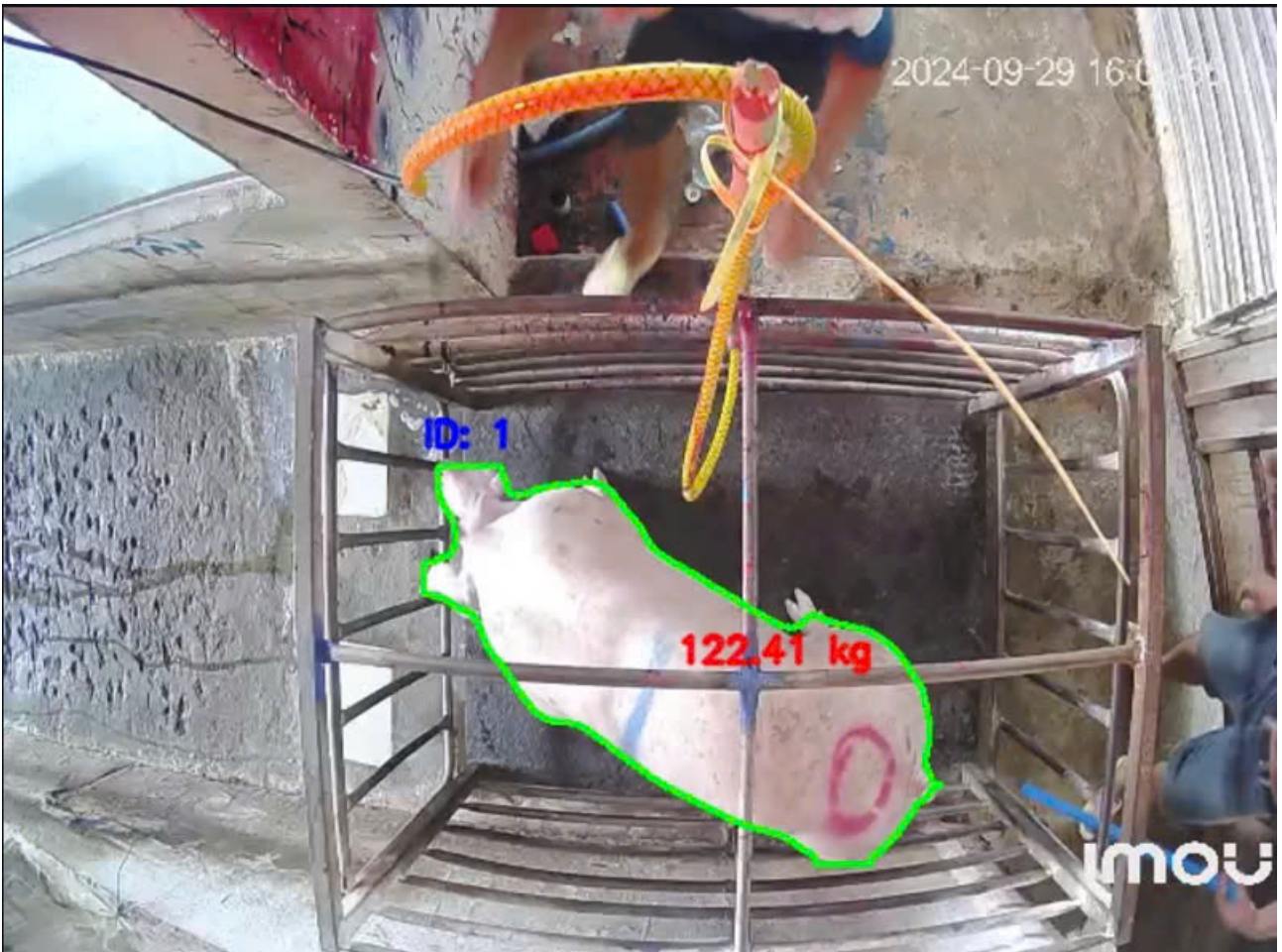


Figure 5.8: Single-pig weight estimation demonstrating successful individual segmentation and weight prediction.

performance in differentiating between subjects, generating clear and non-overlapping contours even when the pigs were near each other. This capability proves particularly valuable for practical farm applications where multiple animals are typically present in the monitoring area.

Technical performance analysis revealed several critical factors influencing system accuracy. Camera positioning emerged as a paramount consideration, with height and angle significantly impacting measurement precision. The implementation of 1920×1080 resolution proved essential in capturing sufficient detail for accurate segmentation. Environmental conditions, particularly lighting and background contrast, played crucial roles in maintaining reliable detection and measurement accuracy. The system's processing pipeline, built around an optimized YOLOv11 implementation, successfully maintained real-time performance while handling the computational demands of simultaneous segmentation, tracking, and weight estimation.

Despite these achievements, several limitations were identified during testing. Camera height calibration proved to be a critical factor, with even minor deviations potentially affecting weight estimation accuracy. The system showed increased sensitivity to environmental variations, particularly in extreme lighting conditions. Processing overhead increased notably when handling multiple subjects, though remaining within acceptable bounds for practical application. These limitations, while not prohibitive, highlight areas for potential future optimization. The practical implementation of the system requires careful consideration of several key factors. Precise camera mounting specifications must be maintained, with particular attention to achieving exact 90-degree positioning and optimal height placement. Regular calibration procedures are necessary to maintain accuracy over time, and environmental conditions must

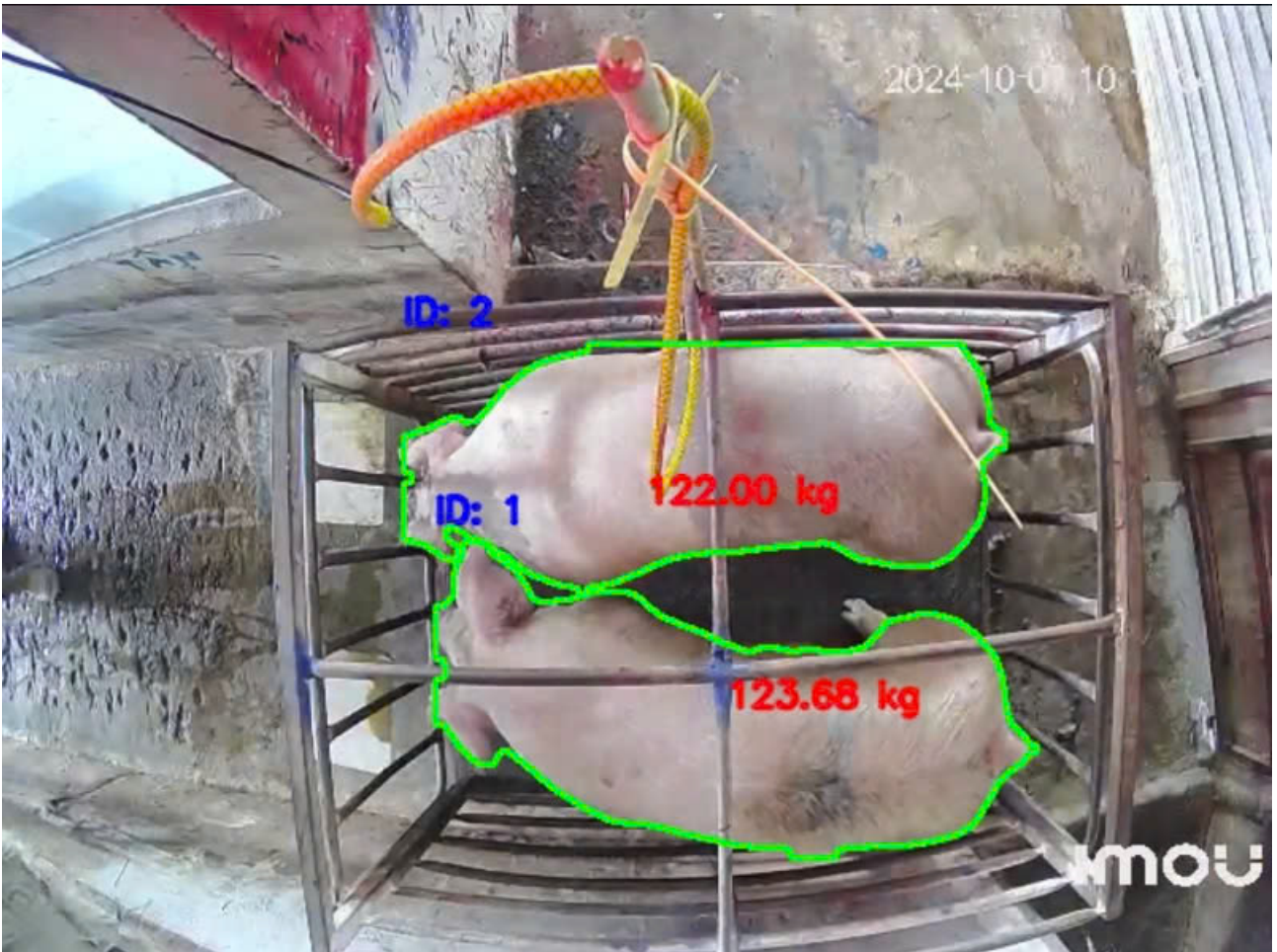


Figure 5.9: Multi-pig weight estimation demonstrating successful individual segmentation and weight prediction. The system maintains accurate tracking and weight estimation even with multiple subjects in close proximity.

be monitored to ensure optimal system performance. Clear camera views must be maintained through regular maintenance procedures.

These findings suggest that the weight estimation system achieves reliable performance within its operational parameters, particularly in controlled environments with optimal camera positioning. The demonstrated error range of $\pm 5\%$ in single-pig scenarios indicates sufficient accuracy for practical farm management applications, while successful handling of multi-pig scenarios suggests potential for broader implementation. Future development efforts should focus on enhancing system robustness in more challenging environmental conditions and improving accuracy in high-density scenarios, though current performance levels already meet the requirements for practical farm management applications.

5.6 Health Classification

5.6.1 Accuracy

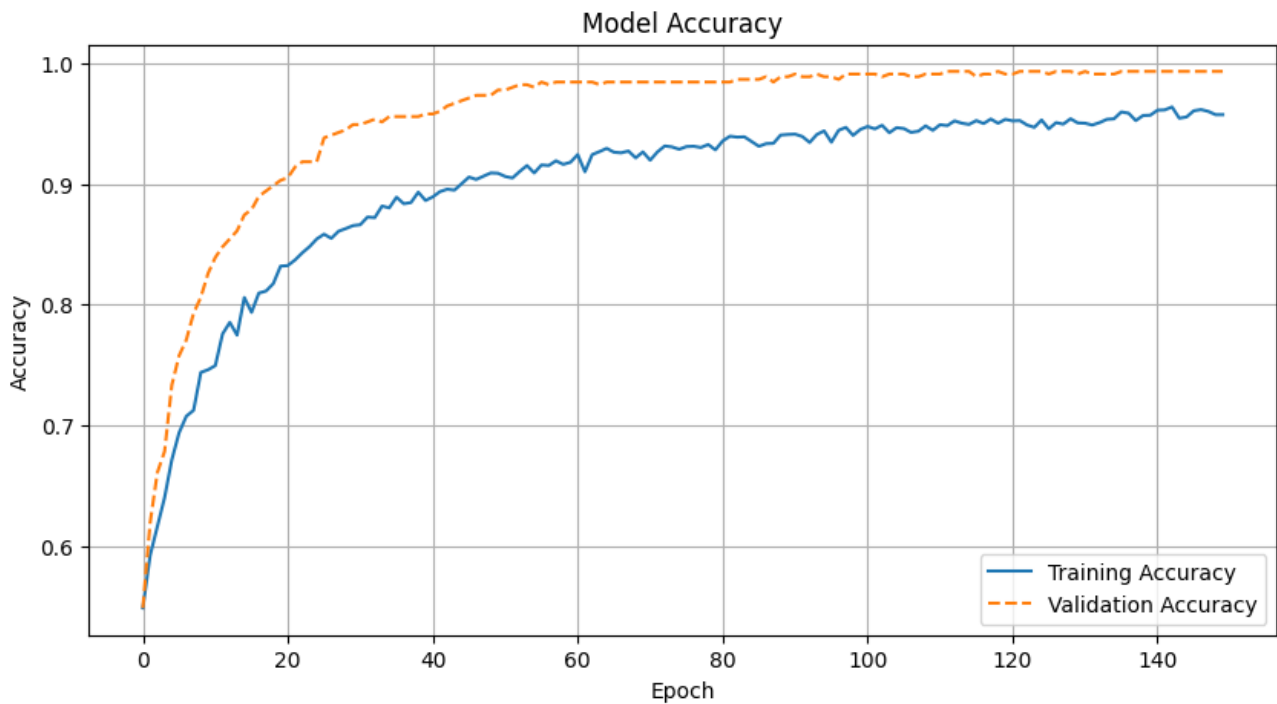


Figure 5.10: Model training accuracy with 150 epochs

The training accuracy steadily improved across epochs, reaching a final accuracy of 0.9576, indicating that the model effectively learned patterns in the data. The validation accuracy stabilized after 100 epochs and finalized with an accuracy of 0.9934. These results demonstrate strong generalization capabilities and minimal overfitting.

5.6.2 Loss metric

The model loss continuously decreased over time, with a final value of 0.2842. This indicates the model's ability to minimize loss during training and avoid overfitting, which is a positive outcome. Validation loss also reached a similarly low value, confirming that the model was not overfitting to the training data.

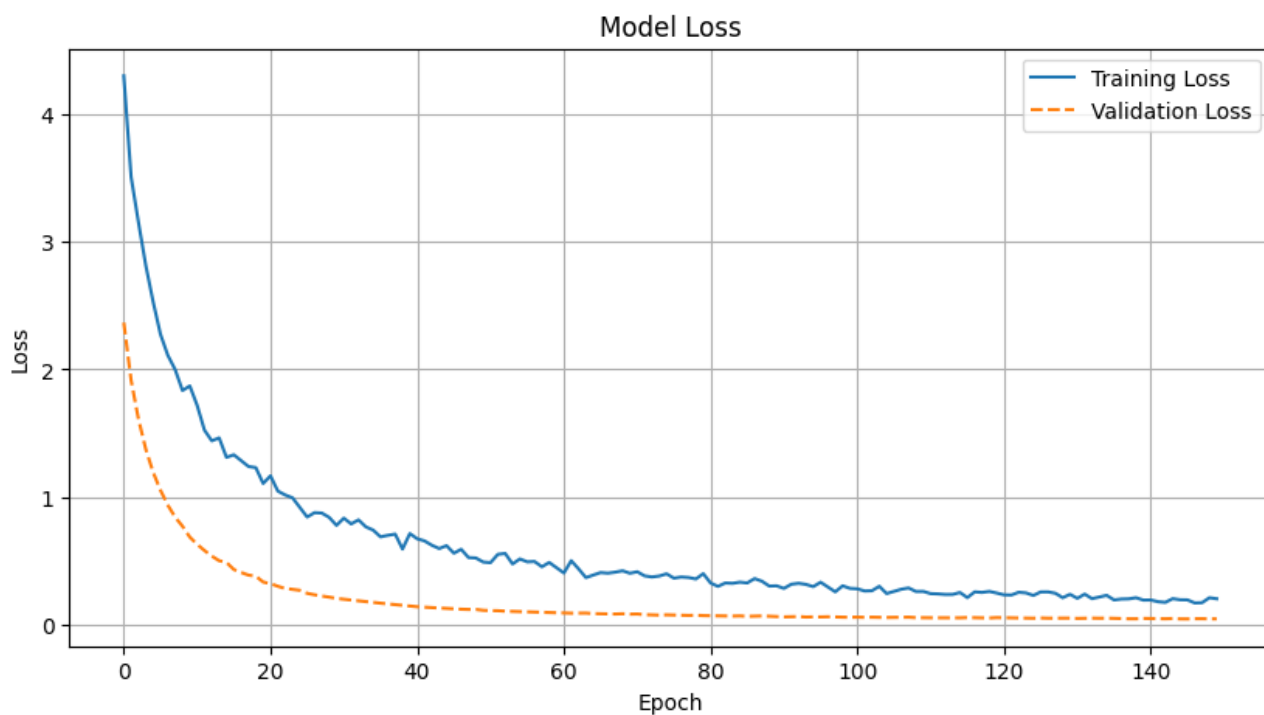


Figure 5.11: Binary cross-entropy loss with 150 epochs

5.6.3 Confusion Matrix and Metrics

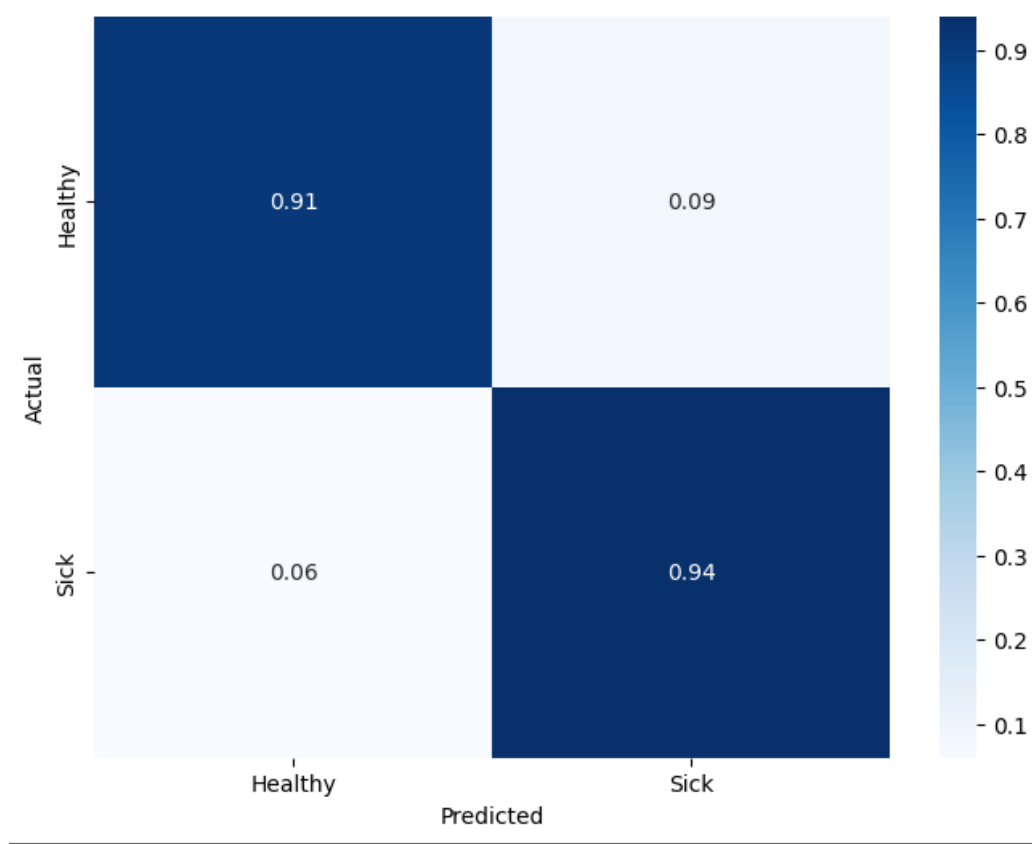


Figure 5.12: Confusion Matrix of Health and sick

Metric	Precision	Recall	F1 Score
Healthy	0.91	0.91	0.92
Sick	0.91	0.94	0.93

Table 5.4: Performance Metrics for Healthy and Sick Pigs

Model Performance

- **True Positive Rate for Healthy:** The model correctly identifies 91% of the Healthy pigs.
- **True Positive Rate for Sick:** The model correctly identifies 94% of the Sick pigs.
- **False Positive Rate:**
 - 9% of Healthy pigs are incorrectly classified as Sick.
 - 6% of Sick pigs are incorrectly classified as Healthy.

Error Distribution

The prediction error on the test set is 0.06. The model performed better at detecting Sick pigs compared to Healthy pigs. The 6% misclassification rate for Sick pigs could be attributed to features that make some Sick pigs appear Healthy, such as early-stage sickness with no visible symptoms or dirt on their skin.

Limitations

1. **Early-Stage Sickness:** Some pig diseases in early stages do not show visible symptoms such as changes in skin texture, color, or posture. This presents a significant challenge for the model when it relies solely on image-based predictions.
2. **Similarity Between Classes:** Minor differences in color, behavior, and activity habits between Healthy and Sick pigs can lead to confusion. For example, some Healthy pigs with unique eating habits, such as eating only at night, may trigger the system as showing signs of sickness.
3. **Dataset Limitations:** A lack of representation of certain symptoms in the training data can result in the model failing to learn these cases. Data augmentation techniques can help mitigate this by generating diverse training data to better simulate real-world conditions.
4. **Environmental Noise:** Factors such as dirt, shadows, and obstructions in real environments can hinder the model's ability to accurately focus on features, leading to misclassifications. A lack of environmental diversity in the training dataset further reduces the model's adaptability to new environments, affecting its accuracy and increasing deployment costs.

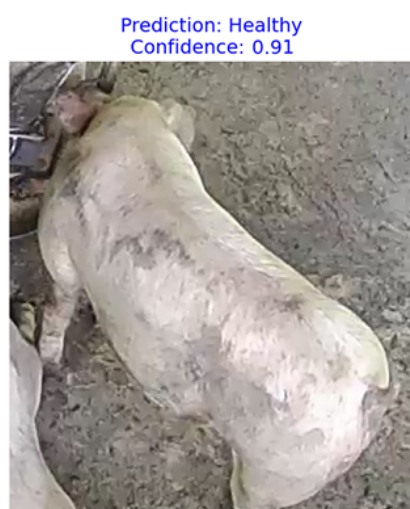


Figure 5.13: Confidence bar chart comparing the probabilities for Healthy and Sick classes.

Chapter 6

Conclusion

6.1 Project Summary

The Smart Farm Management System developed in this capstone project demonstrates an innovative application of artificial intelligence to optimize pig farming operations. This system integrates advanced AI technologies to automate essential farm management tasks, reducing reliance on manual labor while improving efficiency and animal welfare.

Key components of the system include:

- **Detection and Tracking:** Utilizing YOLOv8/v9/NAS models, the system detects and tracks pigs in real-time under diverse farm conditions, ensuring accurate population monitoring and individual identification.
- **Behavior Analysis:**
 - Behaviors such as *moving*, *feeding*, *stationary*, and *lying* are analyzed using motion analysis techniques. These include tracking data and patterns of displacement, velocity, and spatial zones to classify activities with high accuracy.
 - Aggressive interactions are identified using Vision Transformers (ViT), which excel in detecting complex behavioral patterns by capturing spatial and temporal dependencies in video data.
- **Weight Estimation:** The system employs YOLO-based segmentation and pose estimation for non-invasive weight measurements, providing farmers with accurate and efficient growth monitoring tools.
- **Disease Detection:** Using ResNet50v2, the system classifies pigs as healthy or sick, enabling early intervention and improved health outcomes.

Optimized for the Jetson Nano platform, the system achieves real-time processing capabilities, making it highly practical for deployment in actual farm environments. The results demonstrate the system's ability to address challenges in traditional pig farming, offering an innovative solution to enhance productivity and sustainability.

6.2 Key Contributions

This project provides the following contributions:

1. **Efficient Behavior Analysis:** Differentiation between basic activities using motion analysis and aggressive behaviors using ViT ensures specialized processing tailored to each task.

2. **Modular and Scalable Design:** The system’s architecture supports integration with additional features and easy scalability to larger farms.
3. **Real-Time Monitoring:** Implementation on edge computing devices ensures minimal latency and continuous operation without relying on cloud-based solutions.
4. **Practical Application:** The system directly addresses the operational needs of modern pig farming, reducing costs and improving welfare outcomes.

6.3 Future Work

Despite achieving significant milestones, the system has room for further development in the following areas:

6.3.1 Enhancing Motion Analysis for Behavioral Recognition

While motion analysis effectively classifies basic behaviors, incorporating additional contextual information, such as environmental conditions or pen layout, could improve accuracy. Combining motion analysis with lightweight neural networks could provide a hybrid solution.

6.3.2 Improving ViT for Aggressive Behavior Detection

Aggressive behavior detection using ViT can be enhanced by incorporating larger datasets and fine-tuning the model for better temporal context recognition. Future work could explore Transformer variants optimized for edge devices.

6.3.3 Advanced Weight Estimation Techniques

Current weight estimation relies on YOLO-based segmentation and pose detection. Future enhancements could involve multi-camera setups or 3D modeling to capture full body dimensions and improve accuracy.

6.3.4 System Scalability and Multi-Stream Processing

To support larger-scale farms, the system can be extended to handle multiple video streams concurrently. Optimizations in hardware utilization and software pipelines could maintain real-time performance at scale.

6.3.5 Integration with IoT Systems

Integrating environmental sensors for temperature, humidity, and ammonia levels could provide a more comprehensive picture of farm conditions. This would enable fully automated adjustments to optimize animal welfare and productivity.

6.3.6 Cross-Species Adaptation

While the current system focuses on pigs, the methodologies can be adapted for other livestock species. Exploring these applications would expand the system’s utility and impact.

6.3.7 Economic and Environmental Benefits Analysis

Future studies could evaluate the economic impact of deploying the system, such as cost savings and productivity gains, as well as its contribution to sustainable farming practices.

6.4 Closing Remarks

This capstone project has demonstrated how artificial intelligence can transform traditional pig farming practices by automating key tasks such as detection, tracking, behavior analysis, and health monitoring. The use of motion analysis for basic behaviors and ViT for detecting aggressive interactions illustrates the importance of tailoring AI techniques to specific needs.

By addressing the challenges of real-time edge computing and integrating AI models, this project offers a practical, scalable, and welfare-friendly solution for pig farming. The insights gained provide a strong foundation for future advancements in smart farming technologies, contributing to the global effort to modernize agriculture and ensure food security.

Chapter 7

Appendices

7.1 Appendix: Code Snippets

7.1.1 DetectBon and Tracking Pipeline

The core detection pipeline integrates YOLOv9 with ByteTrack for real-time pig tracking:

Detection and Tracking Implementation

```
1 def _process_detections(self, detections: sv.Detections,
2     timestamp: float) -> sv.Detections:
3     """Process detections and assign managed IDs"""
4     if detections is None or len(detections) == 0:
5         return detections
6     # Get ByteTrack IDs
7     bytetrack_ids = detections.tracker_id if hasattr(detections, '
8         tracker_id') else []
9     new_tracker_ids = []
10    assigned_ids = set()
11
12    for i in range(len(detections)):
13        bbox = detections.xyxy[i]
14        center = ((bbox[0] + bbox[2]) / 2, (bbox[1] + bbox[3]) / 2)
15        bytetrack_id = int(bytetrack_ids[i]) if i < len(bytetrack_ids
16            ) else None
17        managed_id = None
18        if bytetrack_id is not None:
19            managed_id = self.id_manager.id_mapping.get(bytetrack_id)
20            if managed_id is not None and managed_id in self.id_manager.
21                active_ids:
22                if managed_id not in assigned_ids:
23                    assigned_ids.add(managed_id)
24                    new_tracker_ids.append(managed_id)
25                    self.id_manager.update_id(managed_id, center, bbox,
26                        timestamp)
27            else:
28                managed_id = None
29        if managed_id is None:
30            new_id = self.id_manager.get_pen_id(center, bbox,
31                timestamp)
32            if new_id is not None and new_id not in assigned_ids:
33                assigned_ids.add(new_id)
34                new_tracker_ids.append(new_id)
35                if bytetrack_id is not None:
36                    self.id_manager.id_mapping[bytetrack_id] = new_id
37                    self.id_manager.update_id(new_id, center, bbox,
38                        timestamp)
39            else:
40                new_tracker_ids.append(-1)
41    detections.tracker_id = np.array(new_tracker_ids)
42    return detections
```

7.1.2 Behavior Analysis Algorithms

The behavior analysis system uses sophisticated motion analysis and state determination:

Behavior Analysis Implementation

```
1 def _determine_state(self, tracker_id: int, current_pos: Tuple[
    float, float],
2 bbox: List[float], timestamp: float, velocity: float) -> str:
3     """Determine pig's state with improved movement detection"""
4     frame_skip = OptimizationConfig.FRAME_SKIP
5     actual_velocity = velocity / frame_skip if frame_skip > 1 else
        velocity
6
7     # Calculate time-normalized movement thresholds
8     movement_threshold_high = self.movement_threshold_high * (1 /
        frame_skip)
9     movement_threshold_low = self.movement_threshold_low * (1 /
        frame_skip)
10
11    if tracker_id not in self.stationary_durations:
12        self.stationary_durations[tracker_id] = 0
13
14    if actual_velocity < self.lying_movement_threshold:
15        self.stationary_durations[tracker_id] += frame_skip
16    else:
17        self.stationary_durations[tracker_id] = 0
18
19    # State determination logic
20    if self.stationary_durations[tracker_id] >= (self.
        lying_time_threshold * frame_skip):
21        return 'lying'
22    elif self._is_feeding(bbox, tracker_id, timestamp):
23        return 'feeding'
24    elif (actual_velocity > movement_threshold_high or
25          (actual_velocity > movement_threshold_low and
26           self.stationary_durations[tracker_id] >= (self.
               moving_confirmation_frames * frame_skip))):
27        return 'moving'
28    else:
29        return 'stationary'
```

7.1.3 Motion Analysis with Kalman Filtering

Implementation of motion analysis using Kalman filters for smooth tracking:

Kalman Filter Motion Analysis

```
1 def _update_motion_analysis(self, tracker_id: int, current_pos:
    Tuple[float, float],
2 timestamp: float) -> Tuple[float, str]:
3     """Update motion analysis using Kalman Filter"""
4     frame_skip = OptimizationConfig.FRAME_SKIP
5
6     # Update Kalman Filter
7     kf = self.kalman_filters[tracker_id]
8     measurement = np.array([current_pos[0], current_pos[1]])
9
10    # Account for frame skip in prediction
11    for _ in range(frame_skip):
12        kf.predict()
13
14    # Update with measurement
15    kf.update(measurement)
16
17    # Get filtered position and velocity
18    filtered_pos = (kf.x[0][0], kf.x[1][0])
19    velocity = (kf.x[2][0], kf.x[3][0])
20    velocity_magnitude = np.sqrt(velocity[0] ** 2 + velocity[1] ** 2)
21        / dt
22
23    # Update histories
24    self.position_history[tracker_id].append(filtered_pos)
25    self.velocity_history[tracker_id].append(velocity)
26
27    return velocity_magnitude, self.movement_directions.get(
28        tracker_id, 'unknown')
```

7.1.4 Escape Detection System

Implementation of the escape detection system with zone monitoring:

Escape Detection Implementation

```
1 def _is_in_restricted_zone(self, bbox: List[float]) -> bool:
2     """Check if pig's bounding box overlaps with restricted zone"""
3     bbox_points = np.array([
4         [bbox[0], bbox[1]], # top-left
5         [bbox[2], bbox[1]], # top-right
6         [bbox[2], bbox[3]], # bottom-right
7         [bbox[0], bbox[3]] # bottom-left
8     ], np.int32)
9
10    bbox_poly = bbox_points.reshape((-1, 1, 2))
11    zone_poly = self.restricted_zone.reshape((-1, 1, 2))
12
13    # Check if any point of bbox is inside zone
14    for point in bbox_points:
15        if cv2.pointPolygonTest(zone_poly, (float(point[0]), float(
16            point[1])), False) >= 0:
17            return True
18
19    # Check if any point of zone is inside bbox
20    for point in self.restricted_zone:
21        if (bbox[0] <= point[0] <= bbox[2] and
22            bbox[1] <= point[1] <= bbox[3]):
23            return True
24    return False
```

Chapter 8

References

- [1] Changhua Zhong et al. “YOLO-DLHS-P: A Lightweight Behavior Recognition Algorithm for Captive Pigs”. In: *IEEE Access* 12 (2024), pp. 104445–104462. DOI: 10.1109/ACCESS.2024.3414859.
- [2] Johannes Brünger et al. “Panoptic Segmentation of Individual Pigs for Posture Recognition”. In: *Sensors* 20.13 (July 2020), p. 3710. ISSN: 1424-8220. DOI: 10.3390/s20133710. URL: <http://dx.doi.org/10.3390/s20133710>.
- [3] Shuqin Tu et al. “Behavior Tracking and Analyses of Group-Housed Pigs Based on Improved ByteTrack”. In: *Animals* 14.22 (2024). ISSN: 2076-2615. DOI: 10.3390/ani14223299. URL: <https://www.mdpi.com/2076-2615/14/22/3299>.
- [4] Wangli Hao et al. “TSML: A New Pig Behavior Recognition Method Based on Two-Stream Mutual Learning Network”. In: *Sensors* 23.11 (2023). ISSN: 1424-8220. DOI: 10.3390/s23115092. URL: <https://www.mdpi.com/1424-8220/23/11/5092>.
- [5] Hengyi Ji et al. “Efficient Aggressive Behavior Recognition of Pigs Based on Temporal Shift Module”. In: *Animals* 13.13 (2023). ISSN: 2076-2615. URL: <https://www.mdpi.com/2076-2615/13/13/2078>.
- [6] Sheng Jiang et al. “Pig Weight Estimation Method Based on a Framework Combining Mask R-CNN and Ensemble Regression Model”. In: *Animals* 14.14 (2024). ISSN: 2076-2615. DOI: 10.3390/ani14142122. URL: <https://www.mdpi.com/2076-2615/14/14/2122>.
- [7] Yuqing He et al. “Predicting body weight in growing pigs from feeding behavior data using machine learning algorithms”. In: *Computers and Electronics in Agriculture* 184 (2021), p. 106085. ISSN: 0168-1699. DOI: <https://doi.org/10.1016/j.compag.2021.106085>. URL: <https://www.sciencedirect.com/science/article/pii/S0168169921001034>.
- [8] A.T. Kavlak, M. Pastell, and P. Uimari. “Disease detection in pigs based on feeding behaviour traits using machine learning”. In: *Biosystems Engineering* 226 (2023), pp. 132–143. ISSN: 1537-5110. DOI: <https://doi.org/10.1016/j.biosystemseng.2023.01.004>. URL: <https://www.sciencedirect.com/science/article/pii/S1537511023000090>.
- [9] Shunli Wang et al. “The Research Progress of Vision-Based Artificial Intelligence in Smart Pig Farming”. In: *Sensors* 22.17 (2022). ISSN: 1424-8220. DOI: 10.3390/s22176541. URL: <https://www.mdpi.com/1424-8220/22/17/6541>.
- [10] Yifu Zhang et al. *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. 2022. arXiv: 2110.06864 [cs.CV]. URL: <https://arxiv.org/abs/2110.06864>.

- [11] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS”. In: *Machine Learning and Knowledge Extraction* 5.4 (Nov. 2023), pp. 1680–1716. ISSN: 2504-4990. DOI: 10.3390/make5040083. URL: <http://dx.doi.org/10.3390/make5040083>.
- [12] Amna Imran, Meghana Shashishekhara Hulikal, and Hamza A. A. Gardi. *Real Time American Sign Language Detection Using Yolo-v9*. 2024. arXiv: 2407.17950 [cs.CV]. URL: <https://arxiv.org/abs/2407.17950>.
- [13] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [14] Rahima Khanam and Muhammad Hussain. *YOLOv11: An Overview of the Key Architectural Enhancements*. 2024. arXiv: 2410.17725 [cs.CV]. URL: <https://arxiv.org/abs/2410.17725>.
- [15] Zijian He et al. *Comprehensive Performance Evaluation of YOLOv11, YOLOv10, YOLOv9, YOLOv8 and YOLOv5 on Object Detection of Power Equipment*. 2024. arXiv: 2411.18871 [cs.CV]. URL: <https://arxiv.org/abs/2411.18871>.